

# Универсальное пособие

## Алгоритмы и структуры данных

---

Билеты 1–24 · 23 страницы · 23 интерактивные демонстрации

Автоматический компактный сборник: весь текст тем (включая раскрытые спойлеры с доказательствами, таблицами и кодом алгоритмов) и визуализации. Навигация приложения и код панели Python-компилятора сюда не входят — интерактивная версия живёт в репозитории и на GitHub Pages.

Дата сборки: 2026-09-15

### Как устроено пособие

- Страница = один или несколько билетов: текст, аналогии, доказательства и код в спойлерах.
- У каждой страницы есть интерактивная демонстрация и панель Python: значения переменных (`i`, `v`, `dist`, `comp`, `color`, `path`, `ops`...) перерисовывают её напрямую.
- В этом PDF демонстрации показаны снимками (когда они отрендерены в CI) и описанием: какие вкладки есть и какие переменные кода ими управляют.
- Демонстрация и синхронизация с компилятором есть у каждой страницы пособия.

# Содержание

---

- 01. Введение. Базовая база: куча, очередь, бинарный поиск, AVL, DFS/BFS
- 02. 1. Дерево отрезков с операциями на отрезках · билет 1
- 03. 2. Двумерная разреженная матрица (Sparse Table) · билет 2
- 04. 3. Декартово дерево (Treap) · билет 3
- 05. 4. Splay-дерево · билет 4
- 06. 5. Двумерная матрица префиксных сумм · билет 5
- 07. 6. Графы. Представление графов, DFS, BFS · билет 6
- 08. 7. Графы. Планарные. Покраска · билет 7
- 09. 8. Графы. Компоненты связности · билет 8
- 10. 9. Графы. Топологическая сортировка · билет 9
- 11. 10. Графы. Компоненты сильной связности (SCC) · билет 10
- 12. 11. Графы. Мосты · билет 11
- 13. 12. Графы. Точки сочленения · билет 12
- 14. 13. Графы. Эйлеров цикл · билет 13
- 15. 14. Графы. Поиск кратчайшего пути. Дейкстра · билет 14
- 16. 15. Графы. Поиск кратчайшего пути. Форд-Беллман · билет 15
- 17. 16. Графы. Поиск кратчайшего пути. Флойд · билет 16
- 18. 17. Графы. Поиск кратчайшего пути. Ускорения · билет 17
- 19. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка · билеты 18, 19, 20
- 20. 21. Алгоритм Кнута-Морриса-Пратта (KMP) · билет 21
- 21. 22. Z-функция — Сдвиг и наложение · билет 22
- 22. 23. Алгоритм Ахо-Корасик · билет 23
- 23. 24. Классы сложности, сведение задач · билет 24

# Введение. Базовая база: куча, очередь, бинпоиск, AVL, DFS/BFS

Введение

## Базовая база

Базовая база — шесть кирпичиков, без которых остальные билеты не читаются: дерево как массив, куча, очередь и стек, бинарный поиск, встроенная сортировка и AVL-дерево. Кодим по минимуму: там, где за нас уже написал Python, показываем одну строку и её сложность.

### 1. Дерево как массив: дети $2n$ и $2n+1$

Дерево не обязано жить указателями. Если занумеровать вершины сверху вниз и слева направо с единицы, то родственников выдаёт простая арифметика: у вершины  $n$  дети сидят в ячейках  $2n$  и  $2n+1$ , а родитель — в  $n // 2$ . Никаких объектов и ссылок: один список, который дружит с кэшем процессора.

```
tree = [None, 40, 20, 60, 10, 30, 50, 70] # tree[0] не используем, нумерация с 1
#      корень n=1
#      дети 40: 2*1=2 → 20   и   2*1+1=3 → 60
#      дети 20: 2*2=4 → 10   и   2*2+1=5 → 30
#      родитель 30: 5 // 2 = 2 → 20
```

Цена: пустые места в массиве, если дерево перекошено. Поэтому так хранят почти полные деревья — кучу (ниже) и дерево отрезков из билета 1. В нумерации с нуля формулы сдвигаются: дети  $2n+1$  и  $2n+2$ , родитель  $(n-1)//2$ .

► Почему именно  $2n$  и  $2n+1$  (спойлер)

На уровне  $k$  полного дерева ровно  $2^k$  вершин, а перед ним суммарно  $2^k - 1$ . Значит, первый номер уровня  $k$  — это  $2^k$ , и дети вершины  $n$  уезжают ровно на длину своего уровня вперёд:  $2n$  и  $2n+1$ . Та же арифметика держит кучу и кучеобразные структуры везде, включая `heapq`.

### 2. Куча: минимум на вершине за $O(1)$ , всё остальное за $O(\log n)$

Куча (`min-heap`) — почти полное дерево в массиве из пункта 1, у которого каждый родитель  $\leq$  своих детей. Отсюда магия: минимум всегда лежит в `h[0]`, а вставка и снятие минимума стоят  $O(\log n)$  — элемент просачивается вверх или вниз по цепочке родителей/детей. Полностью сортировать кучу не нужно: ей достаточно держать кандидата на вершине.

```
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappush(h, 3)
h[0] # 1 — минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за  $O(\log n)$ 
```

Куча — двигатель Дейкстры (билет 14) и Прима (билет 19): «достань наименьший» там нужен миллионы раз, и список с `min()` за  $O(n)$  это бы утопил.

### 3. Очередь и стек: FIFO и LIFO за $O(1)$

Очередь (FIFO: первый вошёл — первый вышел) и стек (LIFO: последний вошёл — первый вышел) отличаются одной операцией снятия. В Python очередь — это `collections.deque`: оба конца работают за  $O(1)$ . Обычный список вместо очереди — мина: `lst.pop(0)` сдвигает весь хвост и стоит  $O(n)$ .

```
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)
```

Вся разница обходов графа из билета 6 — какая из этих двух структур стоит в цикле: очередь даёт BFS (волнами), стек — DFS (вглубь).

#### 4. Бинарный поиск: $O(\log n)$ по отсортированному

Если массив отсортирован, элемент ищется не перебором, а отбрасыванием половин: сравнил с серединой — понял, в какой половине жить, — и так  $\log_2 n$  раз. Миллион элементов — это 20 сравнений. В Python за это отвечает модуль `bisect`.

```
from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньшего 30
```

Без сортировки бинарный поиск не работает — поэтому следующий кирпичик про неё.

#### 5. Сортировка по умолчанию: `sorted()` и `.sort()`

Писать свой квиксорт на экзамене не нужно: в Python встроен Timsort —  $O(n \log n)$  в худшем случае и  $O(n)$  на почти отсортированных данных. Внутри он ищет уже упорядоченные куски (раны) и аккуратно сливает их, как в сортировке слиянием, попутно пользуясь вставками на коротких участках. Сортировка устойчивая: равные элементы не перемешиваются.

```
sorted([3, 1, 2]) # [1, 2, 3] – новый список,  $O(n \log n)$ 
sorted([(1, "b"), (1, "a")], key=lambda t: t[0]) # равные по ключу сохраняют порядок
```

Что лежит внутри, одной строкой: `runs + merge + insertion`. На вопросах «почему не  $O(n^2)$ » отвечают: слияние двух ранов длины  $k$  стоит  $O(k)$ , а уровней слияния  $\log n$ .

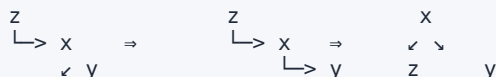
#### 6. AVL: условие баланса и повороты стрелочками

AVL-дерево — двоичное дерево поиска (слева меньше, справа больше), у которого для каждой вершины выполнено условие баланса: высоты левого и правого поддеревьев отличаются не больше чем на 1, то есть  $|h_L - h_R| \leq 1$ . Разность  $h_L - h_R$  называют фактором баланса; из условия следует высота  $O(\log n)$ , а значит поиск, вставка и удаление за  $O(\log n)$  даже в худшем случае.

Малый поворот (цепочка вправо-вправо стрелочками):



Зигзаг (вправо-влево): сначала малый поворот нижнего изгиба, потом малый поворот верхнего — две перерисовки стрелок:



Поворот — это не «взять сыновей и переставить», а перерисовать три стрелки: поменять родителя у среднего звена и перевесить одно внутреннее поддерево на бывшего верха. После вставки достаточно не более двух поворотов вдоль пути вставки, и условие  $|h_L - h_R| \leq 1$  снова везде верно.

► *Формулы малого поворота вокруг z (спойлер)*

$x = \text{right}(z)$   $\text{right}(z) = \text{left}(x)$  # внутреннее поддерево уезжает к  $z$   $\text{left}(x) = z$  #  $z$  становится левым ребёнком  $x$  высоты пересчитываются только у  $z$  и  $x$  — остальные вершины стрелок не меняли

## 7. DFS и BFS: один цикл, две структуры

Оба обхода графа — это «возьми вершину из тары, отметь посещённой, положи соседей обратно в тару». Вся разница в таре: стек уходит вглубь (DFS), очередь расходится волнами (BFS). Оба работают за  $O(V + E)$ : каждая вершина и каждое ребро обрабатываются один раз.

```
def go(graph, start, tank):          # tank: список-стек или deque-очередь
    seen, tank = {start}, tank([start])
    while tank:
        v = tank.pop() if isinstance(tank, list) else tank.popleft()
        for to in graph[v]:
            if to not in seen:
                seen.add(to); tank.append(to)
    return seen
```

Дальше эта пара расцветает в билеты 6–13: компоненты связности, мосты, топосортировка — всё это надстройки над этими восемью строками.

### Аналогии

Аналогия 1: Схема кинозала

Дерево как массив — схема мест кинозала: ряд и место считаются арифметикой, и никаких указателей-«usher'ов» не нужно. Место  $n$ , дети на  $2n$  и  $2n+1$  — схема сама говорит, где родственники, потому что нумерация идёт сверху вниз и слева направо.

Аналогия 2: Турнирная сетка

Куча — турнирная сетка на вылет: в финале (вершине) сидит победитель, и чтобы его назвать, не нужно переигрывать все матчи. Снял победителя — сетка чинится за  $\log n$  перестановок, и на вершине снова сильнейший из оставшихся.

### Куда идти дальше

База закрыта — теперь ею пользуются остальные билеты. Кратчайшие пути живут в билетах 14–16 (Дейкстра, Форд-Беллман, Флойд), остовные деревья — в билетах 18–20 (Краскал, Прим, Борувка), обходы и связность — в билетах 6–13 (DFS/BFS, Косарайю, Мосты), строки — в билетах 21–23 (КМП, Z-функция, Ахо-Корасик). Начинать удобно с билета 6: там DFS и BFS из пункта 7 работают на живом графе.

## Демонстрация и связь с компилятором

Интерактивный симулятор базовых структур: стек, очередь, куча

Проверь LIFO, FIFO и приоритетную очередь на живых примерах.

### Основной режим

- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

Снимки демонстрации добавляются автоматически при сборке в CI (headless-браузер проходит по всем вкладкам). В локальной сборке без браузера здесь остаётся текстовое описание: что показывает демонстрация и какие переменные кода её перерисовывают.

# Дерево отрезков с операциями на отрезках

## Одной фразой

Строгое правило / Формула:

Каждая вершина  $v$  хранит агрегат своего отрезка  $[l, r]$ :  $tree[v] = merge(tree[2v], tree[2v+1])$ , где  $m = (l + r) / 2$ . Запрос и обновление —  $O(\log n)$ , память —  $4n$ .  
Массив из  $n$  чисел накрывают двоичным деревом: корень отвечает за весь отрезок  $[0, n-1]$ , каждый внутренний узел делит свой отрезок пополам и отдаёт половины детям  $2v$  и  $2v+1$ , а листья — это одиночные элементы массива. В узле лежит не «кусочек массива», а ответ на нём: сумма, минимум, максимум, НОД, количество единиц, И/ИЛИ — любая операция, которая умеет склеивать два соседних ответа в один (merge).  
Отсюда всё остальное. Запрос «сколько на отрезке  $[L, R]$ » спускается от корня и на каждом уровне встречается один из трёх случаев: отрезок узла целиком внутри запроса — забираем  $tree[v]$  и не спускаемся; совсем не пересекается — возвращаем нейтральный элемент (0 для суммы,  $+\infty$  для минимума); пересекается частично — идём в обоих детей и склеиваем. Частичных пересечений на уровне бывает не больше двух, поэтому посещаем  $O(\log n)$  вершин. Обновление одного элемента — спуск к листу и пересчёт всех предков на пути обратно (ровно  $height + 1$  узлов).

Высота

$\lceil \log_2 n \rceil + 1$

Запрос / обновление

$O(\log n)$

Память (рекурсивное)

$4n$

Аналогия 1: Корпоративная премия (lazy)

Босс выписывает премию всем 10 000 сотрудников. Вместо 10 000 писем он пишет начальникам департаментов: «внутри всем +5 %». Начальник не пересчитывает ведомость сразу — он кладёт бумажку в ящик ( $lazy[v]$ ) и живёт дальше. Пересчёт случится ровно тогда, когда кто-то снизу придёт спрашивать свой баланс. Это и есть отложенное обновление: «+5 ко всему отрезку» стоит  $O(\log n)$ , а не  $O(n)$ .

Аналогия 2: Матрёшки-коробки

Массив — огромная коробка, внутри две поменьше, и так далее до коробочек-элементов. Нужно покрасить половину массива в синий? Не красим каждый элемент — лепим стикер «внутри всё синее» на большую коробку. Внутрь спускаемся только тогда, когда нас спросили про конкретную маленькую коробочку, и по дороге проталкиваем стикеры вниз (push).

Операции на отрезках (массовое обновление)

«Прибавь  $x$  ко всем на  $[L, R]$ », «присвой всем на  $[L, R]$  значение  $c$ », «переверни биты на  $[L, R]$ » — всё это делается одним спуском: на полностью покрытых узлах обновляем  $tree[v]$  сразу (для «+ $x$ » к сумме:  $tree[v] += x \cdot (r - l + 1)$ ), а детям оставляем обещание в  $lazy[v]$ . Главное — порядок: перед любым спуском в детей вызываем  $push(v)$ , иначе дети считают ответ по устаревшим данным.

| Обновление        | Как меняем $tree[v]$         | Как копим $lazy[v]$                  |
|-------------------|------------------------------|--------------------------------------|
| + $x$ на отрезке  | $tree[v] += x \cdot len$     | $lazy[v] += x$                       |
| = $c$ на отрезке  | $tree[v] = c \cdot len$      | $lazy[v] = c$ (перезапись, не сумма) |
| инверсия битов    | $tree[v] = len - tree[v]$    | $lazy[v] ^= 1$                       |
| $\min$ на отрезке | $tree[v] = \min(tree[v], c)$ | $lazy[v] = \min(lazy[v], c)$         |

Если обновлений два разных типа («присвой» и «прибавь»), lazy хранят парой и проталкивают в строгом порядке: сначала присваивание, потом прибавление.

Ты путаешь:

- Дерево отрезков и префиксные суммы (билет 5): префиксы отвечают на запрос за  $O(1)$ , но строятся один раз и умирают при первом обновлении элемента. Дерево отрезков платит  $O(\log n)$  и живёт под обновлениями.
- Дерево отрезков и разреженная таблица (билет 2): sparse table даёт  $O(1)$  на идемпотентные запросы (min, gcd, И) по статическому массиву; сумма — не идемпотентна, там только дерево отрезков.
- $4n$  и  $2n$ : рекурсивное дерево на отрезке  $[0, n-1]$  с произвольным  $n$  требует  $4n$  ячеек (не  $2n$ !) — иначе на  $n = 6$  поймать переполнение. Ровно  $2n$  хватает только у итеративной версии, где листья лежат в  $tree[n \dots 2n-1]$  и  $n$  — степень двойки (или используется схема «снизу вверх» для любого  $n$ ).
- lazy не «применяется к детям сразу»: push нужен ровно перед спуском. Проталкивать лень во всё дерево — это  $O(n)$  и весь смысл теряется.

Сложность — явно: построение  $O(n)$  (не  $O(n \log n)$ : каждый узел посещается один раз), запрос на отрезке  $O(\log n)$ , одиночное обновление  $O(\log n)$ , массовое обновление с lazy  $O(\log n)$ , память  $O(n)$ . На  $n$  запросов —  $O(n \log n)$  против  $O(n^2)$  у наивного пересчёта.

Где живёт в реальности: счётчики и статистика по диапазону дат, RMQ/RSQ в редакторах и базах, сжатие координат + «сколько точек в прямоугольнике», задачи на «отрезок покрасить / перевернуть / присвоить», дерево отрезков по дереву (HLD), персистентные версии. Если запрос звучит как «на отрезке» и данные меняются — это оно.

► Код: построение и запрос суммы (Скрыто)

```
const int MAXN = 100005;
int tree[4 * MAXN];          // 4n — обязательно, 2n не хватит
int a[MAXN];

void build(int v, int l, int r) {
    if (l == r) { tree[v] = a[l]; return; } // лист
    int m = (l + r) / 2;
    build(2 * v, l, m);                  // левая половина
    build(2 * v + 1, m + 1, r);          // правая половина
    tree[v] = tree[2 * v] + tree[2 * v + 1]; // merge детей
}

int sum(int v, int l, int r, int L, int R) { // сумма на [L, R]
    if (r < L || R < l) return 0;           // не пересекается → нейтральный
    if (L <= l && r <= R) return tree[v];  // покрыт целиком → забрали
    int m = (l + r) / 2;
    return sum(2 * v, l, m, L, R)          // частично → в обоих детей
        + sum(2 * v + 1, m + 1, r, L, R);
}

void update(int v, int l, int r, int i, int x) { // a[i] += x
    if (l == r) { tree[v] += x; return; }
    int m = (l + r) / 2;
    if (i <= m) update(2 * v, l, m, i, x);
    else update(2 * v + 1, m + 1, r, i, x);
    tree[v] = tree[2 * v] + tree[2 * v + 1]; // пересчёт предков
}
```

► Код: массовое обновление, lazy propagation (Скрыто)

```
int lazy[4 * MAXN];          // «обещание»: прибавить к детям

void push(int v, int l, int r) {
    if (lazy[v] == 0) return;
    int m = (l + r) / 2;
    tree[2 * v] += lazy[v] * (m - l + 1); // применяем к левому
    lazy[2 * v] += lazy[v];
    tree[2 * v + 1] += lazy[v] * (r - m); // и к правому
}
```

```

    lazy[2 * v + 1] += lazy[v];
    lazy[v] = 0; // обещание выполнено
}

void add(int v, int l, int r, int L, int R, int x) { // +x на [L, R]
    if (r < L || R < l) return;
    if (L <= l && r <= R) { // покрыт целиком
        tree[v] += x * (r - l + 1);
        lazy[v] += x; // детям — обещание
        return;
    }
    push(v, l, r); // ОБЯЗАТЕЛЬНО до спуска
    int m = (l + r) / 2;
    add(2 * v, l, m, L, R, x);
    add(2 * v + 1, m + 1, r, L, R, x);
    tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

Правило push: вызываем его перед уходом в детей в любом месте, где lazy[v] мог накопиться. Забыл push — получил правильный ответ на маленьких тестах и Wrong Answer на больших.

► *Код: итеративное дерево (2n, снизу вверх) (Скрыто)*

```

n = 8 # для удобства — степень двойки
tree = [0] * (2 * n)
for i in range(n): # листья: tree[n + i] = a[i]
    tree[n + i] = a[i]
for v in range(n - 1, 0, -1): # предки снизу вверх
    tree[v] = tree[2 * v] + tree[2 * v + 1]

def query(l, r): # сумма на полуинтервале [l, r)
    res = 0
    l += n; r += n
    while l < r:
        if l & 1: res += tree[l]; l += 1 # l — правый сын → берём и шагаем
        if r & 1: r -= 1; res += tree[r] # r — правый сын → берём его слева
        l //= 2; r //= 2
    return res

```

Итеративная версия короче и быстрее (нет рекурсии), но массовые обновления с lazy в ней заметно хитрее — на экзамене безопаснее показывать рекурсивную.

► *Вопросы, которые любят задавать (Скрыто)*

- Почему 4n, а не 2n? — при n, не являющемся степенью двойки, дерево несбалансировано: номера узлов доходят до 4n (пример: n = 6).
- Сколько вершин посещает запрос? — не больше  $4 \cdot \log_2 n$ : на каждом уровне «частично пересекающихся» узлов максимум два.
- Почему построение  $O(n)$ , а не  $O(n \log n)$ ? — узлов в дереве  $2n - 1$ , каждый посещается ровно один раз.
- Какие операции подходят? — любые ассоциативные с нейтральным элементом (сумма, min, max, gcd, И, ИЛИ, XOR). Для «количество различных на отрезке» обычное дерево отрезков не годится — там идут в offline-подходы.
- Зачем push в запросе, если ничего не обновляли? — на пути могли остаться чужие lazy от прошлых массовых обновлений; push делает данные детей достоверными.

## Демонстрация и связь с компилятором

### Дерево отрезков

Запросы и обновления на отрезке за  $O(\log n)$ .

Учебное пособие

Python

Скачать PDF

Ещё

Встроенное демо

Сценарий: Дерево отрезков

Массив (2-16 элементов, значения 0-1000)

5, 8, 3, 12, 7, 2

Применить

A[0]=5

A[1]=8

A[2]=3

A[3]=12

A[4]=7

A[5]=2

I = 37

Диапазон запроса [L..R]

L: 1

R: 4

Запрос: sum(A[1..4]) = 30

Построить дерево

Запрос: сверху ↓ vs снизу ↑

Почему / Сравнение

Построение дерева (рекурсия)

1/28

8 8 3 12 7 2

[0..5]

[0..2]

[3..5]

[0..1]

[2..2]

[3..4]

[5..5]

```

1 build(v, l, r) {
2   if (l == r) { tree[v] = A[l]; return; }
3   // -----
4   // -----
5   mid = (l + r) >> 1;
6   build(2*v, l, mid);
7   build(2*v+1, mid+1, r);
8   tree[v] = tree[2*v] + tree[2*v+1];
9 }

```

build(1, 0, 5): mid = ((0+5)/2) = 2. Идём в левого ребёнка.

Пуск

Норма

Дерево отрезков

## Дерево отрезков

Шаг демонстрации: Шаг = спуск по вершине  $v$ : либо читаем  $tree[v]$ , либо толкаем lazy к детям  $2 \cdot v$  и  $2 \cdot v + 1$ .

- $n$  — длина исходного массива; листья дерева лежат в  $tree[n \dots 2 \cdot n - 1]$  ( $n = 8$  на демо)
- $i$  — индекс элемента массива; его лист —  $tree[n + i]$  ( $0 \dots n - 1$ )
- $v$  — текущая вершина дерева отрезков ( $1 \dots 2 \cdot n - 1$ )
- $l, r$  — границы отрезка, за который отвечает вершина  $v$  ( $0 \dots n - 1$ )
- $m$  — середина отрезка  $m = (l + r) // 2$  — граница между детьми ( $l \leq m < r$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Двумерная разреженная матрица (Sparse Table)

## Двумерная разреженная таблица

**Суть: заранее посчитать ответы на всех блоках-степенях двойки**

Строгое правило / Формула (1D — основа всего):

$$st[i][j] = op(st[i][j-1], st[i + 2^{(j-1)}][j-1])$$

Запрос  $[l, r]$ :  $k = \lfloor \log_2(r - l + 1) \rfloor$ , ответ =  $op(st[l][k], st[r - 2^k + 1][k])$

Ячейка  $st[i][j]$  хранит ответ на блоке длины  $2^j$ , который начинается в позиции  $i$ . Любой отрезок покрывается двумя такими блоками — даже если они перекрываются. Перекрывание не мешает, когда операция идемпотентна:  $op(x, x) = x$ . Это  $\min$ ,  $\max$ ,  $\gcd$ , побитовые И и ИЛИ.

Двумерный вариант — то, что спрашивают в билете:

$$st[r][c][kx][ky] = op \text{ из четырёх квадрантов } 2^{(kx-1)} \times 2^{(ky-1)}$$

Запрос  $(r1, c1) \dots (r2, c2)$ :  $kx = \lfloor \log_2(r2 - r1 + 1) \rfloor$ ,  $ky = \lfloor \log_2(c2 - c1 + 1) \rfloor$ , ответ =  $op$  из четырёх блоков  $kx \times ky$ , покрывающих прямоугольник

Двумерная таблица — это та же одномерная, применённая дважды: сначала по строкам, потом по столбцам. Отсюда и цена: уровней становится  $\log N \times \log M$ , а не  $\log N$ .

Память

$$O(N \cdot M \cdot \log N \cdot \log M)$$

десятки копий матрицы

Запрос

$$O(1)$$

4 обращения к таблице

Обновление элемента

нет

только полная перестройка

## Аналогии

Аналогия 1: Сетка из вафельных трубочек

Префиксная матрица — сетка, где в каждой клетке лежит «сколько трубочек всего накопилось от угла (0,0) до меня». Хочешь посчитать трубочки в прямоугольнике посередине — вычти лишние полосы слева и сверху и верни дважды отрезанный угол. Вычитание работает, потому что у суммы есть обратная операция: трубочки можно не только докладывать, но и убирать обратно. Поэтому хранить копии не нужно: одна тонкая сетка размеров оригинала.

время: построение  $O(N \cdot M)$ , запрос суммы  $O(1)$

память:  $N \times M$  — как оригинал, тонкая

Почему тонкая: каждая клетка аккумулирует весь прямоугольник от угла (0,0). Работает ТОЛЬКО для суммы: менять элементы нельзя (полный пересчёт), а минимум так не накопишь — обратной операции нет.

Аналогия 2: Торт-пирамидка

Разреженная таблица минимумов — торт, жирный по памяти и по времени построения, зато красивый, как пирамидка: каждый из  $\log N \times \log M$  уровней — почти полная копия матрицы (уровень  $(kx, ky)$  хранит минимумы всех блоков  $2^{kx} \times 2^{ky}$ ), и ответ на запрос собирается из четырёх готовых блоков за  $O(1)$ . Торт пекут только тогда, когда обратной операции нет: крем обратно в торт не запихнёшь — «вычесть» минимум из минимума нельзя. Зато перекрытие блоков не мешает ( $\min(x, x) = x$ ), и цена  $O(1)$  — сто коржей вместо одной матрицы.

время: построение  $O(N \cdot M \cdot \log N \cdot \log M)$ , запрос  $\min O(1)$   
память:  $N \times M \times \log N \times \log M$  — для  $1000 \times 1000 \approx 10 \times 10 = 100$  копий матрицы  
Когда нужен:  $\min$ ,  $\max$ ,  $\gcd$  — операции без обратной (перекрытие блоков не мешает). Менять элементы тоже нельзя. Сумму через такой торт считать НЕЛЬЗЯ: перекрытие посчитает элементы дважды.  
Порядок чтения демонстрации ниже: вкладка 1D (как строится таблица по одному измерению) → 2D: построение (уровни  $k$  из четырёх квадрантов) → 2D: запрос (прямоугольник раскладывается на четыре блока).

**Префиксная против разреженной: кто жирнее**

Эти две структуры путают чаще всего, потому что обе отвечают на запрос о прямоугольнике за  $O(1)$ . Но по памяти они различаются на порядки, и они не конкуренты: они решают разные задачи.

Сравнение: двумерная префиксная сумма и двумерная разреженная таблица

Размер указан для матрицы  $N \times M$ ; у префиксной суммы это ровно размер исходной матрицы, у разреженной таблицы — размер, умноженный на число уровней.

| Свойство                 | Префиксная сумма 2D (билет 5)                          | Разреженная таблица 2D (этот билет)                                    |
|--------------------------|--------------------------------------------------------|------------------------------------------------------------------------|
| Память                   | $N \times M$ — как сама матрица                        | $N \times M \times \log N \times \log M$ — десятки копий               |
| Запрос на прямоугольнике | $O(1)$ : четыре угла, включения-исключения             | $O(1)$ : четыре блока степеней двойки                                  |
| Какие операции           | только сумма: у неё есть обратная операция (вычитание) | $\min$ , $\max$ , $\gcd$ , И, ИЛИ: идемпотентные, перекрытие не мешает |
| Изменить элемент         | нельзя — пересчёт всей матрицы $O(N \cdot M)$          | нельзя — пересчёт $O(N \cdot M \cdot \log N \cdot \log M)$             |
| Когда брать              | «сумма на прямоугольнике», матрица не меняется         | «минимум/максимум на прямоугольнике», запросов очень много             |

Почему префиксная тонкая

Одна матрица того же размера. Сумма накапливается: каждая клетка  $P[i][j]$  уже содержит информацию обо всём прямоугольнике от угла  $(0,0)$  до неё. Любой другой прямоугольник выражается через четыре таких угла — прибавить два, вычесть два. Больше хранить нечего.

Почему разреженная жирная

У минимума обратной операции нет: зная минимум на большом прямоугольнике, минимум на его части не вычислишь. Приходится хранить готовый ответ для каждого размера блока, а размеров  $\log N \times \log M$ . На каждом уровне — почти полная копия матрицы, поэтому память растёт в десятки раз.

Вывод, который стоит сказать на экзамене: сумму нельзя считать через разреженную таблицу — перекрывающиеся блоки посчитают общие клетки дважды, а вычесть их нечем. Минимум нельзя считать через префиксную — у минимума нет обратной операции. Структуры не заменяют друг друга, они закрывают разные запросы.

**Где обычно теряют баллы**

- Сумма через разреженную таблицу. Блоки перекрываются, а вычитать нечем — ответ завышен. Для суммы берите префиксные (билет 5).
- Память.  $N = M = 1000$  даёт  $10 \times 10 \times 10^6 = 10^8$  ячеек. Это предел по памяти на большинстве констестов: сначала считайте ячейки, потом пишите код.
- Один  $\log$  вместо двух. В 2D уровней  $\log N \times \log M$ ; таблица  $st[i][j][k]$  с тремя индексами — это 1D по одному измерению и не решает задачу.

- Выход за границу в запросе. Вторые блоки начинаются в  $r2 - 2^{kx} + 1$  и  $c2 - 2^{ky} + 1$  : если перепутать знак, индекс уедет за матрицу.
- Нужны обновления. Разреженная таблица статична. Если элемент меняется между запросами — это дерево отрезков (билет 1), а не она.

► *Код: построение 1D (тот же, что выполняет панель Python)*

```
a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
st = []
for row in a:
    st.append([row] + [None] * (LOG - 1))    # уровень 0 = сам массив

for j in range(1, LOG):                    # уровень j строится из j-1
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])
```

Запрос:  $k = \log_2(r - l + 1)$ , ответ  $\min(st[l][k], st[r - 2^k + 1][k])$ . Ровно эти переменные (a, n, LOG, st, i, j) читает демонстрация внизу страницы.

► *Код: построение и запрос в 2D (C++-стиль, четыре квадранта)*

```
// построение: st[r][c][kx][ky] из четырёх квадрантов предыдущих уровней
for (int kx = 0; (1 << kx) <= N; ++kx)
    for (int ky = 0; (1 << ky) <= M; ++ky)
        for (int r = 0; r + (1 << kx) - 1 < N; ++r)
            for (int c = 0; c + (1 << ky) - 1 < M; ++c) {
                if (kx == 0 && ky == 0) st[r][c][0][0] = a[r][c];
                else if (ky == 0)      st[r][c][kx][0] = min(st[r][c][kx-1][0], st[r + (1 << (kx-1))][c][kx-1][0]);
                else                    st[r][c][kx][ky] = min(st[r][c][kx][ky-1], st[r][c + (1 << (ky-1))][kx][ky-1]);
            }

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2 - (1 << kx) + 1][C2 - (1 << ky) + 1][kx][ky]);
int minimum = min(ans1, ans2);
```

Четыре блока размера  $2^{kx} \times 2^{ky}$  прижимаются к четырём углам запроса и вместе накрывают его целиком; середина посчитана несколько раз, но для минимума это безвредно.

► *Сколько это памяти в числах*

- $100 \times 100$ : уровни  $7 \times 7 \approx 49$  копий  $\rightarrow$  около 490 000 ячеек вместо 10 000.
- $1000 \times 1000$ : уровни  $10 \times 10 = 100$  копий  $\rightarrow 10^8$  ячеек; по 4 байта это 400 МБ — обычно уже нельзя.
- Префиксная сумма для той же  $1000 \times 1000$  —  $10^6$  ячеек, 4 МБ.

Практический вывод: 2D разреженную таблицу строят, когда матрица небольшая (до нескольких сотен на сторону), а запросов очень много и все они про минимум или максимум.

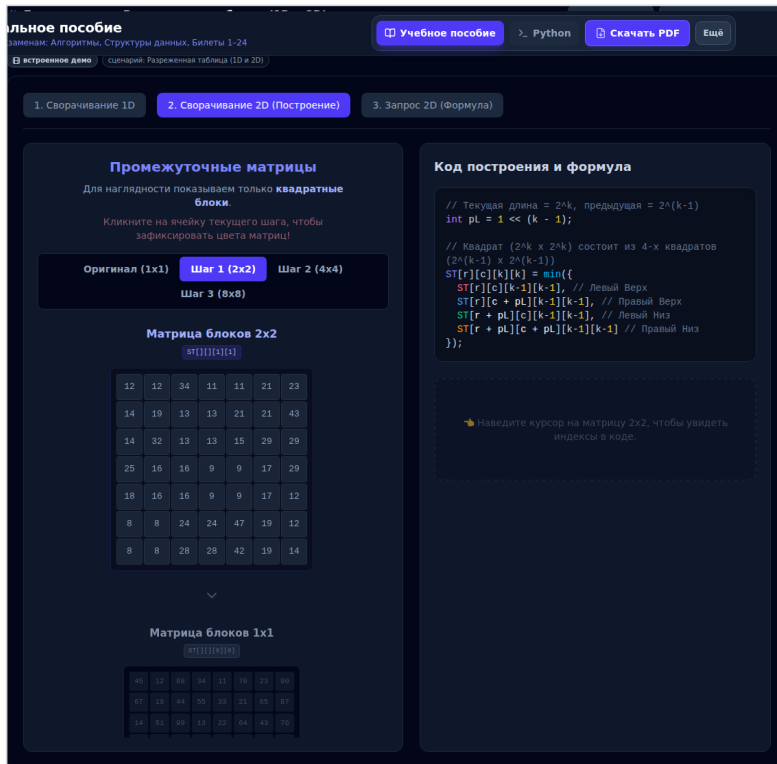
### Читается вместе с этим билетом

- Билет 5, двумерная матрица префиксных сумм — вторая половина этого сравнения: та же скорость запроса, но в десятки раз меньше памяти и только для суммы.
- Билет 1, дерево отрезков — когда между запросами меняются элементы:  $O(\log N)$  на запрос и на обновление, без сотни копий матрицы.
- Билет 16, Флойд — тоже таблица, заполняемая по уровням, но там уровни означают «разрешённые промежуточные вершины», а не размер блока.

## Демонстрация и связь с компилятором

Разреженная таблица (1D и 2D)

Наведите курсор на любую ячейку — подсветится отрезок, за который она отвечает.



Разреженная таблица (1D и 2D)

## Разреженная таблица (1D и 2D)

Шаг демонстрации: Шаг = одна ячейка  $st[i][j] = \min(st[i][j-1], st[i+2^{(j-1)}][j-1])$  при построении таблицы — как в демонстрации слева.

- $a$  — исходный массив, из которого строится таблица (8 чисел на демо)
- $n$  — длина массива  $a$  ( $n = 8$  на демо)
- $LOG$  — число уровней таблицы:  $\log_2 n + 1$  ( $LOG = 4$  на демо)
- $st$  — разреженная таблица:  $st[i][j] = \min$  на блоке длины  $2^j$ , начиная с  $i$  (8 строк  $\times$  4 уровня)
- $i$  — начало блока, за который отвечает ячейка  $st[i][j]$  ( $0 \dots n - 2^j$ )
- $j$  — уровень таблицы: ячейка хранит ответ на блоке длины  $2^j$  ( $0 \dots LOG - 1$ )
- $row$  — очередной элемент  $a$ , из которого вырастает нулевой уровень  $st[i][0] = a[i]$  (значение из  $a$ )
- $L / R$  — границы отрезка запроса — на нём таблица отдаёт минимум за  $O(1)$  ( $1 \dots 6$  на демо)
- $K$  — степень двойки: длина блока  $2^K$ , которым накрывается половина отрезка ( $0 \dots LOG - 1$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Вкладка «2d-build» — Разреженная таблица 2D: построение

Шаг демонстрации: Шаг = новый уровень  $k$ : блоки  $2^k \times 2^k$  дописываются из четырёх квадрантов предыдущего уровня.

- $r, c$  — левый верхний угол блока, для которого считаем минимум ( $0 \dots 8 - 2^k$ )
- $k$  — уровень: блоки  $2^k \times 2^k$  (шаг демонстрации) ( $0 \dots 3$ )
- $st2[(r,c,k)]$  — минимум квадратного блока — склеен из четырёх квадрантов уровня  $k-1$  (4 ключа  $\rightarrow$  значение)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Вкладка «2d-query» — Разреженная таблица 2D: запрос

Шаг демонстрации: Демонстрация кликабельна вручну: выберите прямоугольник и смотрите разложение на блоки.

- $r1, c1, r2, c2$  — углы запрашиваемого прямоугольника (индексы матрицы)
- $kx, ky$  — уровень самого крупного блока, который влезает в высоту/ширину ( $\log_2$  размеров)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Блиц: вопросы по теме

1. Сколько памяти занимает двумерная разреженная таблица для матрицы  $N \times M$ ?

- $N \times M$  — столько же, сколько сама матрица
- $N \times M \times \log N \times \log M$ : на каждом уровне почти полная копия матрицы — верный ответ
- $N \times M \times 4$  — по одной ячейке на каждый угол запроса

Уровней  $\log N$  по строкам и  $\log M$  по столбцам, и на каждом уровне хранится почти полная копия матрицы. Для  $1000 \times 1000$  это примерно  $10 \times 10 = 100$  копий — ровно такова цена ответа за  $O(1)$ . Тонкая здесь как раз префиксная матрица: она занимает  $N \times M$ .

2. Можно ли считать сумму на прямоугольнике через разреженную таблицу?

- Да: минимум и сумма считаются одним приёмом
- Нет: блоки перекрываются, а вычесть перекрытие нечем — верный ответ
- Да, если заранее хранить суммы в нулевом уровне

Два (в 2D — четыре) блока накрывают прямоугольник с перекрытием, и общие клетки попадают в ответ дважды. У суммы есть обратная операция, но применить её к перекрытию нельзя: его размер заранее неизвестен. Сумма на прямоугольнике — это префиксные суммы, билет 5.

3. Какие операции подходят разреженной таблице?

- Любые ассоциативные
- Идемпотентные:  $\min$ ,  $\max$ ,  $\gcd$ , побитовые И и ИЛИ — верный ответ
- Только сумма и произведение

Нужно  $op(x, x) = x$ : тогда дважды накрытая клетка не меняет ответ. Сложение не идемпотентно — именно поэтому оно ломает приём и уходит в префиксные суммы.

4. Между запросами элементы матрицы меняются. Что взять?

- Разреженную таблицу: она перестраивается за  $O(\log N)$
- Префиксную матрицу: достаточно поправить одну клетку
- Дерево отрезков: и таблица, и префиксы статичны — верный ответ

Обе структуры статичны: изменение элемента стоит пересчёта —  $O(N \cdot M \cdot \log N \cdot \log M)$  у разреженной таблицы и  $O(N \cdot M)$  у префиксной. С обновлениями работает дерево отрезков, билет 1.

# Декартово дерево (Treap)

## Tree + Heap = Treap

Строгое правило / Формула:

Treap хранит пары  $(x; y)$ : для ключа  $x$  — бинарное дерево поиска, для приоритета  $y$  — двоичная куча. Пара — точка на декартовой плоскости, отсюда и название. Два базовых метода: Split и Merge.

Построение видно глазами: точки приходят как попало → сортировка по  $y$  (quicksort платит  $\approx n \cdot \log n$  сравнений, counting по маленьким целым  $y$  —  $O(n)$  без сравнений: разница есть, дерево получится одно) → каждая точка вставляется спуском от корня:  $x \leq$  узла — влево, иначе вправо. Порядок рисования линий не важен: точки прибиты, дерево единственно. Весь процесс — в симуляторе под текстом: Собрать (сортировка на выбор, свои точки, чек-лист) или игра «соединить сам» — кликаешь две точки, линия проводится, а проверка мгновенно объясняет, почему нельзя: «у точки уже есть родитель», «получился бы цикл», «равные „!“ не определены». Плюс Split / Merge / Erase по шагам и разборы граблей «Миф  $2k/2k+1$ » и «Поиск  $\neq$  сортировка»: бинарный поиск находит готовый элемент в уже отсортированном массиве за  $O(\log n)$  и ничего не сортирует, поэтому заменить им сортировку по  $y$  при построении дерева нельзя. Или включи Игру: соедини точки сам, а судья-инварианты объяснит каждый запрещённый ход.

малый  $x$  — огромный  $y$

X!Y

! =  $y$  = приоритет = вершина кучи

Аналогия 1: Split — таможня

Split( $t, x_0$ ) — таможня с двумя коридорами: зелёный «все  $x \leq x_0$ », красный «все  $x > x_0$ ». Спуск от корня: узел, который вместе со своим поддеревом целиком проходит по одному коридору, отправляется туда без вскрытия — досматривается только одна ветка. Поэтому split —  $O(h)$ , а не  $O(n)$ .

Аналогия 2: Merge — стыковка колонн

Merge( $a, b$ ) склеивает два дерева при условии «все  $x$  в  $a$  меньше всех  $x$  в  $b$ ». Каждый шаг — одно сравнение: чей  $y$  у корня больше, тот и принимает колонну, а «стыковка» продолжается в одном его поддереве (том, что сохраняет порядок  $x$ ). Итого  $O(h)$  сравнений  $y$  — без перестроек и поворотов.

Ты путаешь:

- «Отсортирую бинарным поиском»: поиск не сортирует; сравнительная сортировка — не быстрее  $n \cdot \log n$ .
- Дети «в массиве по  $2k/2k+1$ » — только у полных деревьев; treap растёт кривым.
- «Убрать можно только самый верхний»: произвольная точка тоже вырезается — её дети сшиваются merge за  $O(h)$ .

Сложность — явно: поиск, вставка, удаление —  $O(\log n)$  в среднем; построение —  $O(n \log n)$  (сортировка +  $n$  спусков). Худший случай — приоритеты вытянулись в цепочку:  $O(n)$  на операцию,  $O(n^2)$  на построение.

Расшифровка X!Y: «!» — это  $y$ : маленький знак держит всё слово — убери, и останется набор букв, как treap без  $y$  останется деревом поиска-палкой.  $X$  — это  $x$ : место по алфавиту. Все стоят по алфавиту, но кричат по «!»: корень самый громкий, родитель всегда громче детей. Равные «!» — выше тот, кто раньше заявил заявку в списке; равные  $x$  — идут влево по уставу.

► Код: [split](#) (Скрыто)

```

pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}

```

► Код: вставка, merge, erase (Скрыто)

```

def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — влево
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок задаёт y
    root = insert(root, key, pri)

```

```

def merge(a, b):
    # все x в a < всех x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key):
    # удалить произвольную точку
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root

```

## Демонстрация и связь с компилятором

Treap: плоскость → дерево → Split/Merge/Erase

7 вкладок: Собрать (сортировка + соединение + автопроверка), Split, Merge, Erase, миф  $2k/2k+1$ , поиск ≠ сортировка, Игра — собери дерево мышкой, судья-инварианты объясняет каждый запрещённый ход.

демонстрация: Treap: плоскость → дерево → Split/Merge/Erase

7 вкладок: Собрать (сортировка + соединение + автопроверка), Split, Merge, Erase, миф  $2k/2k+1$ , поиск ≠ сортировка, Игра — собери дерево мышкой, судья-инварианты объясняет каждый запрещённый ход.

встроенное демо сценарий: Treap: плоскость → дерево → Split/Merge/Erase

## Интерактивный Treap

Пара  $(x; y)$ :  $x$  — ключ,  $y$  — приоритет. Декартова плоскость → дерево

Собрать Split Merge Erase Миф:  $2k/2k+1$  Поиск ≠ сортировка Игра

из КОМПИЛЯТОРА: открой панель Python: переменные `points, mid, key, step` управляют этой демонстрацией

Шаг → шаг 1 из 4

**a: L после split по 6 (все  $x \leq 6$ )**

**b: R после split по 6 (все  $x > 6$ )**

у корней:  $8 > 6 \rightarrow$  корень (45, 8); склеиваем (3, 6) с его левым поддеревом.

Грамотно про «слияние»: у корней сравнивается только приоритет  $y$  — больший становится вершиной, а одно из его поддеревьев (то, что сохраняет порядок  $x$ ) отправляется дальше на склейку. Никакой магии и никаких «капель»:  $O(h)$  сравнений  $y$ .

Treap: плоскость → дерево → Split/Merge/Erase · режим «merge»

демонстрация: Treap: плоскость → дерево → Split/Merge/Erase

7 вкладок: Собрать (сортировка + соединение + автопроверка), Split, Merge, Erase, миф  $2k/2k+1$ , поиск ≠ сортировка, Игра — собери дерево мышкой, судья-инварианты объясняет каждый запрещённый ход.

встроенное демо сценарий: Treap: плоскость → дерево → Split/Merge/Erase

## Интерактивный Treap

Пара  $(x; y)$ :  $x$  — ключ,  $y$  — приоритет. Декартова плоскость → дерево

Собрать Split Merge Erase Миф:  $2k/2k+1$  Поиск ≠ сортировка Игра

из КОМПИЛЯТОРА: открой панель Python: переменные `points, mid, key, step` управляют этой демонстрацией

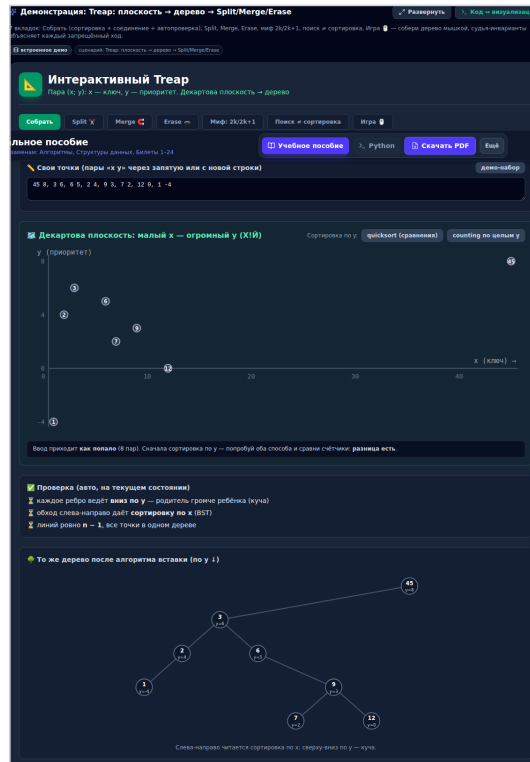
Разрез по  $x = 7$  Шаг → шаг 1 из 5

Исходное дерево: синие  $\rightarrow L (\leq 7)$ , розовые  $\rightarrow R (> 7)$

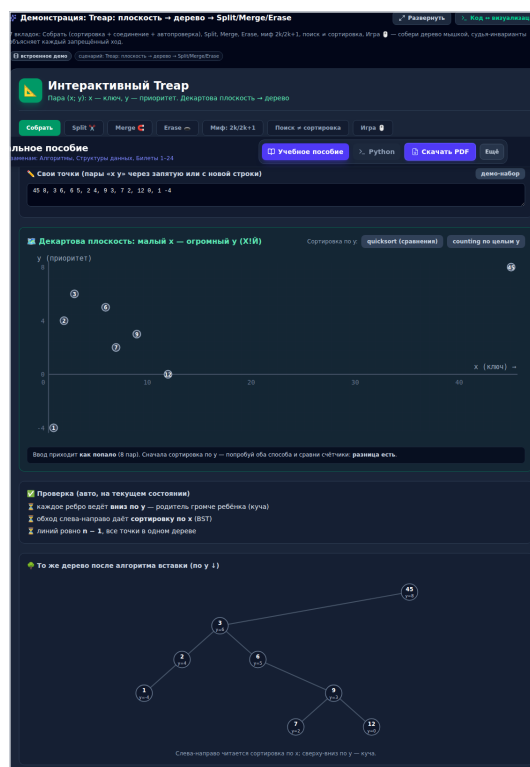
$(7, 2) \leq 7$ : узел и его левое поддерево целиком  $\rightarrow L$ ; дальше режем его правое поддерево.

Как это работает на самом деле: идём сверху с двумя корзинами. Узел, целиком помещающийся в корзину (вместе со своим поддеревом), отдаётся вместе с веткой — дальше режется только одна ветка. Поэтому split —  $O(h)$ , а не  $O(n)$ .

Treap: плоскость → дерево → Split/Merge/Erase · режим «split»



Треар: плоскость → дерево → Split/Merge/Erase · режим «собрать»



Треар: плоскость → дерево → Split/Merge/Erase

## Декартово дерево: build → split → merge → erase

Шаг демонстрации: Шаг = один спуск в split/merge/erase (переменная step) или новая пара (x; y) в наборе points — по нему дерево перестраивается целиком.

- points — набор пар (x; y) — по нему ПЕРЕСТРАИВАЕТСЯ плоскость и само дерево на вкладке «Собрать» (8 пар на демо)
- n — число узлов = число пар (n = 8)
- mid — разрез для вкладки Split: всё с  $x \leq \text{mid}$  уходит в L, остальные в R (2, 3, 6, 7, 9, 12, 45)
- key — какую точку удаляет вкладка Erase (x из набора)

### Декартово дерево: **build** → **split** → **merge** → **erase**

- **step** — номер шага спуска/слияния — листает кадры Split, Merge и Erase (0 ... h)
- **height** — высота дерева: в среднем  $O(\log n)$ , у канонического демо-набора = 5 (1 ... n)
- **root** — корень — узел с максимальным приоритетом  $y$  (узел  $(x; y)$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Splay-дерево

## Одной фразой

Строгое правило / Формула:

Обычное BST без полей балансировки. После каждого обращения — Splay(x): серия поворотов поднимает x в корень. Единственный инструмент — поворот.

Одна операция

может стоить  $O(n)$

гарантии на отдельный запрос нет

Любые  $m$  операций

стоят  $O(m \cdot \log n)$

амортизированное  $O(\log n)$  на операцию

Память

$O(n)$ : ключ + 2 ссылки

легче AVL и красно-чёрного

От какой боли лечит: вставь ключи по возрастанию — и BST выродится в список, поиск станет  $O(n)$ . AVL и красно-чёрные выбирают профилактику: хранят высоту/цвет и чинят дерево сразу. Splay выбирает лечение по факту: ничего не хранит, дереву разрешено временно быть кривым — но каждый пройденный путь заодно распрямляется.

Аналогия 1: Стопка бумаг

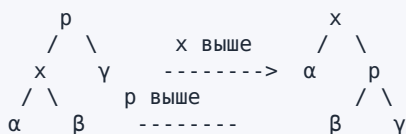
Стопка документов: понадобился лист — вытаскиваешь и, поработав, кладёшь сверху. Сортировку не ведёшь, но через неделю самое нужное само оказывается в верхних сантиметрах, а забытое утонуло. «Взял → положил наверх».

Аналогия 2: Тропинка через газон

Идёшь длинной кривой дорожкой — и после каждого прохода она сама укорачивается примерно вдвое. Первый проход дорогой, зато оплачивает следующие. Раз в год не ходишь — экономии нет; это честная слабость структуры.

Единственный кирпичик — поворот

Поворот меняет местами узел  $x$  и его родителя  $p$ . Одно движение, три перевешенные ссылки. Среднее поддереве  $\beta$  (между  $x$  и  $p$ ) — единственный «груз», который меняет хозяина: его диапазон ключей одинаково влезает и под  $x$ , и под  $p$ .



Обход слева-направо ДО:  $\alpha, x, \beta, p, \gamma$ . ПОСЛЕ:  $\alpha, x, \beta, p, \gamma$  — тот же. Поэтому сломать дерево поиска поворотами невозможно в принципе. В симуляторе ниже каждый поворот разложен на кадры,  $\beta$  подсвечивается в момент «отстёжки», а врезка «Механика одного поворота» держит эту схему перед глазами.

Splay(x): крутим, пока  $x$  не станет корнем

Смотрим на тройку  $x$  (поднимаем),  $p$  (родитель),  $g$  (дед) и выбираем случай:

| Случай  | Как узнать                        | Что крутим ПЕРВЫМ          | Итог                    |
|---------|-----------------------------------|----------------------------|-------------------------|
| Zig     | деда нет, $p$ — корень            | сам $x$ (один поворот)     | $x$ — корень, конец     |
| Zig-Zig | $x$ и $p$ с одной стороны (линия) | $p$ вокруг $g$ , потом $x$ | ветка укоротилась вдвое |

| Случай  | Как узнать                     | Что крутим ПЕРВЫМ            | Итог                       |
|---------|--------------------------------|------------------------------|----------------------------|
| Zig-Zag | х и р с разных сторон (змейка) | х вокруг р, потом х вокруг g | р и g стали двумя детьми х |

И по кругу: определили случай → 1-2 поворота → х ближе к корню → снова смотрим на тройку. Zig случается только последним.

△ Любимый вопрос на экзамене: Zig-Zig — это НЕ «два раза Zig»

бамбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (числа — реальный прогон):  
 наивно (х, потом х): 3 × «два зига» + Zig → высота 8 → 8, бамбук остался бамбуком (перевернутым)  
 Zig-Zig (р, потом х): 3 × Zig-Zig + Zig → высота 8 → 6, х в корне, ветка распалась на куст  
 бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10

Мнемоника: линия — крути СВЕРХУ вниз, змейка — СНИЗУ вверх. Вся разница в коде: rotate(p); rotate(x); против rotate(x); rotate(x); — одна строчка, а без неё амортизации не будет.

Полный прогон: бамбук 8 → 7 → 6 → 5 → 4 → 3 → 2 → 1 (каждый следующий — левый сын), ищем 1. Спуск 8 узлов, затем три Zig-Zig подряд и один Zig (Zig всегда последний): 1 становится корнем, а высота падает с 8 до 6 — линия перестала быть линией. Наивные «два зига» на том же бамбуке дали бы высоту 8: форма осталась бы цепочкой, только перевернутой. Важно не врать себе: на коротком бамбуке из 3-4 узлов высота может и не измениться (4 → 4) — выигрыш гарантирован не на одном запросе, а в сумме. Следующий запрос к 1 — ноль сравнений: мы не только достали ключ, но и починили дерево по пути. Отсюда амортизация: дорогой запрос нельзя пройти много раз подряд — путь кончается, и удешевляется (строго — методом потенциала: потенциал  $\Phi$  = сумма по узлам  $\log_2$  размера их поддеревя, и на каждый поворот  $\Phi$  падает достаточно, чтобы оплатить его; отсюда же теорема о статической оптимальности — splay не хуже идеального статического дерева под твою последовательность запросов).

Ты путаешь:

- Zig-Zig ≠ Zig+Zig: порядок (верхнее ребро первым) сплюсчивает линию вдвое, «два зига» — просто переворот.
- «При повороте между А и В ничего нет» — есть: среднее поддерево β переезжает от низа к верху; в симуляторе оно подсвечивается в кадре «отстёжка».
- «Splay всегда балансирует» — одиночная операция может УВЕЛИЧИТЬ высоту; гарантия только амортизированная.

Сложность — явно: поиск/вставка/удаление — амортизированное  $O(\log n)$ ; одиночная операция — до  $O(n)$ ; n вставок подряд — амортизированное  $O(n \log n)$ .

Все операции — через один splay: find — спуск + splay найденного (нет ключа — splay последнего на пути); insert — BST-вставка + splay нового; erase — splay удаляемого, корень выкинули, у максимума левой половины не будет правого сына — туда вешаем правую. Сильные стороны: простейший код, минимум памяти, самоподстройка под «горячие» ключи, дешёвые split/merge. Слабые: нет гарантии на отдельную операцию, не годится для реального времени, дерево меняется даже при чтении (плохо для многопоточности).

► Код: rotate + splay + find (Скрыто)

```
# узел: { key, left, right, parent }

def rotate_up(x):
    p = x.parent
    g = p.parent
    if p.left is x:
        # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:
        # левый (зеркально)
        p.right = x.left
```

```

    if x.left: x.left.parent = p
    x.left = p
p.parent = x
x.parent = g
if g:
    if g.left is p: g.left = x
    else: g.right = x

def splay(x):
    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None: # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x): # ZIG-ZIG: ВЕРХНЯЯ пара первой
            rotate_up(p)
            rotate_up(x)
        else: # ZIG-ZAG: НИЖНЯЯ пара первой
            rotate_up(x)
            rotate_up(x)
    return x

def find(root, key):
    # поиск: спуск + splay ВСЕГДА
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None

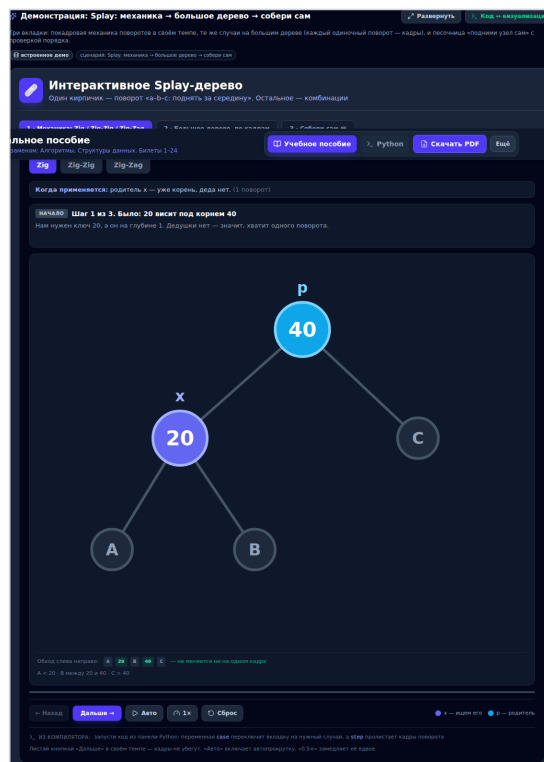
```

Вся разница между случаями — «rotate(p); rotate(x);» против «rotate(x); rotate(x);». Одна строчка — и вся амортизация.

## Демонстрация и связь с компилятором

Splay: механика → большое дерево → собери сам

Три вкладки: покадровая механика поворотов в своём темпе, те же случаи на большом дереве (каждый одиночный поворот — кадры), и песочница «подними узел сам» с проверкой порядка.



Splay: механика → большое дерево → собери сам

### Splay: Zig / Zig-Zig / Zig-Zag на узлах 20-40-60

Шаг демонстрации: Шаг = одно применение случая: переменная case переключает демонстрацию на Zig, Zig-Zig или Zig-Zag, step листает кадры поворота.

- case — какой случай сработал: "zig" / "zig-zig" / "zig-zag" — переключает вкладку демонстрации (строка)
- step — номер применения случая — листает кадры «прицел → поворот» (1 ... 3 на демо)
- xk / x — ключ узла, который поднимаем к корню (в демо это 20 или 40) (20 / 40)
- pk / p — ключ родителя x перед поворотом (20 / 40)
- gk / g — ключ деда — нужен в Zig-Zig и Zig-Zag; 0 (нет деда) означает Zig (60 или 0)
- root — корень дерева после расшейвливания — всегда x (узел)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Двумерная матрица префиксных сумм

**Суть: одна клетка знает сумму всего прямоугольника от угла**

Строгое правило / Формула (1D — основа всего):

$P[0] = 0, P[i] = P[i-1] + a[i-1]$

Сумма  $a[l...r] = P[r+1] - P[l]$

Двумерный вариант — включения-исключения:

$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i-1][j-1]$

Сумма  $(r1,c1)...(r2,c2) = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]$

Здесь  $S[i][j]$  — сумма прямоугольника от угла (0,0) до (i-1, j-1), а нулевая строка и нулевой столбец — рамка из нулей. Именно так считает демонстрация внизу страницы, поэтому в коде везде +1.

Две системы индексации — не путайте их на экзамене

| Запись                                        | Что хранит P                                | Запрос суммы                                       |
|-----------------------------------------------|---------------------------------------------|----------------------------------------------------|
| С рамкой из нулей<br>(демонстрация, код ниже) | $P[i] = a[0] + ... + a[i-1]$ , размер n + 1 | $P[r+1] - P[l]$                                    |
| Без рамки, размер n                           | $P[i] = a[0] + ... + a[i]$ , размер n       | $P[r] - P[l-1]$ , при l = 0 нужно особое ветвление |

Рамка из нулей стоит одной лишней строки и столбца, зато убирает все проверки на «вышли за границу» — поэтому в 2D почти всегда пишут именно так.

Память

$O(N \cdot M)$

столько же, сколько сама матрица

Запрос

$O(1)$

четыре обращения к таблице

Обновление элемента

нет

пересчёт всей таблицы  $O(N \cdot M)$

## Аналогии

Аналогия 1: Сетка из вафельных трубочек

Префиксная матрица — сетка, где в каждой клетке лежит «сколько трубочек всего накопилось от угла (0,0) до меня». Хочешь посчитать трубочки в прямоугольнике посередине — вычти лишние полосы слева и сверху и верни дважды отрезанный угол. Вычитание работает, потому что у суммы есть обратная операция: трубочки можно не только докладывать, но и убирать обратно. Поэтому хранить копии не нужно: одна тонкая сетка размеров оригинала.

время: построение  $O(N \cdot M)$ , запрос суммы  $O(1)$

память:  $N \times M$  — как оригинал, тонкая

Почему тонкая: каждая клетка аккумулирует весь прямоугольник от угла (0,0). Работает ТОЛЬКО для суммы: менять элементы нельзя (полный пересчёт), а минимум так не накопишь — обратной операции нет.

Аналогия 2: Торт-пирамидка

Разреженная таблица минимумов — торт, жирный по памяти и по времени построения, зато красивый, как пирамидка: каждый из  $\log N \times \log M$  уровней — почти полная копия матрицы (уровень  $(kx, ky)$  хранит минимумы всех блоков  $2^{kx} \times 2^{ky}$ ), и ответ на запрос собирается из четырёх готовых блоков за  $O(1)$ . Торт пекут только тогда, когда обратной операции нет: крем обратно в торт не запихнёшь — «вычесть» минимум из минимума нельзя. Зато перекрытие блоков не мешает ( $\min(x, x) = x$ ), и цена  $O(1)$  — сто коржей вместо одной матрицы.

время: построение  $O(N \cdot M \cdot \log N \cdot \log M)$ , запрос  $\min O(1)$

память:  $N \times M \times \log N \times \log M$  — для  $1000 \times 1000 \approx 10 \times 10 = 100$  копий матрицы

Когда нужен:  $\min$ ,  $\max$ ,  $\gcd$  — операции без обратной (перекрытие блоков не мешает). Менять элементы тоже нельзя. Сумму через такой торт считать НЕЛЬЗЯ: перекрытие посчитает элементы дважды.

Порядок чтения демонстрации ниже: сначала 1D (лента с нарастающим итогом, переменные  $a, n, P, i$ ), затем 2D (матрица, рамка из нулей, подсветка четырёх углов запроса — переменные  $i, j, S, r1, c1, r2, c2$ ).

Почему она тонкая и чего она не умеет

Тонкая: одна матрица

Память — ровно  $N \times M$ , как у исходной матрицы (плюс рамка из нулей). Никаких копий и уровней: сумма накапливается, и каждая клетка уже хранит ответ для своего прямоугольника от угла. Всё остальное получается вычитанием.

Не умеет: всё, что нельзя вычесть

Приём работает только потому, что у суммы есть обратная операция. Для минимума, максимума или  $\gcd$  вычитания не существует: зная минимум на большом прямоугольнике, минимум на его части не восстановить. Такие запросы — это билет 2, разреженная таблица, и платит она за это памятью  $N \times M \times \log N \times \log M$ .

Короткое сравнение (полная таблица — в билете 2)

|                     | Префиксная сумма 2D (этот билет) | Разреженная таблица 2D (билет 2)         |
|---------------------|----------------------------------|------------------------------------------|
| Память              | $N \times M$                     | $N \times M \times \log N \times \log M$ |
| Запрос              | $O(1)$                           | $O(1)$                                   |
| Операция            | сумма (обратимая)                | $\min, \max, \gcd$ (идемпотентные)       |
| Обновление элемента | нет                              | нет                                      |

Скорость запроса одинаковая, цена памяти — разная на порядки. Это не две версии одного решения, а решения двух разных задач.

Где обычно теряют баллы

- Смешали индексации. В формуле с рамкой нулей запрос — это  $S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]$ ; записанный без +1 он даёт ответ со сдвигом на клетку.
- Переполнение. Суммы до  $10^9$  элементов по  $10^9$  — это  $10^{18}$ : нужен 64-битный тип. В Python он из коробки, в C++ — long long .
- Минимум через префиксные. Не работает: обратной операции нет. Нужна разреженная таблица (билет 2).
- Обновления между запросами. Префиксная матрица статична; при одном изменённом элементе пересчёт стоит  $O(N \cdot M)$ . С обновлениями — дерево отрезков (билет 1).
- Знак угла. Левый верхний угол вычитается дважды, поэтому его прибавляют один раз. Потерянное  $+ S[r1][c1]$  занижает каждый ответ.

► Код: 1D (тот же, что выполняет панель Python)

```

a = [3, 1, 4, 1, 5, 9, 2, 6] # исходный массив
n = len(a)                  # префиксов будет n + 1
P = [0]                     # P[0] = 0 — пустой префикс
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)

```

Переменные  $a$ ,  $n$ ,  $P$ ,  $i$  — их читает 1D-часть демонстрации: лента подсвечивает, из чего сложилась текущая клетка.

#### ► Код: 2D — построение и запрос прямоугольника

```

A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3], [4, 5, 6, 7]]
n, m = 4, 4

S = [] # рамка из нулей + рабочие клетки
for rr in range(n + 1):
    S.append([0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник включительно
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]
print("итога:", ans)

```

Переменные  $i$ ,  $j$ ,  $S$ ,  $r1$ ,  $c1$ ,  $r2$ ,  $c2$  двигают 2D-вкладку демонстрации: при построении пересчитывается клетка  $S[i][j]$ , при запросе подсвечиваются четыре угла формулы.

#### ► Почему формула верна (доказательство в две строки)

Пусть  $S(i, j)$  — сумма прямоугольника от  $(0,0)$  до  $(i-1, j-1)$ . Тогда прямоугольник от  $(0,0)$  до  $(i-1, j-1)$  распадается на четыре непересекающиеся части: клетка  $A[i-1][j-1]$ , полоса сверху  $S(i-1, j) - S(i-1, j-1)$ , полоса слева  $S(i, j-1) - S(i-1, j-1)$  и их общий угол. Складывая, получаем  $S(i, j) = S(i-1, j) + S(i, j-1) - S(i-1, j-1) + A[i-1][j-1]$ : угол  $S(i-1, j-1)$  вошёл в обе полосы, поэтому вычитается один раз.

Запрос — та же логика наоборот: большой прямоугольник минус две полосы плюс дважды вычитенный угол.

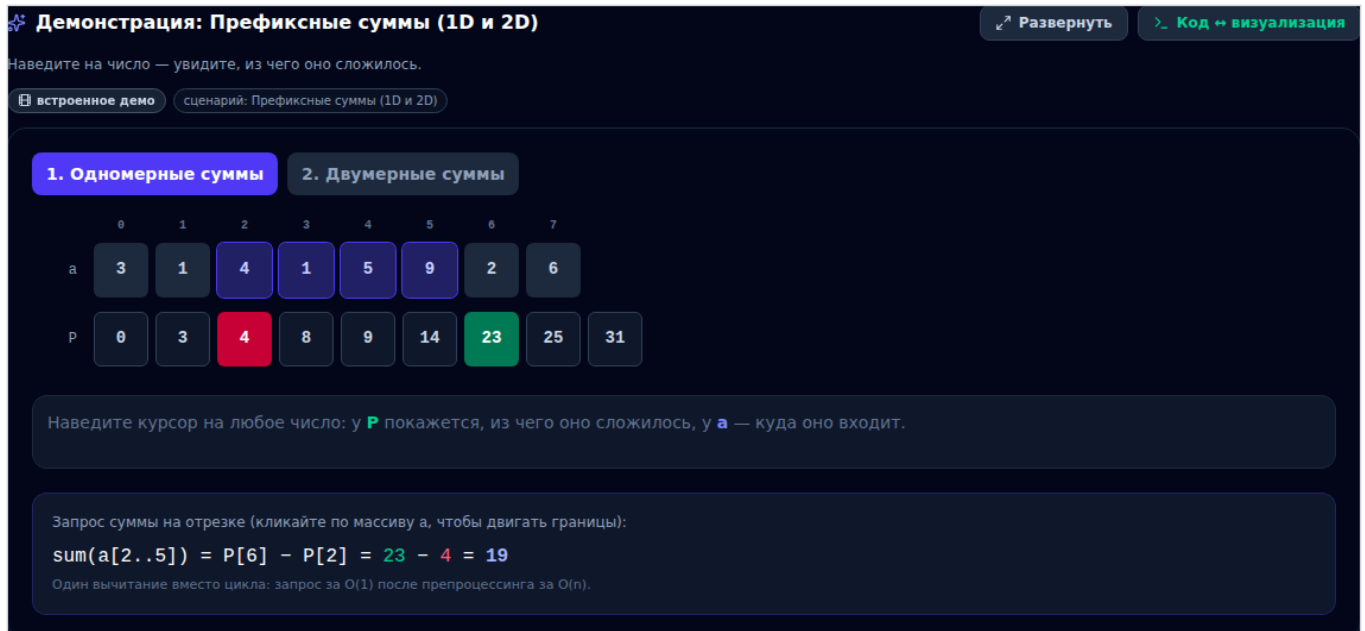
### Читается вместе с этим билетом

- Билет 2, двумерная разреженная таблица — та же скорость запроса, но в десятки раз больше памяти и другие операции ( $\min$ ,  $\max$ ,  $\gcd$ ). Полное сравнение там.
- Билет 1, дерево отрезков — если элемент матрицы меняется между запросами:  $O(\log N \cdot \log M)$  на запрос и на обновление вместо пересчёта всей таблицы.
- Билет 16, Флойд — тоже таблица, заполняемая клетка за клеткой по соседям; отличается тем, что там каждый проход меняет смысл таблицы, а не накапливает сумму.

## Демонстрация и связь с компилятором

### Префиксные суммы (1D и 2D)

Наведите на число — увидите, из чего оно сложилось.



Префиксные суммы (1D и 2D)

## Префиксные суммы (1D и 2D)

Шаг демонстрации: Шаг = заполнение очередного  $P[i] = P[i-1] + a[i-1]$  в 1D-демонстрации слева.

- $a$  — исходный массив 1D-демонстрации (8 чисел на демо)
- $n$  — длина массива  $a$ ; префиксов будет  $n + 1$  ( $n = 8$  на демо)
- $P$  — префиксные суммы:  $P[i] = a[0] + \dots + a[i-1]$ ,  $P[0] = 0$  ( $n + 1$  значение)
- $i$  — граница префикса: на шаге считаем  $P[i] = P[i-1] + a[i-1]$  ( $1 \dots n$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Вкладка «2d» — Префиксные суммы 2D (прямоугольники)

Шаг демонстрации: Демонстрация кликабельна вручну: наведите на ячейку — подсветятся её четыре угла формулы.

- $i, j$  — текущая ячейка таблицы  $S$  (1-индексация) ( $1 \dots n, 1 \dots m$ )
- $S[i][j]$  — сумма прямоугольника  $(0,0)-(i-1,j-1)$  — по двум соседям минус перекрытие (пересчёт на каждом шаге)
- $r1, c1, r2, c2$  — углы запроса; ответ = четыре угла формулы включений-исключений (индексы матрицы)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Блиц: вопросы по теме

1. Сколько памяти занимает двумерная префиксная матрица?

- $N \times M$  — как исходная матрица, плюс рамка из нулей — верный ответ
- $N \times M \times \log N \times \log M$
- $N + M$ : одна строка и один столбец

Сумма накапливается: каждая клетка хранит сумму прямоугольника от угла  $(0,0)$  до себя, поэтому копий и уровней нет. Жирная здесь как раз разреженная таблица (билет 2): у неё  $\log N \times \log M$  уровней по почти полной копии матрицы.

2. Запрос суммы прямоугольника  $(r1,c1) \dots (r2,c2)$  в записи с рамкой из нулей — это...

- $S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]$
- $S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]$  — верный ответ
- $S[r2+1][c2+1] - S[r1][c1]$

С рамкой из нулей  $S[i][j]$  — это сумма прямоугольника до  $(i-1, j-1)$ , поэтому правый нижний угол запроса берётся с  $+1$ . Вариант без  $+1$  даёт ответ со сдвигом на клетку — классическая ошибка на ровном месте.

### 3. Почему левый верхний угол в формуле прибавляется обратно?

- Он входит и в левую, и в верхнюю полосу — его вычли дважды — верный ответ
- Он всегда равен нулю, поэтому прибавление безопасно
- Так компенсируется рамка из нулей

*Отрезая полосу слева и полосу сверху, мы дважды отрезали их пересечение — левый верхний угол. Прибавление возвращает баланс: это и есть формула включений-исключений.*

### 4. Можно ли тем же приёмом считать минимум на прямоугольнике?

- Да, если заменить вычитание на `min`
- Нет: у минимума нет обратной операции, нужна разреженная таблица — верный ответ
- Да, но только для неотрицательных чисел

*Весь приём держится на вычитании. Зная минимум на большом прямоугольнике, минимум на его части не восстановить — обратной операции нет. Для `min`, `max` и `gcd` берут разреженную таблицу (билет 2) и платят за это памятью  $N \times M \times \log N \times \log M$ .*

### 5. Изменили один элемент матрицы. Что станет с префиксной таблицей?

- Поправится одна клетка за  $O(1)$
- Нужен пересчёт  $O(N \cdot M)$ : от этой клетки зависит весь дальний угол — верный ответ
- Ничего не станет: таблица остаётся верной

*Каждая клетка  $S[i][j]$  включает в себя все предыдущие, поэтому одно изменение пересчитывает всю таблицу. Если обновления частые, это задача дерева отрезков (билет 1).*

# Графы. Представление графов, DFS, BFS

## Графы. Представление, DFS, BFS

{/\* База представлений \*/}

### Матрица vs Списки смежности

Сложность хранения и обхода:

Матрица:  $O(V^2)$  памяти. Обход за  $O(V^2)$ . Списки смежности:  $O(V + E)$ . Обход за  $O(V + E)$ .

Аналогия: Матрица = Таблица

Матрица — таблица всех жителей Земли против всех. 99.9% таблицы пусто! Выглядит глупо, но проверка "знает ли Вася Петю" мгновенна.

Аналогия: Списки = Контакты

Списки — записная книжка в телефоне. У каждого записано только те, кого он знает.

Оптимально для почти всех задач.

{/\* DFS vs BFS \*/}

### Два короля обходов: DFS и BFS

♂ DFS (Поиск в глубину) = Обход с возвратом (Backtracking)

Что делает: Поиск в глубину с возвратом (backtracking). Идёт вглубь по неисследованным рёбрам до упора. Когда попадает в тупик — делает шаг назад по стеку вызовов и пробует следующую ветку.

- Где используется:
- Топологическая сортировка (Билет 9)
- Поиск мостов и точек сочленения (Билеты 11, 12)
- Компоненты сильной связности Косарайю (Билет 10)
- Детектирование циклов (через 3 цвета) и проверка связности.

BFS (Поиск в ширину) = Волна

Что делает: Распространение волнами по слоям расстояния. За один шаг открывает всех соседей, затем соседей соседей. Работает через Очередь (FIFO). Гарантирует кратчайший путь по числу рёбер.

- Где используется:
- Кратчайший путь в Невзвешенном графе (лабиринт, граф ходов)
- Проверка двудольности графа (2-раскрашиваемость)
- Алгоритм Ахо-Корасик (построение суффиксных ссылок в боре)
- Двухнаправленный BFS для ускорения поиска в невзвешенных графах (для взвешенных используется двухнаправленная Дейкстра).

► Код обходов C++ (Основа)

```
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}
```

```
// BFS (Очередь FIFO)
queue<int> q;
q.push(start_node);
vis[start_node] = true;

while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

## Демонстрация и связь с компилятором

## DFS и BFS на графе

Демонстрация: DFS и BFS на графе

встроенное демо

сценарий: DFS и BFS на графе

DFS (В глубину)

BFS (В ширину / Волна)

А

В

С

Д

Е

Ф

Г

СТЕК ВЫЗОВОВ

Узел: А ... Топ

▶ Авто

⏮

⏪

⏩

⏭

ЛОГ ВЫПОЛНЕНИЯ [ШАГ 1/20]

Зашли в А. Идем вглубь первой непосещенной вершины!

## DFS и BFS на графе

## DFS и BFS на графе

Шаг демонстрации: Шаг = переход из текущей вершины  $v$  к непосещённому соседу  $to$  (DFS — вглубь, BFS — по слоям).

- $v$  — текущая вершина обхода (вершины демо-графа)
- $to / u$  — очередной сосед вершины  $v$  из списка смежности  $adj[v]$  ( $adj[v]$ )
- $visited$  — множество уже посещённых вершин — гарантия от циклов (пополняется на каждом шаге)
- $order$  — порядок, в котором обход дошёл до вершин (строка под графом)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

Шаг демонстрации: Шаг = переход из текущей вершины  $v$  к непосещённому соседу  $to$  (DFS — вглубь, BFS — по слоям).

- `v` — текущая вершина обхода (вершины демо-графа)
- `to / u` — очередной сосед вершины `v` из списка смежности `adj[v]` (`adj[v]`)
- `visited` — множество уже посещённых вершин — гарантия от циклов (пополняется на каждом шаге)
- `order` — порядок, в котором обход дошёл до вершин (строка под графом)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Графы. Планарные. Покраска

## Планарные графы и раскраска

### 7.1 Формула Эйлера

Формула Эйлера:

$$V - E + F = 2$$

$V$  — вершины

$E$  — рёбра

$F$  — грани (включая внешнюю)

Следствие: Для планарного графа с  $V \geq 3$ :  $E \leq 3V - 6$ . Если рёбер больше — граф гарантированно непланарен. Рёбра будут пересекаться.

$K_5$ : Полный граф на 5 вершинах

У  $K_5$ :  $V=5$ ,  $E=10$ . Проверяем  $E \leq 3V-6$ :  $10 > 9$  — БАХ, непланарен! Все 5 вершин соединены со всеми. На плоскости это звезда с пересечениями. Теорема Куратовского:  $K_5$  и  $K_{3,3}$  — два эталона непланарности. Если внутри графа спрятано подразделение  $K_5$  или  $K_{3,3}$  (то есть эти графы с вершинами, «размноженными» на рёбрах) — плоской раскладки не будет.

$K_{3,3}$ : 3 дома и 3 колодца

Три дома хотят соединиться с тремя колодцами (каждый дом — с каждым колодцем). Нарисовать без пересечений невозможно. Это  $K_{3,3}$ . У него  $V=6$ ,  $E=9$ . Формула  $E \leq 3V-6$  даёт  $9 \leq 12$  — проходит, но он всё равно непланарен! Почему? Потому что у двудольного графа минимальный цикл — 4, а не 3. Отсюда более строгое ограничение:  $E \leq 2V - 4$ .  $9 > 8$  — попался!

#### ► Доказательство непланарности $K_5$ (Скрыто)

Доказательство через формулу Эйлера:

Пусть  $K_5$  планарен.  $V = 5$ ,  $E = 10$ .  $F = E - V + 2 = 10 - 5 + 2 = 7$  граней. Каждая грань ограничена минимум 3 рёбрами (простых циклов длины 1-2 нет). Сумма длин граней =  $2E = 20$ . Отсюда:  $2E \geq 3F \Rightarrow 20 \geq 21$  — ПРОТИВОРЕЧИЕ! Значит,  $K_5$  не может быть планарным. Теорема доказана.

### 7.2 Покраска: раскраска вершин и хроматическое число

Строгое правило / Формула:

Раскраска вершин — цвет каждой вершине так, чтобы у смежных цвета различались.  $\chi(G)$  — минимум цветов. Жадная раскраска даёт  $\chi \leq \Delta + 1$ , где  $\Delta$  — максимальная степень.

Планарность и покраска связаны напрямую: раскраска карты (чтобы соседние страны отличались цветом) — это раскраска вершин двойственного графа, а двойственный к планарному тоже планарен. Отсюда знаменитая теорема о четырёх красках (Аппель и Хакен, 1976): любой планарный граф красится в 4 цвета. Это было первое крупное доказательство, где решающий перебор сделал компьютер.

Аналогия 1: Расписание экзаменов

Вершины — предметы, ребро — «эти два предмета выбрали одни и те же студенты». Цвет — слот времени. Раскрасить граф = составить расписание без конфликтов, а  $\chi(G)$  = минимальное число слотов. Жадный алгоритм — это «берём предметы в каком-то порядке и ставим каждый в самый ранний свободный слот».

Аналогия 2: Частоты вышек

Сотовые вышки, которые «слышат» друг друга, нельзя сажать на одну частоту. Частота = цвет, интерференция = ребро. Диапазон частот дорогой, поэтому нужно ровно  $\chi$  цветов — ни больше. Та же модель у распределения регистров в компиляторе: «живые» одновременно переменные не могут лежать в одном регистре.

| Факт            | Формулировка                                   | Почему важно                                                                                 |
|-----------------|------------------------------------------------|----------------------------------------------------------------------------------------------|
| Жадная оценка   | $\chi \leq \Delta + 1$                         | у вершины не больше $\Delta$ соседей, значит один цвет из $\Delta + 1$ всегда свободен       |
| Теорема Брукса  | $\chi \leq \Delta$                             | кроме полных $K_n$ ( $\chi = \Delta + 1$ ) и нечётных циклов ( $\chi = 3$ при $\Delta = 2$ ) |
| Двудольность    | $\chi = 2 \Leftrightarrow$ нет нечётных циклов | проверяется одним BFS/DFS-покрасом за $O(V + E)$ — единственная «лёгкая» раскраска           |
| Планарные графы | $\chi \leq 4$ (и это достижимо: $K_4$ )        | 5 красок доказывается руками через цепи Кемпа, 4 — только перебором                          |
| Раскраска рёбер | $\Delta \leq \chi' \leq \Delta + 1$ (Визинг)   | для двудольных $\chi' = \Delta$ (Кёниг)                                                      |
| Сложность       | $\chi \leq 2$ — P; $\chi \leq 3$ — NP-полно    | даже для планарных графов « $\chi \leq 3$ » остаётся NP-полным (билет 24)                    |

Почему из планарности следует 6 (и даже 5) красок без компьютера. Из  $E \leq 3V - 6$  и суммы степеней  $2E = \sum \deg(v)$  следует, что в любом планарном графе найдётся вершина степени не больше 5 (иначе  $2E \geq 6V$ , то есть  $E \geq 3V$  — противоречие). Убираем её, красим остальное по индукции, возвращаем: у неё максимум 5 соседей, значит шестой цвет точно свободен. Чтобы спуститься до пяти, нужен фокус с цепями Кемпа: если соседи заняли все 5 цветов, смотрим на две вершины с цветами 1 и 3 и проверяем, связаны ли они путём из чередующихся 1-3; если нет — перекрашиваем компоненту и освобождаем цвет.

Ты путаешь:

- Раскраска вершин, рёбер и граней — три разные задачи. Грани планарного графа красятся в 4 цвета (это dual к вершинам), рёбра — в  $\Delta$  или  $\Delta + 1$ .
- $\chi \leq \Delta + 1$  не значит  $\chi = \Delta + 1$ : для дерева  $\Delta$  может быть большим, а  $\chi$  всегда 2 (дерево двудольно).
- «Планарный — значит 5-списочный»: списочная раскраска планарных графов тоже решается 5 цветами, но 4 цветами — уже не всегда (списочный вариант теоремы о 4 красках неверен; верна теорема о 5 списочных).
- $E \leq 3V - 6$  не доказывает планарность — это только необходимое условие.  $K_{3,3}$  его проходит, но непланарен. Достаточный критерий — теорема Куратовского: граф планарен  $\Leftrightarrow$  в нём нет подграфа, который является подразделением  $K_5$  или  $K_{3,3}$ . Эквивалентная формулировка Вагнера — через миноры: нет  $K_5$  и  $K_{3,3}$  как миноров.

Сложность — явно: жадная раскраска  $O(V + E)$  при готовом порядке и  $O(V^2)$  «в лоб» проверкой цветов соседей; проверка двудольности  $O(V + E)$ ; точное  $\chi$  — NP-трудно.

► Код: жадная раскраска и проверка двудольности (Скрыто)

```
# Жадная раскраска: не больше Δ + 1 цвета
color = [-1] * n          # -1 = ещё не покрашена
for v in order:           # порядок влияет на число цветов!
    used = set()
    for to in graph[v]:
        if color[to] != -1:
            used.add(color[to]) # цвета уже покрашенных соседей
    c = 0
    while c in used:         # минимальный свободный
        c += 1
    color[v] = c
chi = max(color) + 1       # χ, который выдал жадный алгоритм

# Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
```

```

for start in range(n):
    if col[start] != -1:
        continue
    col[start] = 0
    dq = deque([start])
    while dq:
        v = dq.popleft()
        for to in graph[v]:
            if col[to] == -1:
                col[to] = col[v] ^ 1      # второй цвет
                dq.append(to)
            elif col[to] == col[v]:      # одноцветные соседи → нечётный цикл
                bipartite = False

```

Порядок важен: жадная раскраска в порядке убывания степеней (largest-first) на практике даёт заметно меньше цветов, чем случайный порядок.

## Демонстрация и связь с компилятором

Планарность и покраска:  $K5/K3,3$ ,  $\chi \leq \Delta + 1$ , двудольность, Визинг

Четыре вкладки: непланарные  $K5$  и  $K3,3$ , жадная покраска вершин, проверка двудольности BFS-ом и покраска рёбер. Массив `color` из вашего кода красит вершины напрямую.

Планарность и покраска:  $K5/K3,3$ ,  $\chi \leq \Delta + 1$ , двудольность, Визинг

### Покраска вершин: $\chi \leq \Delta + 1$

Шаг демонстрации: Шаг = одна вершина  $v$ : смотрим цвета её соседей и берём наименьший свободный — массив `color` сразу перекрашивает граф.

- $n$  — число вершин графа на демонстрации ( $n = 8$ )
- `adj` — списки смежности — тот же граф, что нарисован слева (8 списков)
- `color` — массив цветов вершин (0 = не покрашена). Визуализация красит кружки ПРЯМО по нему (1 ...  $\Delta + 1$ )
- $v$  — вершина, которую красим сейчас — пульсирует жёлтым (0 ...  $n - 1$ )
- $u$  — сосед вершины  $v$ , чей цвет уже занят (`adj[v]`)
- $\Delta$  — максимальная степень  $\Delta = \max \deg(v)$  — верхняя оценка  $\chi \leq \Delta + 1$  ( $\Delta = 3$ )
- $\chi$  — сколько цветов реально понадобилось (1 ...  $\Delta + 1$ )
- $i / \text{step}$  — номер шага — перематывает кадры демонстрации (0 ...  $n$ )

### Покраска вершин: $\chi \leq \Delta + 1$

- $V, E, F$  — для вкладки «Планарность»: вершины, рёбра, грани ( $V - E + F = 2, E \leq 3V - 6$ ) (K5: 5/10 · K3,3: 6/9)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «greedy» — Жадная покраска вершин ( $\chi \leq \Delta + 1$ )

Шаг демонстрации: Шаг = вершина  $v$  получает наименьший цвет, не занятый соседями; массив `color` перекрашивает граф.

- `color` — цвета вершин (0 = не покрашена) — граф красится по этому массиву (1 ... 3)
- $v$  — текущая вершина (0 ... 7)
- `delta / chi` — максимальная степень и число использованных цветов (3 / 3)
- `i / step` — номер кадра (0 ... 8)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «bipartite» — Двудольность: BFS в 2 цвета

Шаг демонстрации: Шаг = сосед  $u$  получает цвет  $3 - \text{color}[v]$ ; совпадение цветов на ребре означает нечётный цикл.

- `color` — разметка на две доли: 1 и 2 (0 = не покрашена) — красит граф ( $\{1, 2\}$ )
- $v$  — вершина, из которой BFS смотрит на соседей (0 ...  $n-1$ )
- $u$  — сосед, которого красим (или с которым случился конфликт) (`adj[v]`)
- `queue` — очередь BFS ( $\leq n$ )
- `ok` — вердикт: граф двудольный (нет нечётного цикла) или нет (True/False)
- `i / step` — номер кадра (0 ...)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «edges» — Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$ )

Шаг демонстрации: Шаг = ребро  $e$  получает наименьший цвет, свободный одновременно в обеих его вершинах.

- `ecolor` — цвета рёбер — рёбра на графе красятся по этому массиву (1 ...  $\Delta+1$ )
- $e$  — номер текущего ребра (0 ...  $m-1$ )
- $v, u$  — концы текущего ребра — подсвечиваются (0 ...  $n-1$ )
- `inc` — для каждой вершины: какие цвета рёбер уже заняты ( $\leq \deg(v)$ )
- `chi_e / delta` —  $\chi'$  — сколько цветов понадобилось рёбрам, и максимальная степень (3 / 3)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Блиц: вопросы по теме

1. Планарный граф имеет  $V=6, E=12$ . Возможно ли это?

- Да, если это двудольный граф
- Нет, нарушается неравенство  $E \leq 3V - 6$  — верный ответ
- Да, если нет треугольных граней

$E \leq 3V - 6 \Rightarrow 12 \leq 12$  — на грани проходит. Но если граф двудольный (нет треугольников), то  $E \leq 2V - 4 = 8$ . А у нас 12. Так что такой граф непланарен.

2. Сколько граней у планарного графа с  $V=7$  и  $E=10$ ?

- 3 грани
- 5 граней — верный ответ
- 10 граней

$F = E - V + 2 = 10 - 7 + 2 = 5$ . Не забудь про внешнюю бесконечную грань — она тоже считается!

3. Почему K5 непланарен?

- Потому что  $10 > 3 \cdot 5 - 6 = 9$  — нарушение — верный ответ
- Потому что Эйлер был против

- Потому что 5 вершин — несчастливое число

У  $K_5$ :  $V=5$ ,  $E=10$ ,  $3V - 6 = 9$ .  $10 > 9$  — противоречие. Отсюда по формуле Эйлера следует, что граф непланарен.

# Графы. Компоненты связности

## Компоненты связности

### 8.1 Поиск кусков графа

Строгое правило / Формула:

Компонента связности — максимальный по включению подграф, в котором между любой парой вершин есть путь. Внешний цикл «для каждой непосещённой вершины запустить DFS/ BFS и покрасить всё достигнутое в новый цвет» даёт и разметку, и количество компонент.

Два рабочих способа. Обходами: один проход по вершинам, из каждой ещё не посещённой запускаем DFS или BFS и присваиваем всем достигнутым вершинам номер компоненты  $comp[v] = c$ . Счётчик  $c$  увеличивается ровно тогда, когда понадобился новый запуск, — поэтому в конце  $c =$  числу компонент. Время  $O(V + E)$ , память  $O(V)$ . Через DSU (систему непересекающихся множеств): начинаем с  $V$  одиночных множеств, на каждое ребро делаем  $union(u, v)$ , в конце число различных корней = число компонент. Это нужно, когда рёбра добавляются на лету и после каждого добавления спрашивают «сколько компонент сейчас?».

Аналогия 1: Острова

Ты десантируешься на любой кусок земли и красишь его краской. Смотришь на карту: остались серые (неисследованные) зоны? Прыгаешь туда и красишь второй остров. Сколько раз пришлось прыгать через океан — столько и компонент. Краска — это  $comp[v]$ , прыжок через океан — увеличение счётчика  $c$ .

Аналогия 2: Группы друзей (DSU)

Каждый человек сначала сам по себе. Приходит новость «А и Б дружат» — склеиваем их компании в одну ( $union$ ). Кто представитель компании? Любой, кого выбрали «старшим» ( $find$  со сжатием пути). Через минуту все разбиты на компании, и вопрос «в одной ли компании А и В?» — это  $find(A) == find(B)$ . Так же работает Краскал (билет 18): ребро берут, только если концы в разных компонентах.

### 8.2 Ориентированные графы: три разных вопроса

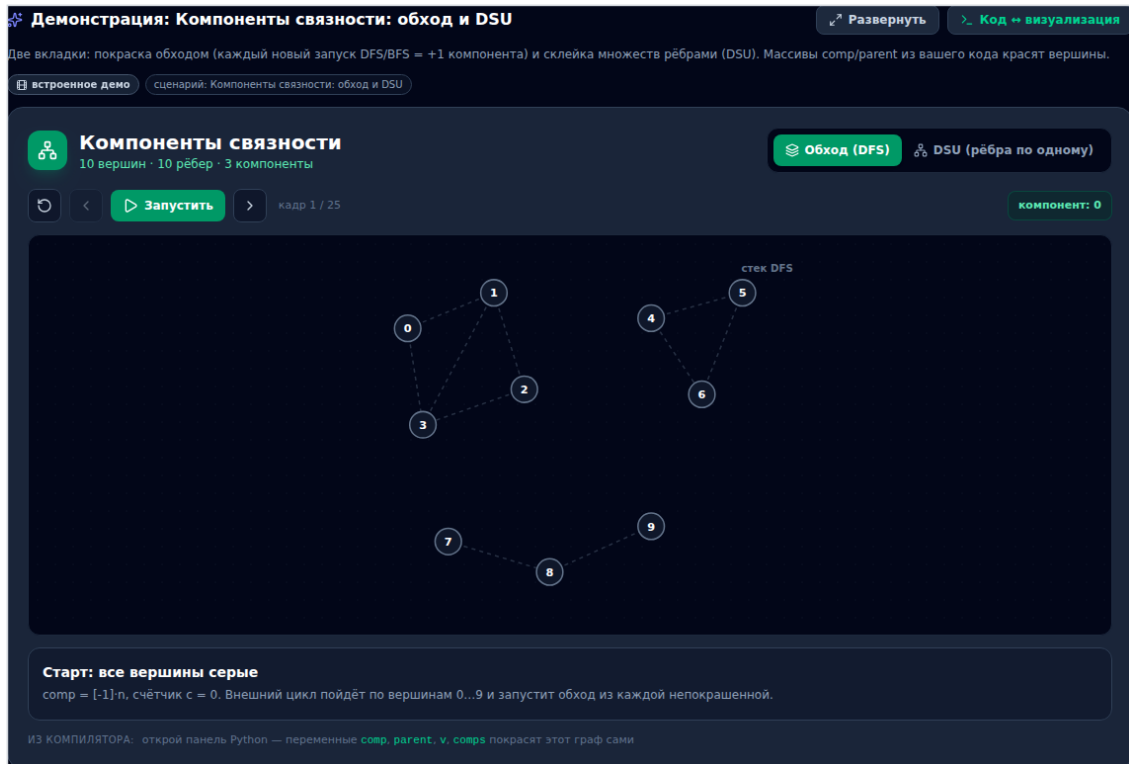
| Что спрашиваем          | Определение                      | Как считать                                      | Сложность  |
|-------------------------|----------------------------------|--------------------------------------------------|------------|
| Слабая связность        | забыли про стрелки — граф связан | DFS/BFS/DSU по базовому неориентированному графу | $O(V + E)$ |
| Сильная связность (SCC) | из любой вершины достижима любая | Косарайю или Тарьян (билет 10)                   | $O(V + E)$ |
| Достижимость из одной   | до кого дойдёт старт             | один DFS/BFS из $s$ (стрелки не забываем!)       | $O(V + E)$ |

Классическая ошибка: запустить BFS по орграфу «как по неориентированному» и назвать результат SCC. Обход из  $s$  даёт только вершины, достижимые из  $s$ ; для SCC нужна достижимость в обе стороны.

## Демонстрация и связь с компилятором

Компоненты связности: обход и DSU

Две вкладки: покраска обходом (каждый новый запуск DFS/BFS = +1 компонента) и склейка множеств рёбрами (DSU). Массивы  $comp/parent$  из вашего кода красят вершины.



Компоненты связности: обход и DSU

### Компоненты связности: DFS-покраска

Шаг демонстрации: Шаг = вершина  $v$  достаётся из стека/очереди; каждый новый запуск обхода из непокрашенной вершины даёт  $+1$  к счётчику comps.

- $n$  — число вершин; в демонстрации граф из 10 вершин и трёх компонент ( $n = 10$ )
- $adj$  — списки смежности — тот же граф, что нарисован слева (10 списков)
- $comp$  — разметка: номер компоненты вершины ( $-1$  = ещё не покрашена). Визуализация красит вершины ПРЯМО по этому массиву ( $-1 \dots comps$ )
- $used$  — булев вариант разметки — для BFS и для проверки «уже посещена» (True/False)
- $comps$  — счётчик найденных компонент связности — большая цифра под графом ( $0 \dots 3$ )
- $c$  — номер компоненты, которую красим сейчас ( $1 \dots comps$ )
- $v$  — вершина, где обход находится сейчас — пульсирует ( $0 \dots n-1$ )
- $stack$  — фронт обхода: стек (DFS) или очередь (BFS); подсвечивается на графе ( $\leq n$  вершин)
- $i / step$  — номер шага — перематывает покадровую демонстрацию ( $0 \dots 2n$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «dfs» — Компоненты связности: DFS-покраска

Шаг демонстрации: Шаг = вершина  $v$  достаётся из стека; соседи из  $adj[v]$  красятся в цвет текущей компоненты  $c$ .

- $comp$  — номер компоненты каждой вершины ( $-1$  = не покрашена) — по нему красится граф ( $-1 \dots 3$ )
- $v$  — текущая вершина обхода ( $0 \dots 9$ )
- $stack$  — стек обхода (для BFS замените  $pop()$  на  $pop(0)$ ) ( $\leq 10$ )
- $comps / c$  — счётчик компонент и номер текущей ( $1 \dots 3$ )
- $i$  — номер шага — перематывает кадры демонстрации ( $0 \dots$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «dsu» — Компоненты связности: DSU по рёбрам

Шаг демонстрации: Шаг = одно ребро  $(u, v)$ :  $union$  склеивает множества, если  $find(u) \neq find(v)$ , и уменьшает comps.

- $parent$  — DSU-массив:  $parent[v]$  = представитель множества. Визуализация рисует множества по корням  $find(v)$  ( $0 \dots n-1$ )

### Вкладка «dsu» — Компоненты связности: DSU по рёбрам

- rank — высота дерева DSU — union by rank не даёт дереву выродиться в бамбук ( $\leq \log n$ )
- e / step — номер обрабатываемого ребра — перематывает кадры (0 ... m-1)
- u, v — концы текущего ребра — подсвечиваются на графе (0 ... n-1)
- comps — сколько множеств осталось: стартуем с n, каждое слияние уменьшает на 1 (n → 3)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Графы. Топологическая сортировка

## Топологическая сортировка

### Упорядочивание ДАГ

Строгое правило / Формула:

DFS с сохранением вершин в стек В МОМЕНТ ВЫХОДА (post-order). Вывод: стек задом-наперед.

Аналогия 1: Учеба в вузе

У тебя есть предметы. "Матан 2" нельзя взять, если не сдал "Матан 1". Топологическая сортировка — это расписание, которое гарантирует, что ты не встретишь предмет, пререквизиты которого ты еще не прошел.

Ограничения: Циклы

Не работает на графах с циклами. "Чтобы устроиться на работу нужен опыт. Чтобы получить опыт, нужна работа". Алгоритм выявит цикл и скажет, что отсортировать невозможно.

ОПАСНОСТЬ НА ЭКЗАМЕНЕ: Простой bool visited НЕ ЛОВИТ циклы!

Обычный массив bool visited[N] отличает лишь «был ли в вершине вообще». Если в графе есть ориентированный цикл, код с bool visited молча выдаст некорректный ответ, не заметив проблему! Правильно: используем раскраску в 3 цвета: • 0 (белый) — еще не посещали; • 1 (серый) — зашли, вершина прямо сейчас лежит в стеке рекурсии DFS; • 2 (чёрный) — полностью обработали и вышли. Если во время обхода из серой вершины мы встречаем рёбрышко в серую вершину (color == 1) — это обратное ребро и гарантированный цикл!

► *Корректный код C++ с детекцией цикла (Скрыто)*

```
vector<int> adj[MAXN];
int color[MAXN]; // 0=белый, 1=серый (в стеке), 2=чёрный
vector<int> ans;
bool has_cycle = false;

void dfs(int v) {
    color[v] = 1; // Зашли — вершина серая
    for (int to : adj[v]) {
        if (color[to] == 0) {
            dfs(to);
            if (has_cycle) return;
        } else if (color[to] == 1) {
            // Переход в серую вершину = НАЙДЕН ЦИКЛ!
            has_cycle = true;
            return;
        }
    }
    color[v] = 2; // Вышли — вершина чёрная
    ans.push_back(v); // Сохраняем в момент ВЫХОДА (post-order)
}

bool topological_sort(int n) {
    fill(color, color + n + 1, 0);
    has_cycle = false;
    ans.clear();
    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) dfs(i);
    }
    if (has_cycle) return false; // Граф содержит цикл!
    reverse(ans.begin(), ans.end()); // Разворачиваем post-order
    return true;
}
```

## Демонстрация и связь с компилятором

### Топологическая сортировка

Шаг демонстрации: Шаг = выход рекурсии из вершины  $v$ : она дописывается в `order`, потом `order` разворачивается.

- $v$  — текущая вершина DFS (0 ... 4 на демо)
- $to$  — вершина, куда ведёт ребро из  $v$  (`adj[v]`)
- $i$  — перебор стартовых вершин во внешнем цикле (вдруг граф несвязен) (0 ... 4)
- `order` — список выхода из рекурсии; после разворота — топологический порядок (растёт на каждом выходе)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

*Снимки демонстрации добавляются автоматически при сборке в CI (headless-браузер проходит по всем вкладкам). В локальной сборке без браузера здесь остаётся текстовое описание: что показывает демонстрация и какие переменные кода её перерисовывают.*

# Графы. Компоненты сильной связности (SCC)

## Компоненты сильной связности (SCC)

### 10.1 Максимальный клуб взаимопомощи

Строгое определение:

Компонента сильной связности (SCC) — это максимальное подмножество вершин ориентированного графа, в котором существует путь из любой вершины в любую другую. «Это просто цикл?» — Нет! SCC — это максимальная группа, где каждый может дойти до каждого.

► Код: Алгоритм Косарайю (Скрыто)

Два прохода DFS:

1. Запускаем DFS на исходном графе. По выходу из вершины (post-order) добавляем её в список `order`.
2. Разворачиваем этот список (получаем порядок убывания времени выхода — `reverse postorder`). Замечание: Это не топологическая сортировка самого графа (поскольку граф содержит циклы!), но этот порядок является топосортом для DAG конденсации.
3. Переворачиваем все рёбра в графе (Транспонированный граф  $G^T$ ).
4. Идем по `order`: если вершина не посещена в  $G^T$ , запускаем от неё DFS. Все вершины, достигнутые за этот запуск — образуют очередную SCC!

## Демонстрация и связь с компилятором

Компоненты сильной связности (Косарайю)

### Компоненты сильной связности (Косарайю)

Шаг демонстрации: Шаг = строка алгоритма: в фазе 1 DFS кладёт вершину в `order`, в фазе 2 вершина из `order` (с конца) запускает DFS по транспонированному графу и красит свою SCC.

- `adj / adjT` — граф и его транспозиция: `adjT[to]` содержит всех, кто ведёт в `to` (5 вершин, 6 рёбер)
- `v` — текущая вершина DFS (в фазе 1 — по графу, в фазе 2 — по обратному) (0 ... 4)
- `to` — сосед по ребру: в фазе 1 прямой, в фазе 2 — обратный (0 ... 4)
- `used` — помеченные первым проходом вершины (`true/false` по каждой)
- `order` — порядок ВЫХОДА из DFS первого прохода — фаза 2 читает его с конца (растёт до 5 вершин)
- `i` — индекс прохода по `order` на втором этапе — идём с конца (`len(order)-1 ... 0`)
- `stack` — стек DFS второго прохода: вершины текущей компоненты (пустеет к концу SCC)
- `u` — вершина, снятая со стека во втором проходе (0 ... 4)
- `comp` — номер компоненты сильной связности каждой вершины — им красит демонстрация (0, 1)
- `c` — счётчик компонент: каждая новая SCC получает свой номер (0 ... 1)
- `n` — число вершин (5)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

Снимки демонстрации добавляются автоматически при сборке в CI (*headless-браузер* проходит по всем вкладкам). В локальной сборке без браузера здесь остаётся текстовое описание: что показывает демонстрация и какие переменные кода её перерисовывают.

# Графы. Мосты

## Мосты: алгоритм Тарьяна + код

### 11.1 Алгоритм Тарьяна

Ключевые массивы:

$tin[v]$  — время захода DFS в вершину  $v$ .

$low[v]$  — минимальный  $tin$ , достижимый из поддерева  $v$  (через обратные рёбра).

УСЛОВИЕ МОСТА:

$low[u] > tin[v]$

( $v$  — родитель,  $u$  — сын в DFS-дереве)

Если из сына  $u$  и всех его потомков нельзя попасть в предка  $v$  (кроме ребра  $(v,u)$ ) — это мост. Единственная ниточка. Порвётся — граф распадётся.

Мост: Единственная веревка

Представь, что ты альпинист. Ты спускаешься в пещеру (вершина  $v$ ) к напарнику (вершина  $u$ ). Из пещеры  $u$  нет других выходов — только та самая дыра, через которую ты спустился. Если верёвка (ребро  $v-u$ ) оборвётся — напарник навсегда замурован. Это и есть мост. Если бы из пещеры  $u$  был чёрный ход наверх (обратное ребро) — напарник бы выбрался. И тогда это не мост.

Аналогия: Кровеносный сосуд

Граф — это кровеносная система. Ребро — сосуд. Если перерезать один сосуд, но кровь может пойти по другому пути (коллатерали) — это не мост, живём. Если же перерезать единственную артерию, ведущую к органу, и альтернатив нет — орган отмирает. Вот это мост. Алгоритм DFS ищет такие «критические артерии».

► [Полный код на C++ \(Скрыто\)](#)

Классическая реализация —  $O(V + E)$ . Каждая вершина и ребро — константное число проходов.

```
#include <vector>
#include <algorithm>

using namespace std;

int n; // число вершин
vector<vector<int>> adj; // список смежности
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++; // устанавливаем время входа

    for (int to : adj[v]) {
        if (to == p) continue; // не идём назад к родителю

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v); // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}
```

```

    }
  }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

Визуализатор под текстом: DFS идёт по графу, tin/low пишутся в таблицу, а мост подсвечивается в момент проверки  $low[u] > tin[v]$ . Кнопка «Код ↔ визуализация» откроет Python-панель, где этот же обход можно выполнить самому — шаги кода двинут демонстрацию.

## Демонстрация и связь с компилятором

Мосты: DFS + tin/low

Мосты: DFS + tin/low

### Мосты: DFS + tin/low

Шаг демонстрации: Шаг = строка псевдокода слева; ребро  $(v, to)$  — мост, когда  $low[to] > tin[v]$ .

- $v$  — текущая вершина DFS (A...G на демо)
- $to$  — сосед; ребро  $(v, to)$  проверяем на мост ( $adj[v]$ )
- $tin[v]$  — время входа в вершину — метка глубины обхода ( $timer++$ )
- $low[v]$  — минимум  $tin$ , куда можно добраться из поддерева  $v$  обратным ребром ( $\leq tin[v]$ )
- $timer$  — глобальный счётчик времени входа (+1 на каждый вход)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Блиц: вопросы по теме

1. После DFS у нас  $low[u] = 4$ , а  $tin[v] = 2$ . Ребро  $(v, u)$  — мост?

- Да, так как  $low[u] (4) > tin[v] (2)$  — верный ответ
- Нет,  $low[u]$  должно быть меньше или равно
- Только если граф связный и неориентированный

*Условие моста:  $low[u] > tin[v]$ . В нашем случае  $4 > 2$  — верно, значит, это мост. Из поддерева  $u$  нет пути к предкам  $v$ .*

2. Для чего нужен массив  $low[v]$ ?

- Для того чтобы было больше переменных в коде
- Чтобы знать минимальный  $tin$  предка, до которого можно «допрыгнуть» по обратным рёбрам — верный ответ
- Чтобы хранить глубину рекурсии

*$low[v]$  хранит минимальное время входа ( $tin$ ), достижимое из поддерева  $v$  через обратные рёбра. Это ключевая метрика для поиска мостов и точек сочленения.*

3. Какова временная сложность алгоритма поиска мостов?

- $O(V^2)$  — ищем мосты в квадрате
- $O(V + E)$  — каждый элемент один раз — верный ответ
- $O(2^V)$  — полный перебор

*DFS обходит каждую вершину и каждое ребро ровно один раз. Линейная сложность  $O(V + E)$  — это гениально.*

# Графы. Точки сочленения

## Точки сочленения (Хрупкость)

### 12.1 Бутылочное горлышко графа

Строгое правило (через DFS Тарьяна):

Точка сочленения (Cut Vertex) — вершина неориентированного графа, удаление которой увеличивает число компонент связности.

УСЛОВИЕ (не для корня DFS):

$low[to] \geq tin[v]$

(Из сына "to" нельзя подняться СТРОГО выше "v")

Важно: Это работает только в неориентированных графах! Если мосты рвут связи между островами, то мосты крепятся к точкам сочленения. То есть, все точки вокруг мостов — это точки сочленения (кроме случаев, когда берег - это тупик со степенью 1).

Аналогия: Главный перекресток

Представь город, где два района соединены только одной площадью (перекрестком). Если на этой площади случится авария и её перекроют (удалят вершину) — районы будут полностью отрезаны друг от друга. Эта площадь — точка сочленения. Это про хрупкость системы.

Телеком: Центральный свитч

У тебя несколько компьютеров в одном кабинете подключены к свитчу А, а в другом — к свитчу Б. И эти свитчи соединены кабелем (мостом). Сами свитчи — это точки сочленения. Сгорел свитч А — весь кабинет номер один выпал из сети, граф распался!

► *Душный мод: Код и корень DFS (Скрыто)*

Особый случай: Корень дерева DFS.

Для стартовой вершины обхода правило  $low[to] \geq tin[v]$  выполняется ВСЕГДА (потому что выше неё прыгать некуда). Поэтому для неё отдельное правило: корень является точкой сочленения тогда и только тогда, когда у него больше 1 независимого ребенка в дереве DFS.

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

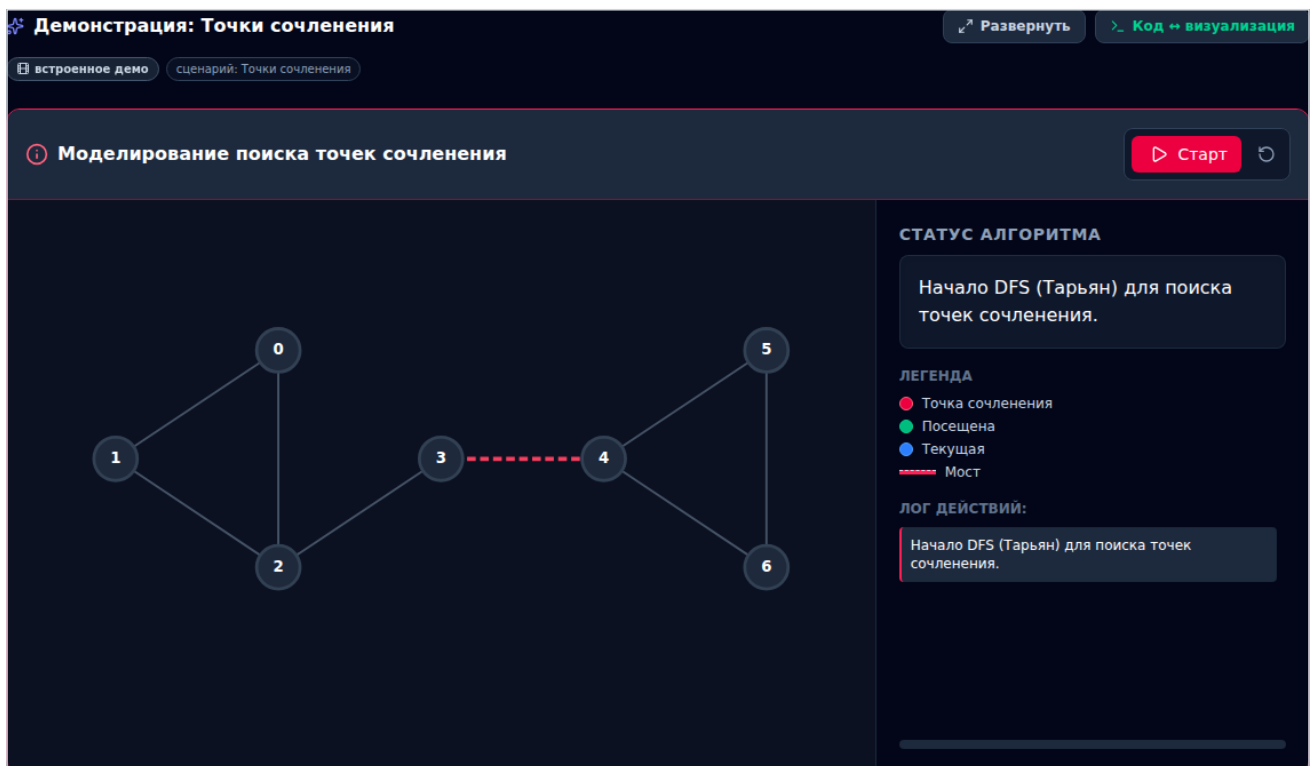
Взаимосвязь с Компонентами сильной связности (SCC) из Билета 10

Это глубокий дуальный мост между ориентированными и неориентированными графами:

- Точки сочленения (Билет 12) определены на неориентированных графах и показывают физическую уязвимость связей — вынь одну вершину, и весь континент распадется на изолированные осколки.
- SCC (Билет 10) склеивают ориентированные циклы в единые независимые "конгломераты".
- Как точка сочленения рушит SCC? (Конденсация графа) Если любой орграф максимально сжать по алгоритму Косарайю, свернув каждую SCC в одну супер-вершину, мы получим ациклический граф (DAG). Если мы снимем направление с его ребер (сделаем неориентированным), то любая точка сочленения в этом полученном скелете — это и есть критическая SCC! Если удалить эту супер-вершину, то рухнет глобальный маршрут между другими клубами сильной связности. Одно слабое звено изолирует целые касты SCC навсегда.

## Демонстрация и связь с компилятором

### Точки сочленения



Точки сочленения

#### Точки сочленения

Шаг демонстрации: Шаг = обновление  $low[v]$  по ребру; вершина  $v$  — точка сочленения, когда  $low[to] \geq tin[v]$ .

- $v$  — текущая вершина DFS (вершины демо-графа)
- $to$  — сосед: либо ребёнок в DFS-дереве, либо обратное ребро ( $adj[v]$ )
- $tin[v]$  — время входа в вершину (порядок обхода) (1, 2, 3...)
- $low[v]$  — минимальный  $tin$ , достижимый из поддерева  $v$  одним обратным ребром ( $\leq tin[v]$ )
- $i$  — перебор стартовых вершин для случая несвязного графа (0 ...  $n-1$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Графы. Эйлеров цикл

---

### Эйлеров цикл и эйлеров путь

#### 13.1 Определения

Строгое правило:

Эйлеров путь — проходит все РЁБРА ровно один раз. Вершины повторять можно.

Эйлеров цикл — то же самое + замкнутый (начало = конец).

Признак существования:

Путь: ровно 2 вершины с нечётной степенью (начало и конец).

Цикл: ВСЕ вершины чётные.

Аналогия 1: Путь — из дома в бар, домой не вернуться

Ты вышел из дома (нечётная), прошёлся по всем улицам города (по каждой ровно раз, это же Эйлер), и усталый пришёл в бар (вторая нечётная). Домой вернуться не смог, потому что ноги не идут. Вот и путь. Начало не равно концу.

Ловушка: цикл — это вернулся домой. Гамильтонов синдром

Гамильтон — это болезнь. Синдром Гамильтона: ты одержимо хочешь посетить каждую вершину ровно раз и вернуться домой. Ты обходишь все бары (вершины) ровно по разу. Какие улицы при этом топтать — плевать. Главное — зайти и выйти, не заходя дважды. Это NP-полная хрень. Никто не знает, как решать быстро. Если скажешь, что придумал алгоритм — ты либо Цукерберг, либо врешь. Запомни: Гамильтон = Госпитализация (тебя упекут за NP-полноту).

► *Душный режим: Формальное доказательство (Скрыто)*

Теорема Эйлера:

Связный граф содержит эйлеров цикл  $\Leftrightarrow$  степень каждой вершины чётная. Эйлеров путь (не цикл)  $\Leftrightarrow$  ровно 2 вершины нечётной степени (это старт и конец).

Почему Гамильтон — NP-полный:

Задача коммивояжёра (TSP) — это взвешенная версия гамильтонова пути. Доказано: полиномиальный алгоритм для гамильтонова цикла (а значит и для TSP) означал бы  $P = NP$ . В общем, не парься. Просто запомни ассоциацию: Гамильтон = больничка.

### Демонстрация и связь с компилятором

Эйлеров путь и цикл

Демонстрация: Эйлеров путь и цикл

Развернуть
Код + визуализация

встроенное демо
сценарий: Эйлеров путь и цикл

Эйлер: пройди все рёбра ровно по одному разу
Сброс

Рёбер: 0/6

Клики на вершину, чтобы начать обход.

**Критерий Эйлера (для связного графа): 2 нечётные (B и C) → есть только ЭЙЛЕРОВ ПУТЬ: старт в одной нечётной, финиш в другой**  
Степени: A=2, B=3, C=3, D=2, E=2. Эйлеров цикл существует ⇔ все степени чётны; путь ⇔ ровно две нечётны. Алгоритм Иера строит маршрут за  $O(V + E)$ .

ИЗ КОМПИЛЯТОРА: открой панель Python и запусти код: массив `path` нарисует маршрут прямо на этом графе, а `deg/odd` пересчитают критерий

## Эйлеров путь и цикл

### Эйлеров маршрут: алгоритм Иера

Шаг демонстрации: Шаг = переход по неиспользованному ребру: `used[i] = True`, вершина дописывается в `path`. Ребро «сгорает» — второй раз по нему идти нельзя.

- `path` — маршрут — последовательность вершин; рисуется поверх графа с номерами рёбер ( $\leq E + 1$  вершин)
- `last / v` — вершина, где обход находится сейчас ( $0 \dots n-1$ )
- `used` — какие рёбра уже пройдены (по номеру в списке `edges`) — подсвечиваются (`True/False`)
- `deg` — степени вершин: чётные всюду → цикл, ровно две нечётные → путь (словарь  $v \rightarrow \text{deg}(v)$ )
- `odd` — список нечётных вершин — по его длине работает критерий Эйлера (0 или 2 вершины)
- `stack` — стек отката алгоритма Иера: из вершины без свободных рёбер возвращаемся назад ( $\leq E$ )
- `step / i` — номер шага обхода ( $0 \dots E$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Блиц: вопросы по теме

1. В связном графе ровно 2 вершины с нечётной степенью, остальные — чётные. Что можно сказать про эйлеров обход?

- Есть эйлеров цикл (вернись домой!)
- Есть эйлеров путь (из одной нечётной в другую), но цикла нет — верный ответ
- Ни пути, ни цикла нет — это полный провал

Если нечётных вершин ровно 2 — это эйлеров путь. Чтобы был цикл, все вершины должны быть чётными. Стартуем в одной нечётной, заканчиваем в другой.

2. Почему гамильтонов цикл — это NP-полная задача, а эйлеров — нет?

- Потому что Гамильтон — болезнь, и лечить её тяжело
- Потому что для эйлерова цикла есть простой критерий (чётность степеней), а для гамильтонова — нет — верный ответ
- Потому что Эйлер был умнее Гамильтона

Для эйлерова цикла работает критерий чётности — проверяется за  $O(V+E)$ . Для гамильтонова нет ни одного простого локального критерия. Это NP-полная задача, которую не решить быстрее перебора.

3. Куда ведёт эйлеров путь, если нечётных вершин больше 2?

- Всё ещё можно пройти по всем рёбрам ровно раз
- Такого графа не существует

- Эйлера пути нет — нужно удалять лишние рёбра — верный ответ

*Если нечётных вершин больше 2, эйлера пути не существует. Придётся удалять рёбра (или добавлять) так, чтобы осталось ровно 2 нечётные или 0.*

# Графы. Поиск кратчайшего пути. Дейкстра

## Дейкстра (Dijkstra)

### 14.1 Суть алгоритма

Строгое правило / Формула (релаксация ребра):

if  $d[v] > d[u] + w(u,v)$  then  $d[v] = d[u] + w(u,v)$

Дейкстра решает задачу SSSP (single source shortest paths): из одной стартовой вершины  $s$  считает кратчайшие расстояния до всех остальных. Единственное требование к графу — неотрицательные веса рёбер. Алгоритм растит «облако» вершин, расстояния до которых уже окончательные, и на каждом шаге забирает в него ту непосещённую вершину  $u$ , у которой  $d[u]$  минимально.

Почему минимум сразу окончательный? Любой конкурентный путь до  $u$  обязан выйти из облака через какую-то непосещённую вершину  $x$ , а у неё  $d[x] \geq d[u]$  (иначе выбрали бы её). Дальше путь только прибавляет неотрицательные веса, значит короче  $d[u]$  уже не станет. Отсюда и вся механика: извлечь минимум → зафиксировать → прорелаксировать все исходящие рёбра. Полное доказательство — в спойлере ниже.

Время (двоичная куча)

$O((V + E) \cdot \log V)$

Время (массив, плотный граф)

$O(V^2 + E)$

Память

$O(V + E)$

Аналогия 1: Курьер с навигатором

Курьер выезжает из пиццерии и мысленно «разливает» волну: сначала ближайшая точка, потом следующая по близости и так далее. Как только он приехал в точку — маршрут до неё финальный: дешевле уже не будет, потому что все дороги платные (веса  $\geq 0$ ) и любой объезд начинается не ближе, чем он сейчас. Ровно это и делает алгоритм: волна расстояний расходится от старта в порядке возрастания.

Ограничения: минус ломает жадность

Дейкстра не пересматривает зафиксированные вершины. Три вершины:  $s \rightarrow a = 2$ ,  $s \rightarrow b = 3$ ,  $b \rightarrow a = -2$ . Верно  $d[a] = 1$  (через  $b$ ), но алгоритм сначала вытащит  $a$  с двойкой и зафиксирует её, а улучшение  $3 - 2 = 1$  придёт слишком поздно. Есть отрицательные веса — идите в Форда—Беллмана (билет 15) или в Джонсона (билет 17).

### 14.2 Шаг за шагом на демо-графе

Демонстрация справа — доставка пиццы: 7 точек (Пиццерия А, Дом 1 В, Парк С, Офис D, Дом 2 Е, ТЦ F, Метро G) и 11 односторонних дорог. Старт — пиццерия А. В компиляторе шаг соответствует извлечению вершины  $u$  из кучи или релаксации ребра ( $u \rightarrow v$ ): переменные  $u$ ,  $v$ ,  $w$  и массив  $dist$  перерисовывают картинку напрямую.

1. Инициализация.  $dist[A] = 0$ , у остальных  $\infty$ ; в кучу кладём  $(0, A)$ ;  $prev$  пуст.
2. Извлечь минимум. Достаём пару  $(du, u)$  с наименьшим  $du$ . Если  $u$  уже в  $done$  — пара устарела, пропускаем («ленивое удаление»).
3. Зафиксировать.  $done.add(u)$ : расстояние окончательное, больше не меняется.
4. Релаксировать рёбра. Для каждого  $(v, w)$  из  $u$ : если  $du + w < dist[v]$ , обновляем  $dist[v]$ , запоминаем  $prev[v] = u$  и кладём  $(dist[v], v)$  в кучу.
5. Повторять, пока куча не пуста. Недостижимые вершины так и останутся с  $\infty$ .

Протокол работы (ровно то, что показывает демонстрация)

| Шаг | Извлечена u (dist) | Что изменили релаксации                                         |
|-----|--------------------|-----------------------------------------------------------------|
| 1   | Пиццерия A (0)     | $B \leftarrow 4, C \leftarrow 2$                                |
| 2   | Парк C (2)         | $B \leftarrow 3$ (было 4), $G \leftarrow 5, E \leftarrow 10$    |
| 3   | Дом 1 B (3)        | $G \leftarrow \text{нет}$ ( $3 + 4 = 7 > 5$ ), $D \leftarrow 8$ |
| 4   | Метро G (5)        | $D \leftarrow 6$ (было 8), $E \leftarrow 7$ (было 10)           |
| 5   | Офис D (6)         | $F \leftarrow 9$                                                |
| 6   | Дом 2 E (7)        | $F \leftarrow 8$ (было 9)                                       |
| 7   | ТЦ F (8)           | исходящих рёбер нет                                             |

Итог:  $\text{dist} = A\ 0, C\ 2, B\ 3, G\ 5, D\ 6, E\ 7, F\ 8$ . Из 11 проверок рёбер 10 улучшили  $\text{dist}$ . Путь до ТЦ по  $\text{prev}$ :  $A \rightarrow C \rightarrow G \rightarrow E \rightarrow F = 2 + 3 + 2 + 1 = 8$ ; до Офиса:  $A \rightarrow C \rightarrow G \rightarrow D = 6$ .

Что именно хранит алгоритм

| Структура                      | Содержимое                              | Зачем                                                      |
|--------------------------------|-----------------------------------------|------------------------------------------------------------|
| $\text{dist}[v]$               | лучший известный путь $s \rightarrow v$ | верхняя оценка ответа; становится точной при фиксации $v$  |
| $\text{prev}[v]$               | предок $v$ на текущем лучшем пути       | восстановление самого пути, а не только длины              |
| $\text{done} / \text{visited}$ | вершины с окончательным $\text{dist}$   | не обрабатываем повторно, отсекаем устаревшие пары из кучи |
| $\text{pq}$ (куча)             | пары (расстояние, вершина)              | извлечение минимума за $O(\log V)$ вместо $O(V)$ перебором |

### 14.3 Корректность и сложность

Инвариант: в момент, когда вершина  $u$  извлекается из кучи,  $\text{dist}[u]$  равно длине кратчайшего пути  $s \rightarrow u$ . Отсюда два следствия: после завершения алгоритма  $\text{dist}[v] = \delta(s, v)$  для всех достижимых  $v$ , и  $\text{dist}[v] = \infty$  для недостижимых.

#### ►► Доказательство инварианта (от противного)

Пусть  $u$  — первая вершина, извлечённая с неверным значением:  $\text{dist}[u] > \delta(s, u)$ . Возьмём настоящий кратчайший путь  $s \rightsquigarrow u$  и пусть  $y$  — первая вершина на нём, которая ещё не извлечена, а  $x$  — её предшественник (он уже извлечён, иначе  $y$  не первая).

Так как  $x$  извлечена раньше и для неё всё верно, релаксация ребра  $(x, y)$  уже состоялась, значит  $\text{dist}[y] \leq \delta(s, x) + w(x, y) = \delta(s, y)$ . Путь до  $y$  — префикс пути до  $u$ , веса неотрицательны, поэтому  $\delta(s, y) \leq \delta(s, u) < \text{dist}[u]$ .

Итого  $\text{dist}[y] < \text{dist}[u]$ , но куча вернула  $u$  — противоречие с тем, что извлекается минимум. Значит, такой  $u$  нет. ■

Где именно нужен неотрицательный вес: в шаге « $\delta(s, y) \leq \delta(s, u)$ ». С отрицательным ребром хвост пути может оказаться дешевле префикса, и неравенство разваливается — это ровно контрпример из карточки «Ограничения».

#### ►► Откуда берётся сложность (и почему куча лучше массива)

- Куча. Каждая вершина извлекается один раз ( $V$  операций  $\text{pop}$ ), каждое ребро релаксируется один раз, и успешная релаксация добавляет  $\text{push}$ . Итого  $O((V + E) \log V)$  — с «ленивым удалением» в куче может лежать до  $E$  пар, но логарифм от этого не меняется по порядку.
- Массив. Поиск минимума линейным перебором —  $O(V)$ , и таких поисков  $V$ : итого  $O(V^2 + E)$ . На плотных графах ( $E \approx V^2$ ) это быстрее кучи и не требует памяти под очередь.

- Куча Фибоначчи.  $O(E + V \log V)$  теоретически, но большая константа: в олимпиадах почти не используется.
- Специальные веса. Веса 0/1 → 0-1 BFS на деке за  $O(V + E)$ ; малые целые веса → корзины Диала. Всё это — билет 17.

## 14.4 Реализации

| Вариант                               | Сложность           | Когда брать                                         |
|---------------------------------------|---------------------|-----------------------------------------------------|
| heapq (Python) / priority_queue (C++) | $O((V + E) \log V)$ | разреженные графы, универсальный выбор              |
| set / куча с decrease-key             | $O((V + E) \log V)$ | нужно «честно» уменьшать ключ, без дублей в очереди |
| массив + линейный поиск минимума      | $O(V^2 + E)$        | $V \leq 5000$ или плотная матрица смежности         |
| 0-1 BFS (дек)                         | $O(V + E)$          | веса только 0 и 1                                   |
| Диал (корзины)                        | $O(E + V \cdot C)$  | целые веса 0...C, C небольшое                       |

►► Код: Дейкстра на куче (Python) + восстановление пути

```
import heapq

INF = float("inf")

def dijkstra(graph, start):
    """graph[v] = [(сосед, вес), ...]; веса >= 0."""
    dist = {v: INF for v in graph}
    prev = {v: None for v in graph}
    dist[start] = 0
    done = set()
    pq = [(0, start)]          # (расстояние, вершина)

    while pq:
        du, u = heapq.heappop(pq)
        if u in done:          # устаревшая пара – ленивое удаление
            continue
        done.add(u)            # dist[u] окончателен
        for v, w in graph[u]:
            if du + w < dist[v]: # релаксация ребра (u, v)
                dist[v] = du + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev

def restore(prev, start, target):
    if prev[target] is None and target != start:
        return None          # вершина недостижима
    path = [target]
    while path[-1] != start:
        path.append(prev[path[-1]])
    return path[::-1]

# демо-граф страницы (доставка пиццы, рёбра односторонние)
g = {
    "A": [("B", 4), ("C", 2)],
    "B": [("G", 4), ("D", 5)],
    "C": [("B", 1), ("G", 3), ("E", 8)],
    "D": [("F", 3)],
    "E": [("F", 1)],
    "F": [],
    "G": [("D", 1), ("E", 2)],
}
dist, prev = dijkstra(g, "A")
print(dist)          # A 0, B 3, C 2, D 6, E 7, F 8, G 5
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E', 'F'] – сумма 8
```

Проверьте себя: запустите этот код в панели Python — демонстрация слева подсветит те же вершины, потому что читает ваши `dist`, `prev` и `done`.

►► Код: C++ (*priority\_queue*, ленивое удаление)

```
#include <vector>
#include <queue>
using namespace std;
const long long INF = 4e18;          // не INT_MAX: INF + w переполнится

int n;
vector<vector<pair<int,int>>> g;      // g[u] = {(v, w)}
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>,
                  vector<pair<long long,int>>,
                  greater<pair<long long,int>>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;      // устаревшая запись
        for (auto [v, w] : g[u]) {
            if (du + w < d[v]) {
                d[v] = du + w;
                p[v] = u;
                pq.push({d[v], v});
            }
        }
    }
}

vector<int> path(int s, int t) {      // восстановление s → t
    vector<int> res;
    if (d[t] == INF) return res;      // недостижим
    for (int v = t; v != -1; v = p[v]) res.push_back(v);
    reverse(res.begin(), res.end());
    return res;
}
```

Обратите внимание на тип: `long long` для расстояний. Путь из  $10^5$  рёбер веса  $10^9$  даёт  $10^{14}$  — `int` переполняется.

►► Код:  $O(V^2)$  без кучи (плотный граф)

```
# w — матрица смежности n × n, w[i][j] = INF если ребра нет
d = [INF] * n
used = [False] * n
d[s] = 0

for _ in range(n):
    u = -1
    for v in range(n):
        # ищем минимум среди непосещённых
        if not used[v] and (u == -1 or d[v] < d[u]):
            u = v
    if u == -1 or d[u] == INF:
        break
    used[u] = True
    # остались только недостижимые
    # фиксируем d[u]
    for v in range(n):
        # релаксируем строку u
        if d[u] + w[u][v] < d[v]:
            d[v] = d[u] + w[u][v]
```

Та же логика, только минимум ищется перебором за  $O(V)$ . Память — матрица  $O(V^2)$ , зато нет накладных расходов кучи.

## 14.5 Ловушки и границы применимости

Где обычно теряют баллы

- Отрицательные веса. Алгоритм не просто «может ошибиться» — он гарантированно не рассчитан на них: фиксация вершины необратима. Нужен Форд—Беллман (билет 15) или перевзвешивание Джонсона (билет 17).
- Отрицательные циклы. При них кратчайшего пути вообще не существует — Дейкстра этого не заметит и выдаст конечное число. Сначала проверьте граф Фордом—Беллманом.
- Ленивое удаление. Без проверки `if du > dist[u]: continue` одна вершина обрабатывается несколько раз: сложность растёт, а на больших графах — TL.
- Переполнение INF. `INF + w` с `INT_MAX` уходит в отрицательное число и «улучшает» всё подряд. Берите INF с запасом и/или пропускайте недостижимые `u`.
- Ориентированность. Демо-граф этой страницы односторонний: дорога  $A \rightarrow B$  не означает  $B \rightarrow A$ . В неориентированном графе каждое ребро добавляется в оба списка смежности.
- Дейкстра  $\neq$  BFS. BFS — частный случай с единичными весами, где очередь FIFO уже даёт порядок по возрастанию. С разными весами FIFO ломается, нужна куча (или дек для 0/1).
- «Одна пара». Алгоритм считает расстояния до всех вершин. Если нужна только одна цель — добавьте ранний выход при извлечении цели (билет 17), но дешевле от этого он не станет асимптотически.

Связь с другими билетами: Форд—Беллман (15) снимает ограничение на знак весов ценой скорости  $O(V \cdot E)$ ; Флойд—Уоршелл (16) считает все пары за  $O(V^3)$ ; билет 17 собирает ускорения — 0-1 BFS, Диал,  $A^*$ , встречный поиск и Джонсон (Дейкстра после перевзвешивания потенциалов). Представление графа списком смежности и обходы — билет 6.

## Иллюстрация

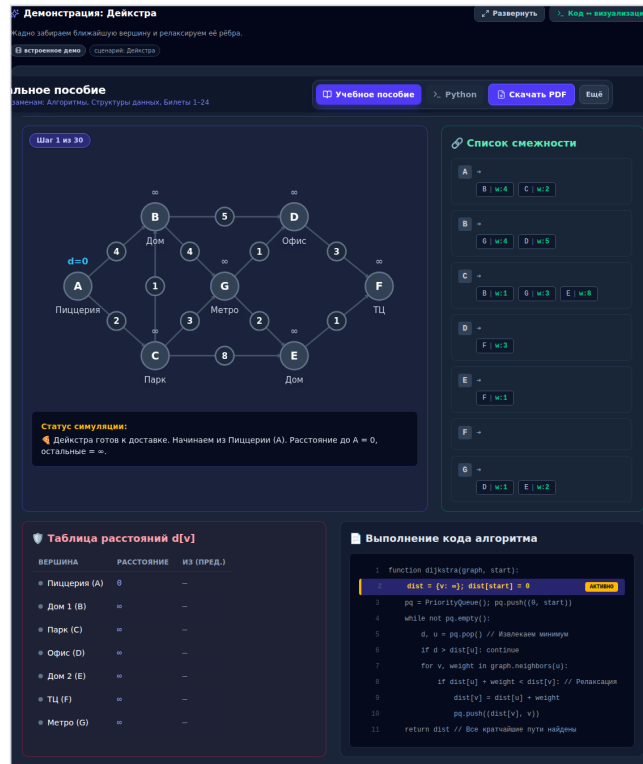


14. Графы. Поиск кратчайшего пути. Дейкстра

## Демонстрация и связь с компилятором

Дейкстра

Жадно забираем ближайшую вершину и релаксируем её рёбра.



Дейкстра

## Дейкстра

Шаг демонстрации: Шаг = старт, извлечение ближайшей вершины  $u$  из кучи или релаксация ребра ( $u \rightarrow v$ ) — как в демонстрации слева.

- $u$  — вершина с минимальным  $\text{dist}$  среди непосещённых — её обрабатываем ( $\text{argmin dist}$ )
- $v$  — сосед вершины  $u$ , до которого пробуем улучшить путь (соседи  $u$ )
- $w$  — вес ребра ( $u, v$ ) (подпись на ребре)
- $\text{dist}[v]$  — лучшее известное расстояние от старта до  $v$  ( $0 \dots \infty$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Графы. Поиск кратчайшего пути. Форд-Беллман

## Форд—Беллман

### 15.1 Жизнь с отрицательными весами

Строгое правило / Формула:

Релаксировать ВСЕ рёбра ровно  $V - 1$  раз; если  $V$ -й проход что-то улучшил — в графе есть отрицательный цикл.

Форд—Беллман решает ту же задачу SSSP, что и Дейкстра, но без требования неотрицательных весов. Плата — скорость: вместо  $O((V + E) \log V)$  получаем  $O(V \cdot E)$ . Никакой кучи и никакого порядка обхода: есть плоский список рёбер  $(u, v, w)$ , и мы  $V - 1$  раз проходим его целиком, релаксируя каждое ребро.

Логика такая: кратчайший путь в графе без отрицательных циклов всегда простой (без повторов вершин), значит в нём не больше  $V - 1$  рёбер. После  $i$ -й итерации точны все расстояния, достижимые путями из  $\leq i$  рёбер. Поэтому  $V - 1$  итераций хватает всегда, а  $V$ -я — диагностическая: улучшение на ней означает, что путь «укорачивается бесконечно», то есть мы попали в отрицательный цикл.

Время

$O(V \cdot E)$

Память

$O(V + E)$

Итераций

$V - 1$  (+1 контроль)

Аналогия 1: Слухи в деревне

Новость («до склада можно доехать за 8 минут») расходится по деревне: за один «день» она проходит ровно одно звено — каждый пересказывает только прямым соседям. Чтобы дойти до точки в четырёх пересказах, нужно четыре дня. Отсюда и предел  $V - 1$ : длиннее цепочки пересказов в деревне из  $V$  жителей просто не бывает. Порядок, в котором жители говорят, не важен — важен только номер дня.

Отрицательный цикл: временная петля

Если на кольце сумма весов отрицательна, каждый проход делает «расстояние» всё меньше:  $-2, -4, -6, \dots$ . Кратчайшего пути не существует (можно крутиться вечно), и ответ « $-\infty$ ».

Контрольная итерация — это детектор: она ловит именно такую бесконечную убыль, а не обычную медленную сходимость.

### 15.2 Шаг за шагом на демо-графе

Демонстрация — снабжение базы: 7 точек (База S, Застава A, Топь 1 B, Мост C, Топь 2 D, Лес E, Склад F) и 9 односторонних дорог, из них одна с отрицательным весом  $B \rightarrow C = -2$  (сплав по течению — быстрее, чем пешком) и обратная  $D \rightarrow A$ , которая в обычном режиме весит  $-2$ , а в режиме «цикл» —  $-6$ . В компиляторе шаг = одна релаксация ребра  $(u, v, w)$  внутри  $i$ -й итерации.

Режим без отрицательного цикла ( $D \rightarrow A = -2$ )

| Итерация | Были изменения? | dist после прохода по 9 рёбрам              |
|----------|-----------------|---------------------------------------------|
| 1        | да              | S 0, A 4, B 5, C 3, D 6, E 7, F 8           |
| 2        | нет             | без изменений → можно выходить досрочно     |
| 3...6    | нет             | гарантия $V - 1 = 6$ итераций уже избыточна |

| Итерация       | Были изменения? | dist после прохода по 9 рёбрам                |
|----------------|-----------------|-----------------------------------------------|
| контроль (7-я) | нет             | отрицательных циклов нет —<br>ответ корректен |

Здесь всё сошлось за одну итерацию, потому что рёбра в списке случайно лежат в «удобном» порядке:  $S \rightarrow A$ ,  $S \rightarrow B$ ,  $B \rightarrow C$ , ... уже передаёт волну от базы до склада за один проход. Поменяйте порядок рёбер местами — и понадобится до шести проходов. Асимптотика от порядка не зависит, а реальное время очень даже.

Режим с отрицательным циклом ( $D \rightarrow A = -6$ )

Кольцо Застава  $A \rightarrow$  Мост  $C \rightarrow$  Топь 2  $D \rightarrow$  Застава  $A$  имеет суммарный вес  $1 + 3 + (-6) = -2$ : каждый круг по нему «дешевеет» на 2.

| Итерация     | dist[A] | dist[C] | dist[D] | dist[F]                                           |
|--------------|---------|---------|---------|---------------------------------------------------|
| 1            | 0       | 3       | 6       | 8                                                 |
| 2            | -2      | 1       | 4       | 6                                                 |
| 4            | -6      | -3      | 0       | 2                                                 |
| $6 = V - 1$  | -10     | -7      | -4      | -2                                                |
| 7 = контроль | -12     | -9      | -6      | всё ещё<br>уменьшается →<br>отрицательный<br>цикл |

Вершина  $B$  остаётся 5: из цикла  $A \rightarrow C \rightarrow D$  в неё рёбер нет, поэтому «заражаются» только достижимые из цикла вершины ( $A, C, D, E, F$ ). Это важный нюанс: отрицательный цикл портит ответ не для всех пар, а только для тех, до которых из него есть путь.

### 15.3 Почему именно $V - 1$ итерация

#### ►► Доказательство (индукция по длине пути)

Утверждение. После  $i$ -й итерации для каждой вершины  $v$  значение  $\text{dist}[v]$  не превосходит длины кратчайшего пути из  $s$  в  $v$ , содержащего не более  $i$  рёбер.

База.  $i = 0$ : путь без рёбер существует только в  $s$ , и  $\text{dist}[s] = 0$ .

Шаг. Пусть утверждение верно для  $i - 1$ . Возьмём кратчайший путь  $P$  из  $s$  в  $v$  из  $\leq i$  рёбер в вершину  $v$ ; его последнее ребро —  $(u, v)$ , а префикс до  $u$  содержит  $\leq i - 1$  рёбер и сам является кратчайшим (иначе заменили бы префикс и получили короче). По предположению после  $i - 1$  итераций  $\text{dist}[u] \leq d(P \text{ до } u)$ , а в течение  $i$ -й итерации ребро  $(u, v)$  обязательно будет обработано и даст  $\text{dist}[v] \leq \text{dist}[u] + w(u, v) \leq |P|$  ■

Следствие 1. Если отрицательных циклов, достижимых из  $s$ , нет, то кратчайший путь прост и содержит  $\leq V - 1$  рёбер  $\rightarrow$  после  $V - 1$  итераций все  $\text{dist}$  точны.

Следствие 2. Если  $V$ -я итерация что-то улучшила, то существует путь из  $s$  в  $v$  из  $\geq V$  рёбер, который короче любого простого пути. В нём повторяется вершина, значит есть цикл; будь его вес  $\geq 0$ , выкидывание цикла не ухудшило бы путь — противоречие. Значит, цикл отрицательный. ■

Сравнение трёх алгоритмов кратчайших путей

| Критерий            | Дейкстра (14)                  | Форд—Беллман (15)              | Флойд—Уоршелл (16)             |
|---------------------|--------------------------------|--------------------------------|--------------------------------|
| Задача              | одна вершина $\rightarrow$ все | одна вершина $\rightarrow$ все | все пары                       |
| Время               | $O((V+E) \log V)$              | $O(V \cdot E)$                 | $O(V^3)$                       |
| Отрицательные рёбра | нет                            | да                             | да                             |
| Отрицательные циклы | молча ошибётся                 | обнаруживает                   | обнаруживает ( $d[i][i] < 0$ ) |
| Представление графа | список смежности               | плоский список рёбер           | матрица                        |

| Критерий       | Дейкстра (14)                   | Форд—Беллман (15)             | Флойд—Уоршелл (16)            |
|----------------|---------------------------------|-------------------------------|-------------------------------|
| Когда выбирать | разреженный граф, веса $\geq 0$ | минусы, нужен детектор циклов | $V \leq 500$ , нужны все пары |

## 15.4 Код

►► Код: Форд—Беллман + ранний выход + поиск отрицательного цикла (Python)

```

INF = float("inf")

def bellman_ford(n, edges, s):
    """edges = [(u, v, w), ...] — ориентированные рёбра."""
    dist = [INF] * n
    prev = [-1] * n
    dist[s] = 0
    changed_vertex = -1

    for it in range(n):
        # n-1 рабочих + одна контрольная
        changed_vertex = -1
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                changed_vertex = v
        if changed_vertex == -1:
            break
        # ничего не изменилось — сошлось досрочно

    if changed_vertex == -1:
        return dist, prev, None
        # отрицательных циклов нет

    # changed_vertex улучшен на n-й итерации:
    # поднимаемся по prev на n шагов — гарантированно заходим внутрь цикла
    y = changed_vertex
    for _ in range(n):
        y = prev[y]

    cycle = [y]
    v = prev[y]
    while v != y:
        cycle.append(v)
        v = prev[v]
    cycle.append(y)
    cycle.reverse()
    return dist, prev, cycle

# демо-граф страницы (9 рёбер, режим с отрицательным циклом)
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [("S","A",4), ("S","B",5), ("B","C",-2), ("A","C",1),
      ("C","D",3), ("C","E",4), ("D","F",2), ("E","F",1)]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D"], idx["A"], -2)], 0)
print([dist[i] for i in range(len(nodes))]) # [0, 4, 5, 3, 6, 7, 8]
print(cyc) # None — цикла нет

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D"], idx["A"], -6)], 0)
print([nodes[i] for i in range(len(nodes))]) # ['D', 'A', 'C', 'D'] — вес -6+1+3 = -2
print([dist2[i] for i in range(len(nodes))]) # всё ещё уменьшается: цикла нет конца

```

Проверка `dist[u] != INF` обязательна: без неё недостижимая вершина с INF «улучшает» соседей ( $INF + w < INF$  — ложь для `float('inf')`), но правда при целочисленном переполнении).

►► Код: C++ (структура *Edge*,  $V - 1$  итераций, детектор цикла)

```

#include <vector>
#include <iostream>
using namespace std;
const long long INF = 4e18;

struct Edge { int u, v; long long w; };

int n; // число вершин
vector<Edge> edges; // плоский список рёбер – порядок не важен
vector<long long> d;
vector<int> p;

// возвращает вершину, улучшенную на «лишней» итерации, или -1
int bellman_ford(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    int x = -1;
    for (int it = 0; it < n; ++it) { // n-1 рабочих + контроль
        x = -1;
        for (const auto &e : edges) {
            if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
                d[e.v] = d[e.u] + e.w;
                p[e.v] = e.u;
                x = e.v;
            }
        }
        if (x == -1) break; // ранний выход
    }
    return x;
}

vector<int> negative_cycle(int x) {
    int y = x;
    for (int i = 0; i < n; ++i) y = p[y]; // заходим внутрь цикла
    vector<int> cycle;
    for (int v = y;; v = p[v]) {
        cycle.push_back(v);
        if (v == y && cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return cycle;
}

int main() {
    int s = 0;
    int x = bellman_ford(s);
    if (x == -1) cout << "отрицательных циклов нет";
    else {
        cout << "цикл: ";
        for (int v : negative_cycle(x)) cout << v << " ";
        cout << "\n";
    }
}

```

### ►► Оптимизации: ранний выход, SPFA (очередь), порядок рёбер

```

from collections import deque

def spfa(n, adj, s):
    """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины."""
    INF = float("inf")
    dist = [INF] * n
    in_queue = [False] * n
    counter = [0] * n # сколько раз вершина попадала в очередь
    dist[s] = 0
    q = deque([s])
    in_queue[s] = True
    while q:
        u = q.popleft()
        in_queue[u] = False
        for v, w in adj[u]:

```

```

    if dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        if not in_queue[v]:
            q.append(v)
            in_queue[v] = True
            counter[v] += 1
            if counter[v] >= n:
                return None    # отрицательный цикл
return dist

```

- Ранний выход. Если за итерацию ничего не изменилось — distances уже финальные, выходим. На «коротких» графах экономит кратно.
- SPFA (Shortest Path Faster Algorithm) — Форд—Беллман на очереди: релаксируем рёбра только из тех вершин, у которых только что улучшился dist . На практике часто почти  $O(E)$  , но худший случай остаётся  $O(V \cdot E)$  , и на специально подобранных тестах (сетка, «каркас» из рёбер) SPFA ловит TL. Критерий цикла: вершина попала в очередь  $\geq V$  раз.
- Порядок рёбер. Формально не важен, фактически решает: рёбра «по течению» обхода дают сходимость за 1–2 итерации, против течения — за  $V - 1$  . В демо-графе список рёбер удачный, поэтому первая же итерация даёт финальные dist .
- DAG. Если граф ациклический, релаксация в топологическом порядке (билет 9) даёт ответ за один проход —  $O(V + E)$  , и отрицательные веса не мешают.

## 15.5 Ловушки

Где обычно теряют баллы

- $V - 1$ , а не  $V$ . Рабочих итераций ровно  $V - 1$  ;  $V$ -я — только детектор. Если сделать  $V$  рабочих, то на графе с отрицательным циклом dist «уедет» ещё дальше, а вывод о корректности станет неверным.
- Переполнение INF.  $INF + w$  при  $INF = INT\_MAX$  превращается в отрицательное число и «улучшает» всё. Пропускайте рёбра из недостижимых вершин или берите INF с запасом и long long .
- Неориентированный граф с минусом. Ребро  $\{u, v\}$  веса  $-3$  — это цикл длины 2 веса  $-6$  : Форд—Беллман всегда сообщит об отрицательном цикле. Минусы в неориентированном графе бессмысленны.
- Цикл есть, но не от старта. Детектор срабатывает только для циклов, достижимых из  $s$  . Нужна проверка всего графа — добавьте фиктивную вершину с нулевыми рёбрами во все вершины (приём из алгоритма Джонсона, билет 17) и запустите от неё.
- «Ответ  $-\infty$ ». Если отрицательный цикл лежит на пути к вершине  $v$  , то  $dist[v] = -\infty$  : печатать конкретное число нельзя, нужен отдельный вывод. Достигается из цикла помечаются обходом в обратном графе.
- SPFA как «быстрый Форд—Беллман». На случайных графах он быстрый, на adversarial-тестах — квадратичный. Если в условии  $V, E \leq 10^5$  и веса неотрицательны, берите Дейкстру.

Связь с другими билетами: один прогон Форда—Беллмана от фиктивной вершины даёт потенциалы  $h(v)$  для алгоритма Джонсона (билет 17), после чего все пары считаются Дейкстрой. Флойд—Уоршелл (16) решает все пары без списка рёбер, но за  $O(V^3)$ . Топологический порядок для DAG — билет 9.

## Иллюстрация



15. Графы. Поиск кратчайшего пути. Форд-Беллман

## Демонстрация и связь с компилятором

Форд—Беллман

Демонстрация: Форд—Беллман

Учебное пособие

Python

Скачать PDF

Ещё

Форд-Беллман (Фура)

Полный дашборд: Граф ↔ Таблица ↔ Список смежности

Обычный

С циклом

Сброс

Пуск

Шаг

Итерация 0 (Шаг 1 из 21)

Граф:

```

graph LR
    S((S)) ---|4| A((A))
    S ---|5| B((B))
    A ---|1| C((C))
    A ---|-2| D((D))
    B ---|-2| C
    C ---|3| D
    C ---|4| E((E))
    D ---|2| F((F))
    E ---|1| F
    
```

Список смежности:

- S → A (4) B (5)
- A → C (1)
- B → C (-2)
- C → D (3) E (4)
- D → F (2) A (-2)
- E → F (1)
- F →

Статус симуляции:

Фура заведена. Начинаем с Базы (S). Расстояние до S = 0, остальные = ∞. Алгоритм требует |V|-1 итераций.

Таблица расстояний d[v]

|           | S - База | A - Застава | B - Топь | C - Мост |
|-----------|----------|-------------|----------|----------|
| S - База  | 0        | ∞           | ∞        | ∞        |
| D - Топь  | ∞        | ∞           | ∞        | ∞        |
| E - Лес   | ∞        | ∞           | ∞        | ∞        |
| F - Склад | ∞        | ∞           | ∞        | ∞        |

Код алгоритма

```

1 function bellman_ford(graph, start, V, E):
2     dist = {v: ∞}
3     dist[start] = 0
4     for i from 1 to V - 1: // |V|-1 итераций по ВСЕМ рёбрам
5         for (u, v, weight) in E:
6             if dist[u] + weight < dist[v]: // Релаксация
7                 dist[v] = dist[u] + weight
8     for (u, v, weight) in E: // Проверка на отрицательный цикл
9         if dist[u] + weight < dist[v]:
10            return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ!"
11    return dist
    
```

Форд—Беллман

### Форд—Беллман

Шаг демонстрации: Шаг = одна релаксация ребра (u, v, w) внутри i-й итерации по всем рёбрам.

- n — число вершин графа — ровно n-1 полных итераций (7 на демо)
- m — число рёбер, по которым проходим на каждой итерации (9 на демо (список E))

### Форд—Беллман

- $i$  — номер итерации (после  $i$  итераций точны все пути из  $\leq i$  рёбер) ( $1 \dots n-1$ )
- $u, v, w$  — текущее ребро: откуда, куда и его вес (перебор  $E$ )
- $\text{dist}[v]$  — расстояние; если обновилось на  $n$ -й итерации — отрицательный цикл ( $0 \dots \infty$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Графы. Поиск кратчайшего пути. Флойд

## Флойд—Уоршелл (динамика на промежуточных вершинах)

### 16.1 Флойд не считает шаги — он считает разрешённые вершины

Формула (выбей на лбу):

$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$ ,  $k$  — ВНЕШНИЙ цикл

Флойд—Уоршелл считает все пары кратчайших расстояний за  $O(V^3)$  и  $O(V^2)$  памяти, работает с отрицательными весами (но не с отрицательными циклами) и пишется в три строки. Вход — матрица  $d$  размера  $V \times V$ :  $d[i][j] = w(i, j)$  для существующих рёбер, 0 на диагонали и  $\infty$  в остальных клетках.

Суть фокуса (по шагам)

Главный герой — внешний цикл  $\text{for } k \text{ in } 0 \dots V-1$ . На шаге  $k$  таблица хранит ответ на вопрос: «каков кратчайший путь  $i \rightarrow j$ , если в качестве промежуточных разрешено использовать только вершины из множества  $\{0, 1, \dots, k\}$ ?». Каждый шаг добавляет в разрешение одну новую вершину:

- Шаг  $k = -1$  (старт): разрешены только прямые рёбра — пересадок нет вообще.
- Шаг  $k = 0$ : разрешаем пересадку через вершину 0: проверяем, не короче ли  $i \rightarrow 0 \rightarrow j$ , чем текущий  $i \rightarrow j$ .
- Шаг  $k = 1$ : разрешены пересадки через  $\{0, 1\}$ ; внутри путей  $i \rightarrow 1$  и  $1 \rightarrow j$  уже может стоять вершина 0.
- Шаг  $k = V - 1$ : разрешены все вершины — таблица содержит финальные расстояния для всех пар.

Время

$O(V^3)$

Память

$O(V^2)$

Практический предел

$V \approx 500$  ( $1.25 \cdot 10^8$  операций)

Аналогия 1: Таблица пересадок в метро

Вы печатаете плакат «из любой станции в любую — сколько минут». Сначала в таблице только соседние станции. Затем говорите: «разрешаю пересадку на станции 1» — и пересчитываете весь плакат, подставляя через 1 там, где это короче. Потом разрешаете станцию 2, 3, ... После того как разрешены все станции, плакат окончательный. Порядок «разрешений» не случайный: он и есть внешний цикл  $k$ .

Аналогия 2: Почему  $k$  обязан быть внешним

Если перепутать циклы местами, вы разрешаете пересадку через станцию  $k$  один раз и больше к ней не возвращаетесь: путь « $i \rightarrow k \rightarrow \dots \rightarrow k \rightarrow j$ » с двумя заходами в ту же пересадку не соберётся. На демо-графе этой страницы порядок  $(i, j, k)$  оставляет  $d[2][1] = \infty$ , хотя верный ответ 4 (путь  $2 \rightarrow 4 \rightarrow 6 \rightarrow 1$ : он дважды использует «поздние» посредники). Проверка перестановкой циклов — в спойлере с доказательством.

### 16.2 Шаг за шагом на демо-графе

Демонстрация показывает матрицу  $7 \times 7$  (вершины  $0 \dots 6$ , 999 вместо  $\infty$ ) и подсвечивает клетку  $(i, j)$ , которая улучшается на текущем посреднике  $k$ . В компиляторе переменные  $k, i, j$  и матрица  $\text{dist}$  управляют картинкой напрямую: измените  $k$  — демонстрация покажет другую «волну» разрешённых пересадок.

Сколько улучшений даёт каждый посредник  $k$

| k | Улучшено клеток | Комментарий                                                      |
|---|-----------------|------------------------------------------------------------------|
| 0 | 0               | в вершину 0 никто не входит — через неё не пройти                |
| 1 | 4               | открывает пути «...→1→2» и «...→1→5»                             |
| 2 | 3               | 2 — вход в подграф 4, 6                                          |
| 3 | 2               | 3 — вход в 4 и 5 «сверху»                                        |
| 4 | 4               | через 4 достижима 6                                              |
| 5 | 0               | из 5 выход только в 6, и тот уже учтён дешевле                   |
| 6 | 10              | ребро 6 → 1 замыкает граф в кольцо: открывается больше всего пар |

Всего 23 улучшения из 343 проверок ( $7^3$ ). Итоговая первая строка:  $0 \rightarrow \{0, 4, 7, 2, 6, 4, 7\}$ , то есть  $d[0][6] = 7$  путём  $0 \rightarrow 3 \rightarrow 4 \rightarrow 6$  ( $2 + 4 + 1$ ). Обратного пути нет:  $d[6][0] = 999$  (недостижимо), всего в финальной матрице 11 таких пар. Диагональ осталась нулевой — отрицательных циклов в графе нет.

Что читать в ответе

| Клетка    | Значение | Смысл                                                                                |
|-----------|----------|--------------------------------------------------------------------------------------|
| $d[i][j]$ | конечное | длина кратчайшего пути $i \rightarrow j$                                             |
| $d[i][j]$ | INF      | $j$ недостижима из $i$ (в орграфе это нормально и несимметрично)                     |
| $d[i][i]$ | 0        | отрицательных циклов через $i$ нет                                                   |
| $d[i][i]$ | $< 0$    | через $i$ проходит отрицательный цикл — ответы для связанных с ней пар не определены |

### 16.3 Динамика, доказательство и порядок циклов

#### ►► Формальная динамика и почему пересчёт «на месте» корректен

Обозначим  $d^{(k)}[i][j]$  — длину кратчайшего пути  $i \rightarrow j$ , все промежуточные вершины которого лежат в  $\{0, \dots, k\}$  (концы пути  $i$  и  $j$  в это множество входить не обязаны).

База:  $d^{(-1)}[i][j] = w(i, j)$  (прямое ребро или  $\infty$ ), на диагонали 0.

Переход: путь из  $i$  в  $j$  с посредниками из  $\{0 \dots k\}$  либо не использует  $k$  — тогда это  $d^{(k-1)}[i][j]$ , — либо использует  $k$  ровно один раз (без отрицательных циклов кратчайший путь прост), разбиваясь на два подпути с посредниками из  $\{0 \dots k-1\}$ :

$$d^{(k)}[i][j] = \min( d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j] )$$

Почему хватает одной матрицы. На шаге  $k$  строка  $k$  и столбец  $k$  не меняются:  $d^{(k)}[i][k] = \min(d^{(k-1)}[i][k], d^{(k-1)}[i][k] + d^{(k-1)}[k][k])$ , а  $d^{(k-1)}[k][k] = 0$  при отсутствии отрицательных циклов — второе слагаемое не меньше первого. Значит, читая  $d[i][k]$  и  $d[k][j]$  во время  $k$ -й итерации, мы читаем именно значения слоя  $k - 1$ , и перезапись  $d[i][j]$  безопасна.

Следствие: если в графе есть отрицательный цикл через  $k$ , то  $d[k][k] < 0$  и рассуждение рассыпается — отсюда и детектор по диагонали. ■

#### ►► Эксперимент: переставим циклы и сломаем ответ

```

INF = 999
M = [[0,4,INF,2,INF,INF,INF],
      [INF,0,3,INF,INF,3,INF],
      [INF,INF,0,INF,2,INF,INF],
      [INF,INF,INF,0,4,2,INF],
      [INF,INF,INF,INF,0,INF,1],
      [INF,INF,INF,INF,INF,0,5],
      [INF,1,INF,INF,INF,INF,0]]

def floyd(M, order="kij"):
    n = len(M)
    d = [row[:] for row in M]
    for a in range(n):
        for b in range(n):
            for c in range(n):
                k, i, j = {"kij": (a, b, c),
                           "ijk": (c, a, b),
                           "ikj": (c, a, b)}[order]
                # ВЕРНО
                # k внутренний
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1 потерян
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
# 4 клетки отличаются; при порядке (i,k,j) их 6

```

Вывод: порядок (k, i, j) — единственно верный. Два других «почти работают» (большинство клеток совпадает), и именно поэтому ошибка так легко проскакивает на маленьких тестах.

## 16.4 Код и варианты

►► *Код: Флойд—Уоршелл + детектор отрицательных циклов + восстановление пути (Python)*

```

INF = float("inf")

def floyd_warshall(n, edges, directed=True):
    d = [[INF] * n for _ in range(n)]
    nxt = [[-1] * n for _ in range(n)]
    for i in range(n):
        d[i][i] = 0 # диагональ обязательно 0

    for u, v, w in edges:
        if w < d[u][v]: # кратчайшее из параллельных рёбер
            d[u][v] = w
            nxt[u][v] = v
        if not directed and w < d[v][u]:
            d[v][u] = w
            nxt[v][u] = u

    for k in range(n): # k – ВНЕШНИЙ цикл
        for i in range(n):
            if d[i][k] == INF:
                continue
            for j in range(n):
                if d[k][j] == INF:
                    continue
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
                    nxt[i][j] = nxt[i][k]

    negative = [i for i in range(n) if d[i][i] < 0]
    return d, nxt, negative

def path(nxt, i, j):
    if nxt[i][j] == -1:
        return None # j недостижима из i
    p = [i]
    while p[-1] != j:
        p.append(nxt[p[-1]][j])
    return p

```

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [("S","A",4), ("S","B",5), ("B","C",-2), ("A","C",1),
      ("C","D",3), ("C","E",4), ("D","F",2), ("E","F",1), ("D","A",-2)]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 6, 7, 8]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 3, 4, 6] → S B C D F = 5 - 2 + 3 + 2 = 8
print(neg) # [] – отрицательных циклов нет

```

### ►► Код: C++ (матрица, защита от переполнения, диаметр и центр графа)

```

#include <vector>
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>> d(n, vector<long long>(n, INF));

void floyd() {
    for (int i = 0; i < n; ++i) d[i][i] = 0;
    for (int k = 0; k < n; ++k) // k – внешний
        for (int i = 0; i < n; ++i) {
            if (d[i][k] == INF) continue; // защита от INF + w
            for (int j = 0; j < n; ++j) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
}

bool has_negative_cycle() {
    for (int i = 0; i < n; ++i) if (d[i][i] < 0) return true;
    return false;
}

long long diameter() { // самая дальняя пара
    long long res = 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][j] < INF) res = max(res, d[i][j]);
    return res;
}

int center() { // вершина с минимальным эксцентриситетом
    int best = -1; long long bestEcc = INF;
    for (int i = 0; i < n; ++i) {
        long long ecc = 0;
        for (int j = 0; j < n; ++j) {
            if (d[i][j] == INF) { ecc = INF; break; }
            ecc = max(ecc, d[i][j]);
        }
        if (ecc < bestEcc) { bestEcc = ecc; best = i; }
    }
    return best;
}

```

### ►► Та же динамика для других задач (замыкание, минимакс, число путей)

| Задача                                | Переход                                           | Начальное значение                              |
|---------------------------------------|---------------------------------------------------|-------------------------------------------------|
| Транзитивное замыкание (достижимость) | $r[i][j]  = r[i][k] \ \&\& \ r[k][j]$             | $r[i][j]$ = есть ребро; $r[i][i] = \text{true}$ |
| Минимакс (самое «узкое» место пути)   | $d[i][j] = \min(d[i][j], \max(d[i][k], d[k][j]))$ | $d[i][j] = w(i,j)$ ; $d[i][i] = 0$              |
| Максимальный путь в DAG-подобной сети | $d[i][j] = \max(d[i][j], d[i][k] + d[k][j])$      | только при отсутствии положительных циклов      |

| Задача                 | Переход                                                                      | Начальное значение                            |
|------------------------|------------------------------------------------------------------------------|-----------------------------------------------|
| Число кратчайших путей | if $d[i][k] + d[k][j] == d[i][j]$ : $cnt[i][j] += cnt[i][k] \cdot cnt[k][j]$ | $cnt = 1$ на рёбрах; следите за переполнением |

Замыкание удобно ускорять битсетами:  $r[i] \mid= r[k]$  при  $r[i][k]$  — это  $O(V^3 / 64)$  вместо  $O(V^3)$ .

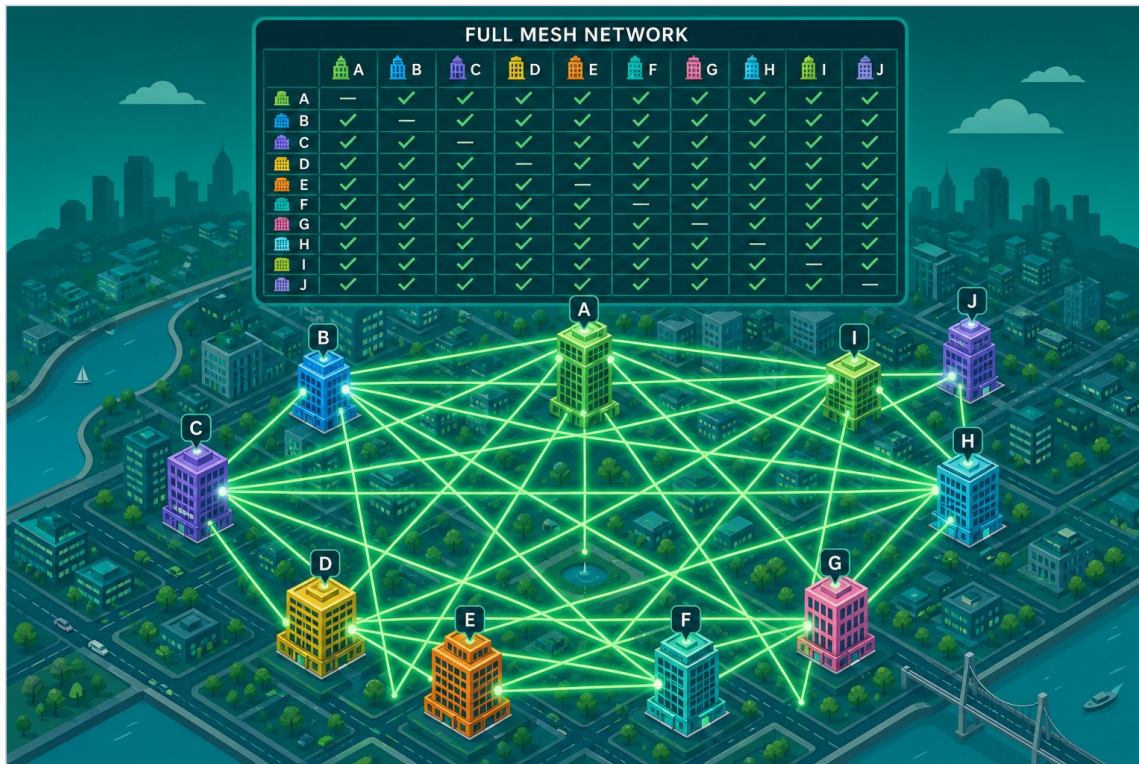
## 16.5 Ловушки и границы применимости

Где обычно теряют баллы

- Порядок циклов.  $k$  — строго внешний. При  $(i, j, k)$  на демо-графе 4 клетки из 49 остаются  $\infty$  вместо верных значений (например,  $d[2][1] = 999$  вместо 4).
- Диагональ.  $d[i][i] = 0$  до начала. Если оставить  $\infty$ , алгоритм «починит» её только при наличии цикла, а пересчёт на месте станет некорректным.
- Переполнение  $INF$ .  $INF + INF$  при  $INF = INT\_MAX$  уходит в минус и «улучшает» всё подряд. Пропускайте пары с  $\infty$  (как в коде выше) или берите  $INF \approx 4 \cdot 10^{18}$  с `long long`.
- Параллельные рёбра. Во входной матрице нужно оставить минимальное из них, иначе  $d[u][v]$  стартует с худшего веса.
- Отрицательные циклы. Формула при  $d[k][k] < 0$  расходится: для пар, связанных с циклом, ответ  $-\infty$ . Сначала проверьте диагональ, затем пометьте такие пары:  $d[i][j] = -\infty$ , если существует  $k$  с  $d[k][k] < 0$ ,  $d[i][k] < \infty$ ,  $d[k][j] < \infty$ .
- Недостижимость  $\neq 0$ . В орграфе  $d[i][j] = \infty$  — обычный ответ (на демо-графе 11 таких пар из 49). Выводить 999 или 0 как «расстояние» нельзя.
- Память.  $V^2$  клеток: при  $V = 5000$  это  $25 \cdot 10^6$  значений ( $\approx 100$  МБ на `int`) — обычно уже за пределами ограничения. Для разреженных графов и одного истока берите Дейкстру.
- Сам путь. Матрица  $d$  даёт только длины. Нужен маршрут — ведите параллельно `next[i][j]` (как в коде) или восстанавливайте рекурсивно через посредников.

Связь с другими билетами: Дейкстра (14) и Форд—Беллман (15) решают ту же задачу от одного истока и дешевле, если все пары не нужны; если все пары нужны, а граф разреженный и веса неотрицательны, запуск  $V$  Дейкстр быстрее ( $O(V \cdot E \log V)$  против  $O(V^3)$ ). Алгоритм Джонсона (билет 17) комбинирует оба подхода: один Форд—Беллман для потенциалов +  $V$  Дейкстр на перевзвешенном графе. Транзитивное замыкание связано с компонентами сильной связности (билет 10): в конденсации достижимость проверяется по DAG.

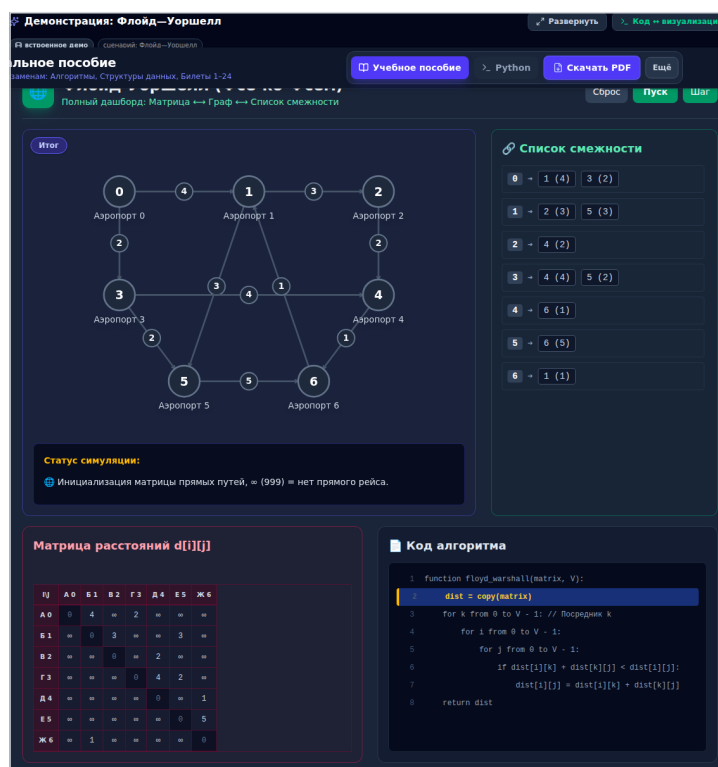
## Иллюстрация



16. Графы. Поиск кратчайшего пути. Флойд

## Демонстрация и связь с компилятором

Флойд—Уоршелл



Флойд—Уоршелл

### Флойд—Уоршелл

Шаг демонстрации: Шаг = инициализация матрицы, смена промежуточной вершины  $k$  или улучшение  $dist[i][j]$  — как в демонстрации слева.

- $n$  — число вершин — размер квадратной матрицы расстояний  $n \times n$  (7 на демо)

### Флойд—Уоршелл

- $k$  — промежуточная вершина — ВНЕШНИЙ цикл; на шаге  $k$  разрешено ходить через вершины  $0 \dots k$  ( $0 \dots n-1$ )
- $i$  — начало пути — строка матрицы (средний цикл) ( $0 \dots n-1$ )
- $j$  — конец пути — столбец матрицы (внутренний цикл) ( $0 \dots n-1$ )
- $\text{dist}[i][j]$  — кратчайший путь  $i \rightarrow j$  среди уже проверенных промежуточных вершин (обновляется на каждом улучшении)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Графы. Поиск кратчайшего пути. Ускорения

## Поиск кратчайшего пути: ускорения

### Зачем ускорять то, что и так работает

Строгое правило / Формула:

Ускорение = та же задача, меньше работы: меньше релаксаций (потенциалы,  $A^*$ ), меньше посещённых вершин (встречный поиск, ранний выход), меньше структур (0-1 BFS, Dial) или перенос работы в предобработку (ALT, contraction hierarchies).

Дейкстра (билет 14), Форд—Беллман (билет 15) и Флойд (билет 16) решают задачу честно и в лоб. На экзамене спрашивают, что сделать, чтобы то же самое посчиталось быстрее. Ниже — весь набор приёмов: от «одной строчки перевзвешивания» до промышленных ускорителей для карт.

Аналогия 1: Такси с навигатором ( $A^*$ )

Дейкстра — это таксист без карты: он честно расширяет зону поиска во все стороны одинаково.  $A^*$  — тот же таксист, но с навигатором: к уже накрученному счётчику  $g(v)$  прибавляется оценка остатка пути по прямой  $h(v)$ , и в первую очередь едут туда, где сумма меньше. Навигатор обязан не завышать остаток ( $h \leq$  истинного): занижать можно, завышать нельзя — иначе маршрут окажется не самым дешёвым. При  $h = 0$  это снова Дейкстра.

Аналогия 2: Две бригады туннеля (встречный поиск)

Нужен путь из  $A$  в  $B$ . Вместо одной бригады, которая роет от  $A$  во все стороны, запускают две: от  $A$  и от  $B$ . Каждая зона роста вдвое меньше, поэтому работа заканчивается раньше. Важная тонкость: останавливаются не в момент встречи, а когда сумма лучших неподтверждённых гипотез с двух сторон уже не может улучшить найденный путь — иначе можно пропустить более дешёвый обход. В графе с отрицательными рёбрами приём ломается: там «встреча» ничего не гарантирует.

| Приём                 | Что экономит                                           | Сложность                                                | Условие применения                                 |
|-----------------------|--------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------|
| Джонсон (потенциалы)  | разрешает Дейкстру при отрицательных рёбрах, все пары  | $O(V \cdot E \cdot \log V)$                              | нет отрицательных циклов                           |
| Ранний выход          | не обходим весь граф ради одной пары                   | та же, но меньше константа                               | нужен путь до одной цели                           |
| Встречный Дейкстра    | две маленькие волны вместо одной большой               | $\approx O(E \log V)$ на практике вдвое меньше раскрытий | известны обе вершины, граф «широкий»               |
| $A^*$                 | раскрывает вершины в сторону цели                      | $O(E \log V)$ , раскрытий меньше                         | есть допустимая эвристика $h(v)$                   |
| 0-1 BFS               | убирает кучу целиком                                   | $O(V + E)$                                               | веса только 0 и 1                                  |
| Dial (корзины)        | куча $\rightarrow$ массив корзин                       | $O(E + V \cdot C)$                                       | целые веса $0 \dots C$ , $C$ небольшое             |
| Куча Фибоначчи        | decrease-key за $O(1)$ амортизированно                 | $O(E + V \log V)$                                        | теоретически; на практике двоичная/d-арная быстрее |
| SPFA (очередь)        | не релаксируем все рёбра $V - 1$ раз, а только «живые» | $O(V \cdot E)$ в худшем, на практике быстро              | отрицательные веса, разреженные графы              |
| ALT / CH / hub labels | переносит работу в предобработку                       | запрос $\ll O(E)$                                        | статический граф, миллионы запросов (карты)        |

## 1. Джонсон: как перестать бояться минусов и полюбить Дейкстру

$$w'(u, v) = w(u, v) + h(u) - h(v)$$

☞ Суть перевзвешивания

Дейкстра (билет 14) быстрая, но ломается на отрицательных весах. Форд—Беллман (билет 15) минусов не боится, но медленный. Алгоритм Джонсона делает финт: переделывает веса всех рёбер так, чтобы они стали  $\geq 0$ , а кратчайшие пути остались теми же.

1. Добавляем фиктивную вершину  $s$  и проводим от неё рёбра веса 0 во все остальные вершины.
2. Запускаем от  $s$  один Форд—Беллман. Он даёт расстояния  $h(v)$  — это наши потенциалы. Заодно он ловит отрицательный цикл: если цикл есть, Джонсон неприменим, и это надо сказать вслух.
3. Перевзвешиваем:  $w'(u, v) = w(u, v) + h(u) - h(v) \geq 0$  (неравенство следует из того, что  $h(v) \leq h(u) + w(u, v)$  — это ровно релаксация, которая уже сошлась).
4. Магия: для любого пути потенциалы внутри сокращаются телескопически — новый вес пути = старый +  $h(\text{старт}) - h(\text{финиш})$ , то есть все пути между фиксированной парой сдвигаются на одну константу. Значит порядок «кто короче» не меняется.
5. Запускаем быструю Дейкстру  $V$  раз (от каждой вершины) по новым весам и возвращаем ответ обратно:  $d(u, v) = d'(u, v) - h(u) + h(v)$ .

Итого для всех пар:  $O(V \cdot E)$  на Форд—Беллман +  $O(V \cdot E \cdot \log V)$  на  $V$  Дейкстр. На разреженном графе ( $E \approx V$ ) это  $O(V^2 \log V)$  против  $O(V^3)$  у Флойда — вот почему Джонсон выбирают для больших разреженных графов, а Флойд — для маленьких или очень плотных.

## 2. Встречный (двунаправленный) Дейкстра

Если известны обе вершины — старт и финиш, — одну волну заменяют двумя: вперёд из  $s$  и назад из  $t$  (по перевёрнутым рёбрам). Волны растут навстречу, и вместо круга радиуса  $R$  приходится рисовать два круга радиуса  $R/2$ : в «широком» графе (дорожная сеть) это экономия в разы. Ключевая деталь — условие остановки: нельзя останавливаться, когда волны «встретились» в одной вершине; надо хранить  $\mu$  = лучший найденный путь ( $\min$  по  $d\_f[v] + d\_b[v]$ ) и останавливаться, когда  $\min(\text{куча\_f}) + \min(\text{куча\_b}) \geq \mu$ . Иначе можно отрезать ещё не раскрытый, но более короткий путь.

## 3. A\*: Дейкстра с компасом

Дейкстра раскрывает вершины ровным кругом: ей всё равно, где цель. A\* сортирует кучу не по  $g(v)$  (уже пройденное расстояние от старта), а по  $f(v) = g(v) + h(v)$ , где  $h(v)$  — оценка остатка до цели. На карте  $h$  — евклидово расстояние по координатам, «по прямой». Два требования: допустимость ( $h(v) \leq$  истинного остатка — никогда не переоцениваем) гарантирует оптимальность ответа; монотонность/согласованность ( $h(u) \leq w(u, v) + h(v)$ ) гарантирует, что вершину не придётся раскрывать повторно. При  $h \equiv 0$  A\* вырождается ровно в Дейкстру — это хороший способ запомнить, что A\* не «другой алгоритм», а Дейкстра с приоритетом.

## 4. Когда куча не нужна: 0-1 BFS и Dial

Если веса рёбер только 0 и 1 (например, «бесплатный переход» против «платного»), кучу заменяют деком: ребро веса 0 кладем в начало, веса 1 — в конец. Дека всегда отсортирована по расстоянию (в ней одновременно живут максимум два уровня), поэтому получаем честный BFS за  $O(V + E)$ . Обобщение — алгоритм Дила (корзины) для целых весов  $0 \dots C$ : держим  $C + 1$  корзину по модулю, идём по кругу, сложность  $O(E + V \cdot C)$ . Оба приёма — стандартный ответ на вопрос «как ускорить Дейкстру, если веса маленькие целые».

## 5. SPFA и выбор кучи

Форд—Беллман можно не прогонять «все рёбра  $V - 1$  раз», а держать очередь вершин, у которых расстояние только что уменьшилось — это SPFA (очередной Беллман—Форд). На реальных графах она в разы быстрее классического прогона, но асимптотика та же,  $O(V \cdot E)$ , и существуют графы, на которых SPFA деградирует (поэтому на соревнованиях её используют осторожно, с эвристиками SLF/LLL). Отдельная линия ускорения — сама куча: двоичная даёт  $O(E \log V)$ , d-арная ( $d = 4 \dots 16$ ) уменьшает высоту и на практике быстрее, куча Фибоначчи даёт теоретические  $O(E + V \log V)$ , но константа убивает весь выигрыш.

## 6. Карты: сжатие координат, ALT, contraction hierarchies

Навигатор считает миллионы запросов по одному и тому же графу, поэтому работу переносят в предобработку. Сначала — сжатие координат: долготы и широты это большие дробные числа, их заменяют плотными индексами  $0 \dots N$ , и уже по ним строят граф и сетки. Дальше — ALT (A\*, Landmarks, Triangle inequality): заранее считают расстояния от нескольких «ориентиров» до всех вершин и из неравенства треугольника получают отличную допустимую эвристику без геометрии. Contraction hierarchies (так устроен OSRM): вершины по важности «сворачивают», добавляя короткие shortcut-рёбра, и запрос идёт двунаправленным поиском вверх по иерархии — микросекунды вместо секунд. Hub labeling идёт дальше: каждой вершине заранее приписывают «метки», и запрос сводится к пересечению двух списков.

Ты путаешь:

- A\* с недопустимой эвристикой ( $h$  больше истинного остатка) находит путь быстро, но не кратчайший. Переоценка = неверный ответ; недооценка = безопасно.
- Джонсон не «ускоряет Дейкстру» на графе без минусов: он делает Дейкстру применимой к отрицательным весам и даёт все пары. На неотрицательных весах  $h \equiv 0$  и перевзвешивание ничего не меняет.
- Встречный Дейкстра останавливается не «когда волны встретились», а когда  $\min_f + \min_b \geq \mu$ . Остановка по встрече даёт неверный ответ на графах с неравными весами.
- 0-1 BFS не работает для весов 0, 1, 2: в деке окажется три уровня, и порядок сломается. Там нужен Dial.
- SPFA асимптотически не быстрее Форда—Беллмана — это та же  $O(V \cdot E)$ , просто на практике меньше бесполезных релаксаций.

Сложность — явно: Джонсон  $O(V \cdot E \cdot \log V)$ , встречный Дейкстра и A\*  $O(E \log V)$  с меньшим числом раскрытий, 0-1 BFS  $O(V + E)$ , Dial  $O(E + V \cdot C)$ , Дейкстра с кучей Фибоначчи  $O(E + V \log V)$ , SPFA  $O(V \cdot E)$  в худшем случае.

► Код: Джонсон целиком (Скрыто)

```
import heapq
INF = float("inf")

def johnson(n, edges):
    # edges: (u, v, w)
    # 1) потенциалы: Форд—Беллман от фиктивной вершины s = n
    h = [0] * (n + 1)
    for _ in range(n):
        # из s все рёбра веса 0 → h(s)=0, h(v)=0 старт
        # V итераций: V-1 на расстояния + 1 на проверку
        changed = False
        for u, v, w in edges:
            if h[u] + w < h[v]:
                h[v] = h[u] + w
                changed = True
        if not changed:
            break
    else:
        return None
    # отрицательный цикл — Джонсон неприменим

    # 2) перевзвешивание: все w' ≥ 0
    g = [[] for _ in range(n)]
    for u, v, w in edges:
        g[u].append((v, w + h[u] - h[v]))

    # 3) Дейкстра от каждой вершины + обратный перевод
    d = [[INF] * n for _ in range(n)]
```

```

for s in range(n):
    d[s][s] = 0
    heap = [(0, s)]
    while heap:
        dist, v = heapq.heappop(heap)
        if dist > d[s][v]:
            continue
        for to, w in g[v]:
            if d[s][to] > dist + w:
                d[s][to] = dist + w
                heapq.heappush(heap, (d[s][to], to))
    for v in range(n):
        if d[s][v] < INF:
            d[s][v] = d[s][v] - h[s] + h[v]      # обратно к настоящим весам
return d

```

► Код: встречный Дейкстра, A\*, 0-1 BFS (Скрыто)

```

# --- Встречный Дейкстра: остановка по min_f + min_b >= mu ---
def bidirectional(g, rg, s, t):      # rg — граф с перевёрнутыми рёбрами
    INF = float("inf")
    df = {s: 0}; db = {t: 0}
    qf = [(0, s)]; qb = [(0, t)]
    mu = INF
    while qf and qb:
        if qf[0][0] + qb[0][0] >= mu:    # вот правильное условие остановки
            return mu
        dist, v = heapq.heappop(qf)
        if dist > df.get(v, INF): continue
        for to, w in g[v]:
            nd = dist + w
            if nd < df.get(to, INF):
                df[to] = nd; heapq.heappush(qf, (nd, to))
            if to in db:
                mu = min(mu, nd + db[to]) # встреча: кандидат в ответ
        dist, v = heapq.heappop(qb)
        if dist > db.get(v, INF): continue
        for to, w in rg[v]:
            nd = dist + w
            if nd < db.get(to, INF):
                db[to] = nd; heapq.heappush(qb, (nd, to))
            if to in df:
                mu = min(mu, nd + df[to])
    return mu

# --- A*: приоритет f = g + h ---
def astar(g, s, t, h):              # h(v) — допустимая эвристика, h(t) = 0
    INF = float("inf")
    gdist = {s: 0}
    heap = [(h(s), 0, s)]           # (f, g, v)
    closed = set()
    while heap:
        f, gv, v = heapq.heappop(heap)
        if v == t:
            return gv                # ранний выход: цель раскрыта → ответ готов
        if v in closed:
            continue
        closed.add(v)
        for to, w in g[v]:
            nd = gv + w
            if nd < gdist.get(to, INF):
                gdist[to] = nd
                heapq.heappush(heap, (nd + h(to), nd, to))
    return INF

# --- 0-1 BFS: веса 0 и 1, дека вместо кучи ---
from collections import deque
def bfs01(g, s, n):                 # g[v] = [(to, w)], w ∈ {0, 1}
    INF = float("inf")
    d = [INF] * n; d[s] = 0
    dq = deque([s])
    while dq:

```

```

v = dq.popleft()
for to, w in g[v]:
    if d[v] + w < d[to]:
        d[to] = d[v] + w
        (dq.appendleft(to) if w == 0 else dq.append(to))
return d

```

#### ► Вопросы, которые любят задавать (Скрыто)

- Почему после перевзвешивания веса неотрицательны? —  $h(v) \leq h(u) + w(u, v)$ , потому что Форд—Беллман сошёл: это и есть условие отсутствия улучшающей релаксации.
- Почему кратчайшие пути не меняются? — потенциалы на пути сокращаются телескопически: сдвиг равен  $h(\text{старт}) - h(\text{финиш})$  и одинаков для всех путей между этой парой.
- Что лучше для всех пар: Флойд или Джонсон? — Флойд  $O(V^3)$  хорош при плотных графах и маленьком  $V$ ; Джонсон  $O(V \cdot E \cdot \log V)$  выигрывает на разреженных.
- Какая эвристика  $A^*$  допустима на карте? — евклидово расстояние по прямой (никогда не длинее дороги); при ограничении скорости можно делить на максимальную скорость.
- Зачем нужен ранний выход в Дейстре? — как только цель извлечена из кучи, её расстояние финализировано: продолжать смысла нет.
- Почему 0-1 BFS работает без кучи? — в деке лежат вершины максимум двух соседних уровней расстояния, то есть она уже отсортирована.

## Демонстрация и связь с компилятором

### Алгоритм Джонсона

Демонстрация: Алгоритм Джонсона
Развернуть
Код ↔ визуализация

встроенное демо
сценарий: Алгоритм Джонсона

● Визуализация: Перевзвешивание алгоритмом Джонсона



**Исходный граф.** Имеется ребро отрицательного веса ( $A \rightarrow B = -2$ ). Если мы запустим быстрый алгоритм Дейкстры, он отработает неверно и сломается на этом ребре.

↺
↻
Далее ▷

Алгоритм Джонсона

#### Алгоритм Джонсона (перевзвешивание)

Шаг демонстрации: Шаг = Беллман-Форд из фиктивной вершины считает  $h[v]$ , затем  $n$  запусков Дейкстры в новых весах.

- $h[v]$  — потенциал вершины — кратчайший путь от фиктивной вершины; делает веса неотрицательными (после Беллмана-Форда)
- $u, v$  — ребро  $(u, v)$  перевзвешивается:  $w'(u, v) = w(u, v) + h[u] - h[v]$  (все рёбра)
- $d[v]$  — расстояние Дейкстры в новых весах; истинный ответ:  $d[v] - h[\text{start}] + h[v]$  (обратное перевзвешивание)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## 20. Графы. Остовное дерево: Краскал, Прима, Борувка

Билеты 18 · 19 · 20

### Остовное дерево: Краскал, Прима, Борувка

#### Что вообще строим

Строгое правило / Формула:

Остовное дерево — подграф, в котором все  $V$  вершин, ровно  $V - 1$  ребро, нет циклов и он связан. MST — такое остовное дерево, у которого сумма весов рёбер минимальна.

Все три алгоритма — жадные, и все три опираются на одну теорему. Свойство разреза (cut property): разрежем вершины на две части как угодно; самое лёгкое ребро, пересекающее разрез, входит в некоторое MST. Алгоритмы отличаются только тем, какой разрез они выбирают на каждом шаге: Краскал — «одна вершина против всех», Прим — «выросшее дерево против остального графа», Борувка — «каждая компонента против всех сразу».

Второй инструмент — свойство цикла (cycle property): самое тяжёлое ребро в цикле никогда не входит в MST. Именно его использует Краскал, когда отбрасывает ребро, замыкающее цикл: раз ребро самое тяжёлое в образовавшемся цикле (рёбра-то мы берём по возрастанию), оно лишнее.

| Алгоритм     | Идея одним предложением                                      | Главная структура      | Сложность                                                                  | Когда брать                                                    |
|--------------|--------------------------------------------------------------|------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------|
| 18 · Краскал | Рёбра по возрастанию веса; берём, если не замыкает цикл      | сортировка + DSU       | $O(E \log E)$                                                              | рёбра даны списком, граф разреженный, нужен простой код        |
| 19 · Прим    | Растём одним связным куском: самое лёгкое ребро наружу       | куча (priority queue)  | $O(E \log V)$ , с кучей<br>Фибоначчи $O(E + V \log V)$ , без кучи $O(V^2)$ | плотный граф, граф задан списками смежности, нужен один «куст» |
| 20 · Борувка | Все компоненты одновременно тянут своё cheapest-ребро наружу | DSU + проход по рёбрам | $O(E \log V)$                                                              | параллельные/GPU-реализации, граф-матрица, много компонент     |

Аналогия к билету 18 · Краскал: прокладка кабеля

Связать все города интернетом. Краскал говорит: «строим с самых дешёвых дорог страны». По всей карте растут независимые кусочки сети (лес), DSU следит, кто уже с кем связан, — и в конце куски сливаются в единое дерево. Порядок рёбер важен, порядок вершин — нет.

✂ Аналогия к билету 19 · Прим: заражение вирусом

Прим ведёт себя как ползучая грибница: стартуем из одного города, и «щупальце» всегда захватывает ближайший непосещённый город. Сеть всё время одна связная — никакого леса. Разрез здесь ровно один: «внутри дерева» против «снаружи».

Аналогия к билету 20 · Борувка: племена

Сначала каждый город — отдельное племя. В первый день каждое племя шлёт гонца к ближайшему чужому племени: к вечеру компонент вдвое меньше. На второй день уже царства шлют гонцов к ближайшему чужому царству. За  $\log_2 V$  дней образуется единая федерация — поэтому  $O(E \log V)$ , а по рёбрам мы ходим «все сразу».

Ты путаешь:

- MST и кратчайшие пути — разные задачи. Дейкстра (билет 14) минимизирует расстояние от одной вершины до каждой; MST минимизирует суммарный вес дерева. Дерево Прима похоже на дерево Дейкстры, но ключ в куче другой: у Прима — вес ребра к дереву, у Дейкстры — расстояние от старта. На графе с одним «длинным» ребром ответы расходятся.
- «MST единственно» — единственно только при попарно различных весах. При равных весах деревьев может быть несколько, но их суммарный вес всегда один.
- Краскал без DSU превращается в  $O(E \cdot V)$ : проверка «не замыкает ли цикл» обходом вместо почти-константного find/union.
- Рёбер в ответе  $V - 1$ , а не  $V$ . Если в конце их меньше — граф был несвязный, и Краскал построил минимальный остовный лес.
- Боровка: два племени могут выбрать друг друга одновременно — ребро добавится один раз, DSU спасает от дубля ( $\text{find}(u) == \text{find}(v) \rightarrow \text{пропускаем}$ ).

Сложность — явно: Краскал  $O(E \log E)$  (сортировка доминирует, DSU почти бесплатный); Прим с двоичной кучей  $O(E \log V)$ , на плотном графе проще взять версию без кучи  $O(V^2)$ ; Боровка  $O(E \log V)$  —  $\log V$  фаз по проходу всех рёбер. Память у всех  $O(V + E)$ .

#### ► Код 18: Краскал + DSU (Скрыто)

```
vector<pair<int, pair<int,int>>> edges; // (вес, (u, v))
vector<int> p, rnk;

int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); } // сжатие пути
bool unite(int a, int b) { // union by rank
    a = find(a); b = find(b);
    if (a == b) return false; // уже в одной компоненте → цикл
    if (rnk[a] < rnk[b]) swap(a, b);
    p[b] = a;
    if (rnk[a] == rnk[b]) rnk[a]++;
    return true;
}

sort(edges.begin(), edges.end()); //  $O(E \log E)$ 
long long cost = 0; int taken = 0;
for (auto [w, uv] : edges) {
    if (unite(uv.first, uv.second)) { // не замыкает цикл → берём
        cost += w;
        if (++taken == n - 1) break; //  $V - 1$  ребро — дерево готово
    }
}
```

#### ► Код 19: Прим с кучей (Скрыто)

```
import heapq

INF = float("inf")
key = [INF] * n # минимальный вес ребра, ведущего в вершину из дерева
used = [False] * n
key[0] = 0
heap = [(0, 0)] # (вес ребра, вершина) ← вот тут отличие от Дейкстры
total = 0

while heap:
    w, v = heapq.heappop(heap)
    if used[v]:
        continue # устаревшая запись в куче
    used[v] = True
    total += w
    for to, weight in graph[v]:
        if not used[to] and weight < key[to]:
            key[to] = weight
            heapq.heappush(heap, (weight, to))
print(total)
```

Сравните с Дейкстрой: там  $\text{dist}[to] = \text{dist}[v] + \text{weight}$  (накапливаем путь от старта), здесь  $\text{key}[to] = \text{weight}$  (только вес последнего ребра). Одна строка — другая задача.

### ► Код 20: Борувка (Скрыто)

```
parent = list(range(n))
def find(v):
    while parent[v] != v:
        parent[v] = parent[parent[v]]
        v = parent[v]
    return v

comps = n
total = 0
while comps > 1:
    best = {}
    for w, u, v in edges:
        cu, cv = find(u), find(v)
        if cu == cv:
            continue
        if cu not in best or w < best[cu][0]:
            best[cu] = (w, u, v)
        if cv not in best or w < best[cv][0]:
            best[cv] = (w, u, v)
    if not best:
        break
    for w, u, v in best.values():
        if find(u) == find(v):
            continue
        parent[find(u)] = find(v)
        comps -= 1
        total += w
print(total)
```

За фазу компонент становится минимум вдвое меньше (каждое слияние убирает хотя бы одну), значит фаз не больше  $\log_2 V$ .

### ► Вопросы, которые любят задавать (Скрыто)

- Почему жадность здесь работает? — cut property: самое лёгкое ребро разреза безопасно добавлять; доказательство обменом ребра в предполагаемом MST.
- Чем Прим отличается от Дейкстры? — ключом в куче (вес ребра против расстояния от старта) и целью (дерево минимального веса против дерева кратчайших путей).
- Какой алгоритм лучше на плотном графе ( $E \approx V^2$ )? — Прим без кучи,  $O(V^2)$ : сортировка E рёбер у Краскала дороже.
- Зачем нужна Борувка, если есть Краскал? — она естественно параллельная: cheapest-рёбра для всех компонент ищутся независимо, поэтому её любят на GPU и в распределённых системах.
- Что если граф несвязный? — все три построят минимальный остовный лес; рёбер будет  $V$  — (число компонент).
- Как найти второе по величине остовное дерево? — для каждого ребра не из MST добавляем его и убираем максимальное ребро на образовавшемся цикле (LCA + максимум на пути).

## Демонстрация и связь с компилятором

Остовное дерево: Краскал, Прима, Борувка

Три алгоритма (билеты 18–20) на одном графе из 9 вершин: вкладка переключает и стратегию роста остова, и код в панели Python. Переменные parent/mst/taken/u/v/w перекрашивают граф напрямую.

Демонстрация: Остовное дерево: Краскал, Прима, Борука

Развернуть
Код > визуализация

Эти алгоритмы (билеты 18-20) на одном графе из 9 вершин: вкладка переключает и стратегию роста остова, и код в панели Python. Переменные parent/mstTaken/vis переключают граф напредую.

встроенные данные: [циклический Остовное дерево: Краскал, Прима, Борука](#)

Учебное пособие
Python
Скачать PDF
Еще

Наблюдая за работой Краскала (Раскраска), Прима (Плессень) и Боруки (Коллективизация) на общем графе из 9 вершин.

Сбросить
Начать

из КОМПЬЮТЕРА: открыть панель Python и запустить код вкладок «Борука»; переменные parent, vis, u, v и переключат этот граф

Бесцветный
Колхоз 1
Колхоз 2
Агрооимплекс

Вит 8 / 5

Ход выполнения

Введение в Коллективизацию (Борука)

Логика «Борука»: Каждая из 9 деревьев параллельно и одновременно ищет свою кратчайшую дорогу к соседней деревне, чтобы объединиться в колхоз.

Шпаргалка: Сравнение кода и сложности

Краскал (Билет 18)
Прима (Билет 19)
Борука (Билет 20)

СЛОЖНОСТЬ ВРЕМЕНИ
 $O(E \log E)$

Обусловлено сортировкой всех ребер графа перед началом объединения.

СЛОЖНОСТЬ ПАМЯТИ
 $O(V + E)$

Для хранения массива ребер и структуры DSU (предок-ребро).

Суть в одном предложении:

Сортируем все ребра от самого дешевого к дороговому, берем их по очереди и красим, вершины в один цвет, образуя циклы.

РЕАЛИЗАЦИЯ НА PYTHON (КРАТКО)

```

# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Массивный выбор ребер
edges.sort() # O(E log E)
mst = []
for u, v, w in edges:
    if union(u, v):
        mst.append((u, v, w))

```

## Остовное дерево: Краскал, Прима, Борука · режим «борука»

Демонстрация: Остовное дерево: Краскал, Прима, Борука

Развернуть
Код > визуализация

Эти алгоритмы (билеты 18-20) на одном графе из 9 вершин: вкладка переключает и стратегию роста остова, и код в панели Python. Переменные parent/mstTaken/vis переключают граф напредую.

встроенные данные: [циклический Остовное дерево: Краскал, Прима, Борука](#)

Учебное пособие
Python
Скачать PDF
Еще

Наблюдая за работой Краскала (Раскраска), Прима (Плессень) и Боруки (Коллективизация) на общем графе из 9 вершин.

Сбросить
Начать

из КОМПЬЮТЕРА: открыть панель Python и запустить код вкладок «Борука»; переменные parent, vis, u, v и переключат этот граф

Бесцветный
Клан 1
Клан 2
Итог

Вит 8 / 22

Ход выполнения

Введение в раскраску (Краскал)

Представьте, что все 9 деревен графа — бесцветны. Мы выписали ребра по возрастанию веса. Готовы начинать!

Шпаргалка: Сравнение кода и сложности

Краскал (Билет 18)
Прима (Билет 19)
Борука (Билет 20)

СЛОЖНОСТЬ ВРЕМЕНИ
 $O(E \log E)$

Обусловлено сортировкой всех ребер графа перед началом объединения.

СЛОЖНОСТЬ ПАМЯТИ
 $O(V + E)$

Для хранения массива ребер и структуры DSU (предок-ребро).

Суть в одном предложении:

Сортируем все ребра от самого дешевого к дороговому, берем их по очереди и красим, вершины в один цвет, образуя циклы.

РЕАЛИЗАЦИЯ НА PYTHON (КРАТКО)

```

# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Массивный выбор ребер
edges.sort() # O(E log E)
mst = []
for u, v, w in edges:
    if union(u, v):
        mst.append((u, v, w))

```

## Остовное дерево: Краскал, Прима, Борука · режим «краскал»

Универсальное пособие · алгоритмы и структуры данных

· Графы. Остовное дерево: Краскал, Прима, Борука · 78

**Демонстрация: Основное дерево: Краскал, Прима, Борувка** [Развернуть](#) [Код > визуализация](#)

Для алгоритма (билеты 18-20) на одном графе из 9 вершин: вкладка переключает и стратегию роста остова, и код в панели Python. Переменные parent/mstTaken/vis/mst переключают граф на текущую.

**Основное пособие** [Учебное пособие](#) [Python](#) [Скачать PDF](#) [Ещё](#)

Наблюдая за работой Краскала (Раскраска), Прима (Плесень) и Борувки (Коллективизация) на общем графе из 9 вершин.

... из КОМПЬЮТЕРА, второй панель Python и запуска код выкладки «Прима», переменные parent, mst, vis, m переключают этот граф.

**Вклад:** [Бесцветный](#) [В кучу \(Heap\)](#) [Плесень](#) Шаг: 0 / 23

**Ход выполнения**

**ПРИОРИТЕТНАЯ СЧЕРЕДЬ (MIN-HEAP):**  
Очередь: пуста

**Введение в Плесень (Прима)**  
Плесень «Плесень» выбирает стартовую вершину (Центр Токио - E). Смотрим на исходящие ребра через Приоритетную очередь: (Heap) и закатываем соседей!

**Шпаргалка: Сравнение кода и сложности** [Краскал \(Билет 18\)](#) [Прима \(Билет 19\)](#) [Борувка \(Билет 20\)](#)

**СЛОЖНОСТЬ ВРЕМЕНИ**  
 **$O(E \log E)$**   
Обусловлено сортировкой всех ребер графа перед началом объединения.

**СЛОЖНОСТЬ ПАМЯТИ**  
 **$O(V + E)$**   
Для хранения массива ребер и структуры DSU (предок-ребро).

**Суть в одном предложении:**  
Сортируем все ребра от самого дешевого к дороговому, берем их по очереди и красим, вершины в один цвет, образуя циклы.

**РЕАЛИЗАЦИЯ НА PYTHON (КРАТКО)**

```
# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Массив ребер
edges.sort() # O(E log E)
mst = []
for u, v, w in edges:
    if union(u, v):
        mst.append((u, v, w))
```

## Основное дерево: Краскал, Прима, Борувка · режим «прим»

**Демонстрация: Основное дерево: Краскал, Прима, Борувка** [Развернуть](#) [Код > визуализация](#)

Для алгоритма (билеты 18-20) на одном графе из 9 вершин: вкладка переключает и стратегию роста остова, и код в панели Python. Переменные parent/mstTaken/vis/mst переключают граф на текущую.

**Основное пособие** [Учебное пособие](#) [Python](#) [Скачать PDF](#) [Ещё](#)

Наблюдая за работой Краскала (Раскраска), Прима (Плесень) и Борувки (Коллективизация) на общем графе из 9 вершин.

... из КОМПЬЮТЕРА, второй панель Python и запуска код выкладки «Краскал», переменные parent, mst, vis, m переключают этот граф.

**Вклад:** [Бесцветный](#) [Класс 1](#) [Класс 2](#) [Итого](#) Шаг: 0 / 22

**Ход выполнения**

**Введение в раскраску (Краскал)**  
«Краскал» — это раскраска! Представьте, что все 9 вершин графа — бесцветные. Мы выписали ребра по возрастанию веса. Готовы начинать!

**Шпаргалка: Сравнение кода и сложности** [Краскал \(Билет 18\)](#) [Прима \(Билет 19\)](#) [Борувка \(Билет 20\)](#)

**СЛОЖНОСТЬ ВРЕМЕНИ**  
 **$O(E \log E)$**   
Обусловлено сортировкой всех ребер графа перед началом объединения.

**СЛОЖНОСТЬ ПАМЯТИ**  
 **$O(V + E)$**   
Для хранения массива ребер и структуры DSU (предок-ребро).

**Суть в одном предложении:**  
Сортируем все ребра от самого дешевого к дороговому, берем их по очереди и красим, вершины в один цвет, образуя циклы.

**РЕАЛИЗАЦИЯ НА PYTHON (КРАТКО)**

```
# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Массив ребер
edges.sort() # O(E log E)
mst = []
for u, v, w in edges:
    if union(u, v):
        mst.append((u, v, w))
```

## Основное дерево: Краскал, Прима, Борувка

### Основное дерево: Краскал и DSU

Шаг демонстрации: Шаг = следующее ребро из отсортированных: если  $\text{find}(u) \neq \text{find}(v)$ , ребро идёт в остов, иначе отбрасывается.

- $n$  — число вершин (A...I); в остов войдёт ровно  $n-1 = 8$  рёбер ( $n = 9$ )
- edges — список рёбер (вес,  $u$ ,  $v$ ) — тот же граф из 9 вершин, что нарисован слева (12 рёбер)
- $i$  — номер текущего ребра в отсортированном списке (0 ... 11)
- $u$ ,  $v$ ,  $w$  — концы и вес ребра, которое рассматривается сейчас — подсвечивается на графе (A...I)

### Остовное дерево: Краскал и DSU

- parent — DSU: представитель множества вершины. Визуализация красит компоненты по корням find(v) (A...I)
- rank — высота дерева DSU — union by rank ( $\leq \log n$ )
- mst — взятые рёбра остова — рисуются жирными ( $\leq 8$  рёбер)
- total — вес остова; на этом графе всегда 51 (веса различны => остов единственен) (0 ... 51)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «kruskal» — Остовное дерево: Краскал и DSU

Шаг демонстрации: Шаг = следующее ребро из отсортированных: find(u)  $\neq$  find(v) => берём в остов.

- i — номер текущего ребра — перематывает кадры демонстрации (0 ... 11)
- u, v, w — концы и вес рассматриваемого ребра — жёлтая подсветка (A...I)
- parent — DSU-массив: цвета компонент на графе берутся из find(v) (A...I)
- mst — рёбра, уже взятые в остов ( $\leq 8$ )
- total — вес остова (51 на этом графе) (0 ... 51)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «prim» — Остовное дерево: Прим (рост из вершины E)

Шаг демонстрации: Шаг = из кучи достаётся самое лёгкое ребро, ведущее из выращенного дерева наружу.

- start — вершина, из которой растим дерево (в демо — E, «Центр Токио») (A...I)
- heap — куча рёбер-кандидатов (вес, куда, откуда) — рисуются голубым ( $\leq m$ )
- taken — вершины, уже вошедшие в остов, — красятся «плесенью» ( $\leq 9$ )
- u, v, w — ребро, которое только что взяли: v добавлена в дерево из u (A...I)
- mst / total — рёбра остова и его вес (51) (8 рёбер)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «boruvka» — Остовное дерево: Борувка (фазы-«пятилетки»)

Шаг демонстрации: Шаг = одна фаза: каждая компонента ОДНОВРЕМЕННО выбирает своё самое дешёвое исходящее ребро.

- comp — DSU: корень компоненты вершины — по нему красятся вершины (A...I)
- cheapest — корень компоненты -> самое дешёвое ребро наружу (выбор этой фазы) ( $\leq$  число компонент)
- phase — номер фазы; на этом графе остов собирается за 2 фазы (1 ...  $\lceil \log n \rceil$ )
- u, v, w — ребро-кандидат, которое компонента рассматривает сейчас (A...I)
- mst / total — взятые рёбра и вес остова (51) (8 рёбер)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Алгоритм Кнута-Морриса-Пратта (КМП)

## Алгоритм Кнута—Морриса—Пратта: префикс-функция и поиск без отката

### Фундамент поиска КМП

Строгое правило / Формула:

$\pi[i]$  — максимальная длина такого префикса строки, который также является её суффиксом, оканчивающимся в позиции  $i$ .

♂ Аналогия 1: Поиск без отката

Мы идём по строке слева направо. В каждой позиции  $i$  мы спрашиваем: «Какой самый длинный кусок из начала всей строки СЕЙЧАС закончился под моим пальцем?» Представь, ты ищешь в тексте слово `abacaba`. Ты дошел до `abacab`, и следующая буква — `x`. Тупой алгоритм начнет искать заново со второй буквы. КМП посмотрит и скажет: «Спокойно: у совпавшего куска `abacab` суффикс `ab` — это ровно начало нашего слова,  $\pi = 2$ . Не перечитываем текст, а продолжаем сравнение с 3-й буквы шаблона». Это поиск за  $O(N)$  без возвратов.

Аналогия 2: LLM стриминг

Для LLM, которая стримит токены по одному, это крутой способ искать запрещенку, не перелопачивая весь контекст на каждом шаге. Если мы частично совпали с запрещённым словом, но потом пришёл не тот токен,  $\pi$ -функция мгновенно говорит нам, с какого места этого слова мы можем продолжить поиск, не возвращаясь назад в потоке!

► *Пример вычисления (Скрыто)*

Тот же пример: `abacabcd`

- `a` : Префиксов нет.  $\pi[0] = 0$
- `ab` : Из начала ("`a`") ничего не совпало с концом ("`b`").  $\pi[1] = 0$
- `abc` : Опять мимо.  $\pi[2] = 0$
- `abca` : О! Начало ("`a`") совпало с текущим концом ("`a`"). Длина — 1.  $\pi[3] = 1$
- `abcab` : Начало ("`ab`") совпало с концом ("`ab`"). Длина — 2.  $\pi[4] = 2$
- `abcaabc` : Начало ("`abc`") совпало с концом ("`abc`"). Длина — 3.  $\pi[5] = 3$
- `abacabcd` : «`d`» всё испортила. Пробуем длину 4: начало `abca` против конца `abcd` — не совпало (последняя буква). Длина 3: `abc` против `bcd` — мимо. Длина 1: `a` против `d` — мимо.  $\pi[6] = 0$

► *Код C++ (Скрыто)*

```
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}
```

### 21.2 Сам поиск Кнута—Морриса—Пратта

$j = \pi[j - 1]$  при несовпадении · вхождение найдено, когда  $j == m$  · время  $O(n + m)$

Префикс-функция нужна не сама по себе: она хранит «куда откатываться». Считаем  $\pi$  для шаблона (pattern) один раз, затем идём по тексту, поддерживая  $j$  — длину текущего совпадения шаблона с хвостом текста. Совпало —  $j += 1$ . Не совпало — откатываемся на  $j = \pi[j - 1]$ : мы уже знаем, что хвост совпавшей части равен началу шаблона, поэтому заново сравнивать его не нужно. Когда  $j$  достигает  $m$  — вхождение найдено; чтобы искать дальше (в том числе пересекающиеся вхождения), делаем  $j = \pi[j - 1]$ .

Почему  $O(n + m)$ , а не  $O(n \cdot m)$ : за один символ текста  $j$  растёт максимум на 1, а каждый откат  $j$  уменьшает. Значит суммарно откатов не больше, чем приростов, — итого  $O(n)$  проходов по тексту плюс  $O(m)$  на  $\pi$ . Это классическая амортизационная оценка, её любят спрашивать.

Ты путаешь:

- $\pi[i]$  считается для префикса  $s[0..i]$ , а не для всей строки: это длина наибольшего собственного префикса, равного суффиксу этого префикса (собственного — то есть не равного всему префиксу).
- Откат —  $j = \pi[j - 1]$ , а не  $j = 0$ : обнуление превращает КМР обратно в наивный алгоритм.
- КМР и Z-функция (билет 22) одинаковы по сложности  $O(n + m)$ , но КМР работает онлайн (символы могут приходить потоком), а Z-функция требует всю строку сразу.
- Наименьший период строки — это  $n - \pi[n - 1]$ , а не  $\pi[n - 1]$ . Строка «ababab»:  $\pi[5] = 4$ , период  $6 - 4 = 2$  («ab»).

► Код: поиск подстроки КМР (Скрыто)

```
def prefix_function(s):
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]      # откат по уже посчитанным  $\pi$ 
        if s[i] == s[j]:
            j += 1
        pi[i] = j
    return pi

def kmp_search(text, pattern):
    m = len(pattern)
    pi = prefix_function(pattern)      #  $O(m)$  один раз
    j = 0
    found = []
    for i, ch in enumerate(text):      #  $O(n)$  по тексту
        while j > 0 and ch != pattern[j]:
            j = pi[j - 1]
        if ch == pattern[j]:
            j += 1
        if j == m:
            found.append(i - m + 1)      # позиция вхождения
            j = pi[j - 1]                # ищем дальше, вхождения могут пересекаться
    return found

# тот же результат одной префикс-функцией:  $\pi$  от  $\text{pattern} + \# + \text{text}$ 
def kmp_via_concat(text, pattern):
    s = pattern + '#' + text
    pi = prefix_function(s)
    m = len(pattern)
    return [i - 2 * m for i in range(m + 1, len(s)) if pi[i] == m]
```

Разделитель «#» обязан не входить ни в шаблон, ни в текст: иначе  $\pi$  «склеит» куски через границу и вхождения будут ложными.

## Демонстрация и связь с компилятором

Префикс-функция (КМП)

Демонстрация: Префикс-функция (КМП)

Развернуть
Код → визуализация

встроенное демо

сценарий: Префикс-функция (КМП)

КМП (π-ФУНКЦИЯ)

Z-ФУНКЦИЯ

СЛОВО: aba

ТЕКСТ: abacaba

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| a | b | a | # | a | b | a | c | a | b | a  |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |

Авто

лог выполнения [шаг 1/36]

Начало. Символ 'a'.  $\pi[0] = 0$ .

Обрати внимание на хитрость с решёткой #. Мы пишем **Шаблон** + # + **Текст**. Это разделяет два мира. Когда значения префикс-функции добегают до длины шаблона — это 100% сигнал, что слово было найдено! Пройди шаги вручную, посмотри, как пальцы ищут совпадения без возврата (в КМП) или используют ранее посчитанный блок для пропуска работы (в Z).

КОД C++ (КМП / ПРЕФИКС-ФУНКЦИЯ)

```

vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}

```

Префикс-функция (КМП)

## Префикс-функция (КМП)

Шаг демонстрации: Шаг = обработка символа  $s[i]$ : откаты по  $j = \pi[j-1]$ , пока не совпадёт  $s[j]$  с  $s[i]$ .

- $n$  — длина строки  $s$ , для которой строим префикс-функцию ( $n = \text{len}(s)$ )
- $i$  — индекс текущего символа — считаем  $\pi[i]$ , идём слева направо ( $1 \dots n-1$ )
- $j$  — длина текущего наилучшего совпавшего префикса (начинаем с  $j = \pi[i-1]$ ) ( $0 \dots i$ )
- $\pi[i]$  — префикс-функция: длина наибольшего префикса, который одновременно и суффикс  $s[0 \dots i]$  (заполняется таблица)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

Универсальное пособие · алгоритмы и структуры данных

Алгоритм Кнута-Морриса-Пратта (КМП) · 83

# Z-функция — Сдвиг и наложение

## Z-функция

Строгое правило / Формула:

$Z[i]$  — длина наибольшего общего префикса между самой строкой (начинающейся с индекса 0) и её суффиксом, начинающимся с  $i$ .

Аналогия 1: Линейка-шаблон

Тут мы берём всю строку и «прикладываем» её к самой себе, начиная с каждой позиции  $i$ . Мы спрашиваем: «Если я начну читать строку отсюда, насколько длинный кусок совпадёт с самым началом строки?» Это самый быстрый способ найти все вхождения подстроки в строку. Ты просто пишешь `Pattern + "$" + Text`, считаешь Z-функцию, и там, где  $Z[i]$  равно длине `Pattern` — там и есть вхождение.

Аналогия 2: LLM Самокопипаст

В LLM Z-функция может применяться для дедупликации или детекта заикливания. Если во время генерации вдруг  $Z[i]$  (где  $i$  указывает назад на тот же текст) выстреливает в большое значение — значит, модель начала копипастить сама себя.

### ► Пример вычисления (Скрыто)

Тот же пример: `abccabacd`

- $Z[0]$  — не считается (или длина строки).
- $Z[1]$  ( `bccabacd` vs `abccabacd` ): Первая буква 'b' != 'a'.  $Z[1] = 0$
- $Z[2]$  ( `cabacd` vs `abccabacd` ): 'c' != 'a'.  $Z[2] = 0$
- $Z[3]$  ( `abacd` vs `abccabacd` ):  $s[3]='a'==s[0]='a'$ ,  $s[4]='b'==s[1]='b'$ ,  $s[5]='c'==s[2]='c'$ , а дальше  $s[6]='d'$  против  $s[3]='a'$  — мимо. Совпало 3 буквы.  $Z[3] = 3$
- $Z[4]$ ,  $Z[5]$ ,  $Z[6]$  — всё по нулям, 'b', 'c', 'd' не 'a'.

### ► Код C++ (Скрыто)

```
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
```

## Демонстрация и связь с компилятором

Z-функция

Демонстрация: Z-функция

Развернуть
Код + визуализация

встроенное демо
сценарий: Z-функция

КМП (п-ФУНКЦИЯ)
**Z-ФУНКЦИЯ**

СЛОВО: **aba**
ТЕКСТ: **abacaba**

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|----|
| a | b | a | # | a | b | a | c | a | b | a  |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |

Авто

лог выполнения [шаг 1/61]  
Начало. Z-box [L=0, R=0]

Обрати внимание на хитрость с решёткой **#**. Мы пишем **Шаблон + # + Текст**. Это разделяет два мира. Когда значения Z-функции добегают до длины шаблона — это 100% сигнал, что слово было найдено! Пройди шаги вручную, посмотри, как пальцы ищут совпадения без возврата (в КМП) или используют ранее посчитанный блок для пропуска работы (в Z).

**КОД C++ (Z-ФУНКЦИЯ)**

```

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}

```

## Z-функция

### Z-функция

Шаг демонстрации: Шаг = вычисление  $z[i]$ : внутри Z-блока  $[l, r]$  берём инициализацию из  $z[i-l]$ , потом досчитаем в лоб.

- $n$  — длина строки  $s$  ( $n = \text{len}(s)$ )
- $i$  — позиция, для которой считаем  $z[i]$  ( $1 \dots n-1$ )
- $l, r$  — правый крайний Z-блок: отрезок, совпадающий с префиксом, с максимальным  $r$  ( $l \leq i \leq r$ )
- $z[i]$  — длина наибольшего общего префикса строки и суффикса с позиции  $i$  ( $0 \dots n-i$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Алгоритм Ахо-Корасик

## Множественный поиск

Основная формула:

Бор (Trie) + Суффиксные ссылки + Терминальные ссылки = Конечный автомат

Позволяет найти сразу множество слов из словаря в большом тексте за время  $O(N)$  совершив всего один проход.

Аналогия 1: Паутина (Бор)

Представь, что мы сплели паутину (Бор) из нужных нам слов. Если нужного продолжения буквы нет — мы падаем по суффиксной ссылке ("нити страховки") в самый длинный из известных суффиксов.

Аналогия 2: Матёшка (Терминальные)

Представь, что мы ищем слова "he" и "she". Если мы прочитали "she", мы одновременно нашли и "he", потому что оно спрятано внутри как матрёшка! Для этого нужны терминальные ссылки.

► Код (Скрыто)

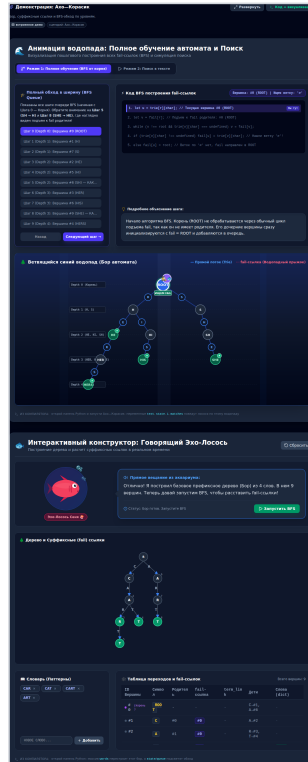
```
int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char c) {
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
    }
    return t[v].go[c];
}
```

## Демонстрация и связь с компилятором

Ахо—Корасик

Бор, суффиксные ссылки и BFS-обход по уровням.



Ахо—Корасик

### Ахо—Корасик: бор, fail-ссылки и автомат

Шаг демонстрации: Шаг = один символ текста: state = go(state, c), затем подъём по fail-ссылкам собирает все образцы-суффиксы. Номера вершин совпадают с картинкой «водопада».

- words — набор образцов — по нему ПЕРЕСТРАИВАЕТСЯ бор в виджете «Эхо-Лосось» (["HERS", "HIS", "SHE", "HE"])
- text — сканируемый текст — подставляется в режим поиска «водопада» (USHERS на демо)
- state — текущее состояние автомата = вершина бора; подсвечивается на картинке (0 ... nodes−1)
- i / c — индекс и сам символ текста, который обрабатывается сейчас (0 ... n−1)
- nxt — переходы бора:  $nxt[(v, ch)] = u$  ( $\leq \Sigma |words|$ )
- fail — fail[v] — вершина наибольшего собственного суффикса (её строит BFS) (0 ... nodes−1)
- term — в каких вершинах заканчиваются образцы (вершина → слово)
- queue — очередь BFS, которой натягиваются fail-ссылки ( $\leq nodes$ )
- matches — найденные вхождения: (индекс конца, слово) ( $\leq n$ )
- nodes — сколько вершин в боре (корень = 0) (10 на демо)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

# Классы сложности, сведение задач

## Классы сложности и сведение задач

### 24.1 Границы возможного

Строгое правило / Формула:

$P$  — решается за полином.  $NP$  — ответ ПРОВЕРЯЕТСЯ за полином (есть короткий сертификат). Сведение  $A \leq_p B$  — преобразовать вход  $A$  ко входу  $B$  за полином, так что ответ сохранится: умеешь решать  $B \rightarrow$  умеешь решать  $A$ .

Класс сложности — это множество задач, решаемых на выбранной модели вычислений (обычно машина Тьюринга) с выбранным бюджетом ресурса (время, память). Важно: классы говорят про задачи, а не про алгоритмы, и про худший случай, а не про «в среднем».

Аналогия 1: Судoku ( $P$  против  $NP$ )

Класс  $P$  — когда алгоритм сам быстро решает пустую доску (сортировка, кратчайший путь,  $MST$ ). Класс  $NP$  — когда решать тяжело, но если тебе дадут заполненную сетку (сертификат), ты за полином проверишь, что там нет нарушений правил. Вопрос « $P = NP?$ » — это вопрос «всякую ли задачу, ответ которой легко проверить, легко и решить?». Открыт с 1971 года и стоит миллион долларов в списке задач института Клэя.

Аналогия 2: Переводчик (сведение)

Сведение  $A$  к  $B$  — ты не говоришь по-китайски (задача  $A$ ), но есть переводчик на английский (сведение за полином) и друг-англичанин (решатель  $B$ ). Спросил друга — перевёл ответ обратно. Вывод:  $B$  не проще  $A$ . Отсюда главный инструмент: если к  $B$  сводится какая-то  $NP$ -полная задача, то и  $B$  —  $NP$ -трудная, и быстрого алгоритма для неё ждать не стоит.

### 24.2 Иерархия классов

Известные включения (все, кроме отмеченных, — нестрогие; строгость хотя бы одного включения в цепочке доказывается теоремой об иерархии времени, так как  $P \subsetneq EXP$ ):

$P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$   $NP$ -полные =  $NP \cap NP$ -трудные ·  $P \subsetneq EXP$  (доказано)

| Класс         | Определение «на пальцах»                                    | Примеры задач                                                                                                    |
|---------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| $P$           | детерминированно за $O(n^k)$                                | сортировка, Дейкстра, Флойд, $MST$ , паросочетание, 2-SAT, проверка двудольности, максимальный поток             |
| $NP$          | сертификат проверяется за полином                           | SAT, 3-SAT, гамильтонов цикл, раскраска в 3 цвета, рюкзак (решающая версия), коммивояжёр «есть ли тур $\leq K$ » |
| co- $NP$      | сертификат есть у ответа «НЕТ»                              | «формула невыполнима», «число составное» (сертификат — делитель)                                                 |
| $NP$ -трудные | не проще любой задачи из $NP$ (к ним сводится всё из $NP$ ) | проблема остановки, раскраска графа, TSP-оптимизация (не решающая версия)                                        |
| $NP$ -полные  | $NP$ -трудные И из $NP$                                     | SAT, 3-SAT, клика, вершинное покрытие, гамильтонов цикл, подмножество сумм                                       |
| $PSPACE$      | полиномиальная память, время не ограничено                  | QBF (кванторная формула), игры с полной информацией на полиномиальном поле                                       |
| $EXP$         | детерминированно за $2^{(n^k)}$                             | обобщённые шахматы/шашки на поле $n \times n$                                                                    |

### 24.3 Сведение: как доказывают NP-полноту

Стандартный рецепт из двух шагов. Шаг 1. Показать, что задача лежит в NP: придумать сертификат полиномиального размера и проверку за полином (для гамильтонова цикла сертификат — сам порядок вершин; проверка — «все вершины по разу и каждая соседняя пара соединена ребром»). Шаг 2. Взять известную NP-полную задачу A и построить полиномиальное сведение  $A \leq_p B$ . Тогда B — NP-трудная, а вместе с шагом 1 — NP-полная.

Точка отсчёта — теорема Кука—Левина (1971/1973): задача выполнимости булевых формул SAT NP-полна. Доказывается «в лоб»: вычисление машины Тьюринга на входе длины n за полиномиальное время можно записать булевой формулой размера  $\text{poly}(n)$ , где переменные — состояние ленты и головки на каждом шаге, а satisfiability этой формулы равносильна принятию входа. Дальше всё катится цепочкой сведений: SAT  $\rightarrow$  3-SAT  $\rightarrow$  клика / вершинное покрытие / независимое множество  $\rightarrow$  гамильтонов цикл  $\rightarrow$  TSP.

Что делать, если задача оказалась NP-полной

- Точный перебор с отсечениями :  $O(2^n \cdot \text{poly})$  — для n до 20–40 это часто приемлемо (динамика по подмножествам, meet-in-the-middle).
- Приближённые алгоритмы : вершинное покрытие — 2-аппроксимация, метрический TSP —  $3/2$  (Кристофидес), жадный set cover —  $\ln n$ .
- Эвристики и метаэвристики : локальный поиск, имитация отжига, генетика — без гарантий, но работают на практике.
- Параметризованная сложность (FPT) : экспонента только по параметру k, например вершинное покрытие за  $O(2^k \cdot n)$ .
- Специальные классы входов : на деревьях, двудольных или планарных графах многие NP-полные задачи становятся полиномиальными (независимое множество на дереве — динамика за  $O(n)$ ).
- Готовые решатели : SAT/ILP-солверы (MiniSat, CaDiCaL, Gurobi) на реальных экземплярах часто быстрее любого своего алгоритма.

Ты путаешь:

- NP — это не «не решается за полином». NP = «проверяется за полином», и  $P \subseteq NP$ : любая задача из P автоматически в NP. Неразрешимых задач в NP нет.
- NP-трудная  $\neq$  NP-полная. NP-трудная может вообще не лежать в NP (проблема остановки — неразрешима, зато NP-трудна). NP-полная = NP-трудная + «сама из NP».
- Направление сведения. Чтобы доказать NP-трудность B, сводят от известной трудной A к B ( $A \leq_p B$ ), а не наоборот. Сведение  $B \leq_p A$  ничего не доказывает про трудность B.
- «NP»  $\neq$  «non-polynomial». N — от nondeterministic, не от «не».
- Экспоненциальный алгоритм не запрещён:  $O(1.5^n)$  или  $O(2^{n/2})$  часто проходит по времени на  $n = 40$ . NP-полнота запрещает только полиномиальный алгоритм при условии  $P \neq NP$ .
- Факторизация чисел лежит в NP и co-NP и не известна как NP-полная — на этом стоит RSA. «Раз она в NP, значит NP-полная» — неверный вывод.

Сложность — явно: проверка сертификата NP-задачи  $O(\text{poly}(n))$ ; наивный перебор сертификатов  $O(2^{\text{poly}(n)})$ ; сведение обязано работать за  $O(\text{poly}(n))$ , иначе доказательство рассыпается.

► Код: верификатор сертификата и пример сведения (Скрыто)

```
# Гамильтонов цикл NP-полон: сертификат = порядок вершин, проверка за  $O(n + m)$ 
def verify_hamilton(n, edges, cert):
    if sorted(cert) != list(range(n)):
        return False
    allowed = {(u, v) for u, v in edges} | {(v, u) for u, v in edges}
    for i in range(n):
        a, b = cert[i], cert[(i + 1) % n]
        if (a, b) not in allowed:
            return False
    return True
```

```

# 3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k вершин, ищем клику размера k
def reduce_3sat_to_clique(clauses):
    # вершина = (номер дизъюнкта, литерал); ребро, если литералы не противоречат
    verts = [(ci, lit) for ci, cl in enumerate(clauses) for lit in cl]
    edges = []
    for i in range(len(verts)):
        for j in range(i + 1, len(verts)):
            ci, li = verts[i]; cj, lj = verts[j]
            if ci == cj: # из одного дизъюнкта брать нельзя
                continue
            if li.lstrip("-") == lj.lstrip("-") and (li[0] == "-" != (lj[0] == "-")):
                continue # x и не-x одновременно — противоречие
            edges.append((i, j))
    return verts, edges, len(clauses) # клика размера k ⇔ формула выполнима

# Проверка задачи из P для контраста: двудольность = 2-раскраска, O(V + E)
def is_bipartite(n, graph):
    from collections import deque
    col = [-1] * n
    for s in range(n):
        if col[s] != -1:
            continue
        col[s] = 0
        dq = deque([s])
        while dq:
            v = dq.popleft()
            for to in graph[v]:
                if col[to] == -1:
                    col[to] = col[v] ^ 1
                    dq.append(to)
                elif col[to] == col[v]:
                    return False
    return True

```

Обрати внимание на разницу: для NP-полной задачи мы умеем быстро проверять готовый ответ, но не находить его; для задачи из P (двудольность) находим и проверяем одинаково быстро.

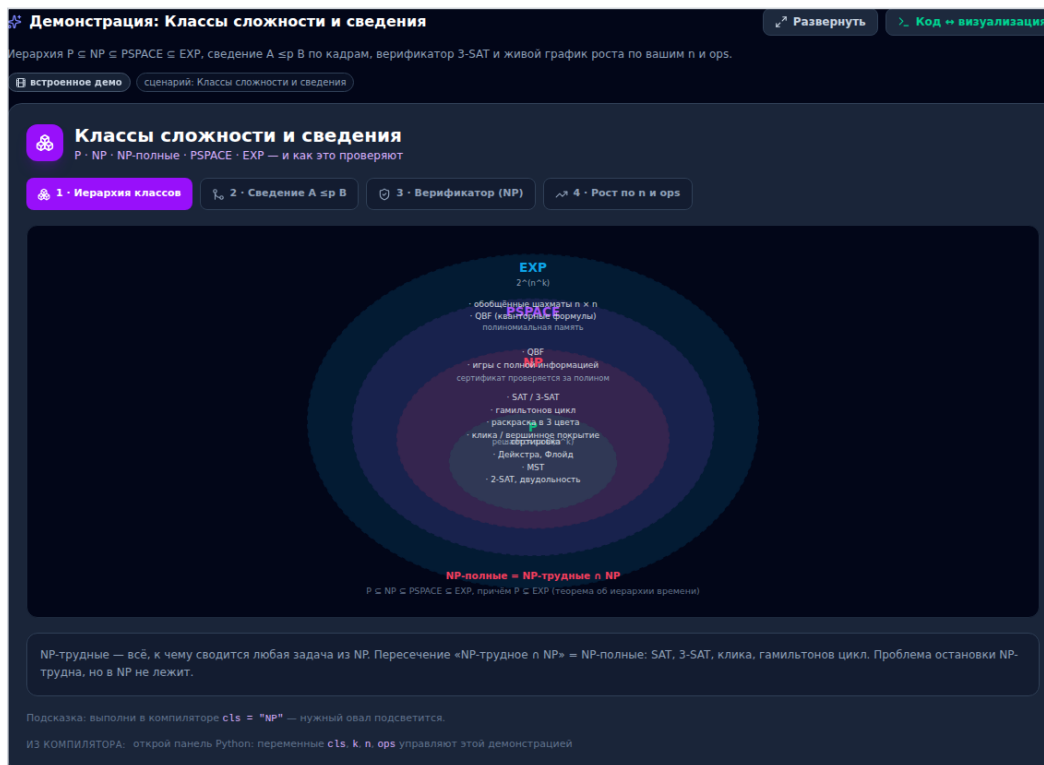
#### ► Вопросы, которые любят задавать (Скрыто)

- Дайте определение NP через верификатор и через недетерминированную машину Тьюринга — и покажите, что они эквивалентны.
- Что такое NP-полная задача и как доказать NP-полноту своей задачи? (сертификат + сведение от известной NP-полной)
- Почему первая NP-полная задача — именно SAT? (теорема Кука—Левина: моделируем машину формулой)
- Если  $P = NP$ , что станет с криптографией и с аппроксимацией? (взлом по сертификату станет полиномиальным; для многих задач исчезнет граница approximability)
- Пример задачи в NP и co-NP, про которую не известно, что она в P? (факторизация / проверка простоты до 2002 года — затем AKS доказал P)
- Чем NP-трудная задача отличается от неразрешимой? (неразрешимая не решается вообще ни за какое время, например проблема остановки)

## Демонстрация и связь с компилятором

Классы сложности и сведения

Иерархия  $P \subseteq NP \subseteq PSPACE \subseteq EXP$ , сведение  $A \leq_p B$  по кадрам, верификатор 3-SAT и живой график роста по вашим  $n$  и  $ops$ .



Классы сложности и сведения

## Классы сложности: рост по $n$ и $ops$

Шаг демонстрации: Шаг = одна точка ( $n$ ,  $ops$ ): визуализация коптит их и оценивает наклон в лог-лог координатах — полином или экспонента.

- $n$  — размер входа — горизонтальная ось графика роста (2 ... 10)
- $ops$  — сколько операций выполнил алгоритм на входе размера  $n$  — вертикальная ось (3 ... 55 на демо (пары  $i \leq j$ ))
- `cls` — имя класса (P / NP / PSPACE / EXP) — подсвечивает овал на схеме иерархии (строка)
- `step / i` — кадр схемы (0 вход → 1 алгоритм → 2 ответ → 3 сертификат) и индекс внутреннего цикла (`step` 0...3, `i` 0...9)
- $k$  — размер сертификата: сколько значений проверяет верификатор (4 на демо)
- `cert` — сертификат — назначение переменных, которое проверяет верификатор ([1, 0, 1, 0])
- `ok` — вердикт верификатора: принят ли сертификат (True/False)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Вкладка «hierarchy» — Иерархия классов $P \subseteq NP \subseteq PSPACE \subseteq EXP$

Шаг демонстрации: Шаг не нужен: схема подсвечивает тот овал, имя которого лежит в переменной `cls`.

- `cls` — имя класса — P, NP, PSPACE или EXP; соответствующий овал подсвечивается (строка)
- $n$  — размер входа задачи, про которую идёт речь ( $\geq 1$ )
- $ops$  — сколько операций потребовалось решить её на входе  $n$  ( $\geq 0$ )
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

## Вкладка «reduce» — Сведение $A \leq_p B$ (3-SAT → КЛИКА)

Шаг демонстрации: Шаг = один кадр схемы сведения: вход  $x \rightarrow$  полиномиальное преобразование  $f \rightarrow$  вход  $y \rightarrow$  решатель  $B \rightarrow$  ответ.

- `step` — номер кадра сведения (0...6) — двигает схему (0 ... 6)
- `cls` — класс задачи  $A$ , которую сводим (NP)
- $n$  — размер входа  $x$ ; после сведения  $|y| = \text{poly}(n)$  ( $\geq 1$ )
- $k$  — размер клики, которую ищет решатель  $B$  (= число дизъюнктов)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «verify» — Верификатор 3-SAT: проверка сертификата

Шаг демонстрации: Шаг = один дизъюнкт: верификатор подставляет сертификат и проверяет, истинен ли хотя бы один литерал.

- cert — сертификат — назначение переменных  $x_1 \dots x_4$ ; подставляется в каждый дизъюнкт ([1, 0, 1, 0])
- k — номер дизъюнкта, который проверяется сейчас (0 ... 3)
- ok — итоговый вердикт: сертификат принят или отвергнут (True/False)
- clauses — сама формула 3-SAT — 4 дизъюнкта по 3 литерала (список кортежей)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.

### Вкладка «growth» — Рост числа операций: полином или экспонента

Шаг демонстрации: Шаг = одна точка (n, ops). Чем больше точек, тем точнее оценка наклона в лог-лог координатах.

- n — размер входа — по нему строится точка на графике (2 ... 10)
- ops — число операций алгоритма на входе размера n (1 ...  $10^5$ )
- i — внутренний счётчик цикла — тоже принимается как номер шага (0 ...)
- Референсный код демонстрации в этот PDF не включён: он открывается в панели Python приложения.